



Bachelorarbeit

Titel der Arbeit // Title of Thesis

Konzeption und Evaluierung hybrider Echtzeit-Rendering-Pipelines zur
selektiven, objektbasierten Stilisierung - Eine empirische Systemanalyse
architektonischer Trade-offs von Forward- und Deferred-Ansätzen in
komplexen 3D-Szenen

Akademischer Abschlussgrad: Grad, Fachrichtung (Abkürzung) // Degree

Bachelor of Science (B.Sc.)

Autorenname, Geburtsort // Name, Place of Birth

Marvin Baumann, Witten

Studiengang // Course of Study

Medieninformatik

Fachbereich // Department

Informatik und Kommunikation

Erstprüferin/Erstprüfer // First Examiner

Prof. Dr. Gregor Lux

Zweitprüferin/Zweitprüfer // Second Examiner

Prof. Dr. Laura Anderle

Abgabedatum // Date of Submission

11.08.2026



Eidesstattliche Versicherung

Baumann, Marvin

Name, Vorname // Name, First Name

Ich versichere hiermit an Eides statt, dass ich die vorliegende Abschlussarbeit mit dem Titel
Konzeption und Evaluierung hybrider Echtzeit-Rendering-Pipelines zur selektiven,
objektbasierten Stilisierung - Eine empirische Systemanalyse architektonischer Trade-offs von
Forward- und Deferred-Ansätzen in komplexen 3D-Szenen
selbstständig und ohne unzulässige fremde Hilfe erbracht habe. Ich habe keine anderen
als die angegebenen Quellen und Hilfsmittel benutzt sowie wörtliche und sinngemäße
Zitate kenntlich gemacht. Die Arbeit hat in gleicher oder ähnlicher Form noch keiner
Prüfungsbehörde vorgelegen.

Ort, Datum, Unterschrift // Place, Date, Signature

Kurzfassung

Die selektive Anwendung verschiedener Stile des Non-Photorealistic Rendering (NPR) führt in komplexen 3D-Szenen zu einer hohen stilistischen Heterogenität. Diese Material-Entropie kollidiert fundamental mit etablierten Echtzeit-Rendering-Architekturen, die strukturell auf homogene Beleuchtungsberechnungen optimiert sind. Es fehlen grundlegende empirische Systeme zur Messung der genauen hardwaretechnischen Limitierungen der jeweiligen Basis-Rendering-Paradigmen in diesem Kontext.

Die vorliegende Arbeit liefert eine holistische Systemanalyse zur Quantifizierung dieses Konflikts. Mithilfe des entwickelten Evaluationsframeworks Hybrid Deferred Rendering Architecture (HyDra) wurden die Basis-Architekturen in Form von Single-Pass-Forward, Single-Pass-Deferred (mittels monolithischer Shader-Struktur) und Multi-Pass-Deferred (mittels Light-Volumes) isoliert. Die anschließende Evaluation erfolgte unter kontrollierter Skalierung von Geometrie, Beleuchtung und Stil-Entropie.

Die Messungen belegen, dass für dieses Szenario keine universell überlegene Architektur existiert. Bei hoher Lichtkomplexität kollabiert die Forward-Pipeline unter multiplikativer Rechenlast und bestimmte Formen der selektiven NPR-Stilisierung sind nicht anwendbar. Der Single-Pass-Deferred-Ansatz skaliert effizient unter komplexen Beleuchtungsszenarien, verzeichnet jedoch bei hoher Stil-Entropie massive Leistungseinbrüche durch Warp-Divergenz und Registerdruck. Die Multi-Pass-Deferred-Architektur löst diese Probleme, sättigt jedoch bei überlappenden Lichtquellen durch übermäßige Read-Modify-Write (RMW)-Zyklen die Video Random Access Memory (VRAM)-Bandbreite der Grafikkarte.

Aus der Synthese dieser Architektur- und Hardware-Flaschenhälse wurde ein heuristisches Entscheidungsmodell abgeleitet. Dieses stellt Softwareentwicklern, neben der entwickelten Evaluationssoftware selbst, ein objektives Werkzeug zur frühzeitigen, szenenspezifischen Architekturbewertung im Kontext selektiver Stilisierung zur Verfügung.

Inhaltsverzeichnis

Kurzfassung	I
Abbildungsverzeichnis	IV
Tabellenverzeichnis	VI
List of Listings	VII
Abkürzungsverzeichnis	VIII
1 Einführung	1
1.1 Motivation & Problemstellung	1
1.2 Ziel der Arbeit & Forschungsfrage	3
1.3 Nutzung Generativer KI	4
1.4 Aufbau der Arbeit	4
2 Grundlagen	5
2.1 Rendering-Paradigmen	5
2.1.1 Forward Rendering	5
2.1.2 Deferred Rendering	6
2.2 Hardware & Performance	8
2.2.1 Overdraw	9
2.2.2 Speicherbandbreite	9
2.2.3 Registerdruck & Warp-Divergence	10
3 Stand der Technik und verwandte Arbeiten	12
3.1 Klassische Rendering-Architekturen und deren Implikationen für das NPR	12
3.2 Rendering-Pipelines und Stilisierung in der Industriepraxis	14
3.3 Limitierungen bestehender Ansätze und Identifikation der Forschungslücke	17
4 Konzeption und Realisierung	19
4.1 Systemarchitektur und Lebenszyklus	19
4.1.1 Frame-Lebenszyklus	19
4.1.2 Szenenverwaltung	22
4.1.3 Datenfluss der Stilisierung und G-Buffer Layout	23
4.2 Pipeline-Architekturen und systematische Restriktionen	24
4.2.1 Forward-Rendering-Pipeline	24

4.2.2	Deferred-Uber Rendering Pipeline	26
4.2.3	Deferred-Volume Rendering Pipeline	28
4.3	NPR-Payloads	30
4.4	Telemetrie & Benchmark-Orchestrierung	33
4.4.1	Szenen- & Zustandsparität	33
4.4.2	Asynchrone Hardware-Telemetrie	34
4.4.3	Engine-Determinismus und Frame-Locking	36
5	Evaluationen	38
5.1	Evaluationsmethodik & Versuchsaufbau	38
5.1.1	Zielsetzung und Hypothesen der Untersuchung	38
5.1.2	Systemkonfiguration und Variablen-Mapping	40
5.1.3	Versuchsablauf	40
5.2	Geometrische Dichte und Overdraw	41
5.2.1	Level of Detail (Mikro-Geometrie)	41
5.2.2	Objektanzahl	43
5.2.3	Overdraw-Dichte	44
5.3	Bandbreite und Auflösung	45
5.3.1	Auflösung	45
5.3.2	Bandbreiten	47
5.4	Beleuchtung	49
5.4.1	Lichtanzahl	49
5.4.2	Lichtintensität & Overlap	51
5.4.3	Shading Overdraw	53
5.5	Hybride Stilisierung & Entropie	54
5.5.1	Stil-Entropie	55
5.5.2	Kernel Bandbreiten Limits	57
5.6	Makro-Benchmark-Synthese	59
6	Diskussion	63
6.1	Zusammenführung und Interpretation der Ergebnisse	63
6.2	Heuristisches Entscheidungsmodell	65
6.3	Kritische Reflexion und Limitationen	68
6.4	Lessons Learned	69
7	Zusammenfassung und Ausblick	71
7.1	Fazit der Arbeit	71
7.2	Ausblick und zukünftige Arbeiten	72
Literatur		i
A Systemkonfiguration und Variablen-Mapping		v
B Struktur des digitalen Anhangs		ix

Abbildungsverzeichnis

1.1	Anwendungsbeispiel für Cel-Shading in <i>Borderlands 4</i>	1
2.1	UML-Aktivitätsdiagramm der Single-Pass-Forward-Rendering-Pipeline . .	6
2.2	UML-Aktivitätsdiagramm des Deferred-Rendering-Paradigmas	7
2.3	UML-Aktivitätsdiagramm der Multi-Pass-Architektur für Deferred-Rendering mittels Light-Volumes	8
3.1	Speicherlayout der Hi-Fi Rush G-Buffer	15
3.2	Visualisierung der Pipeline-Fragmentierung in <i>PRAGMATA</i>	16
3.3	Screenshot aus dem HyDra-Framework	18
4.1	UML-Aktivitätsdiagramm des Frame-Lebenszyklus	21
4.2	Toon-Shading und Kontureffekt-Vergleich in HyDra-Framework	26
4.3	Visueller Vergleich von Pre-Lit und Post-Lit Kuwahara in HyDra-Framework	28
4.4	Vergleich der Lichtakkumulation in der Deferred-Volume-Pipeline	29
4.5	Auswirkung des Framebuffer-Präzisionsformats auf den Kuwahara-Filter .	29
4.6	Gooch-Shading in HyDra-Framework	31
4.7	Kollabieren der selektiven Kuwahara-Bildraumfilterung in HyDra-Framework	32
4.8	UML-Aktivitätsdiagramm der Architektur des Benchmark-Lebenszyklus .	37
5.1	Messung der GPU- und CPU-Laufzeiten bei LOD-Skalierung	42
5.2	Messung von Logik- und Dispatch-Laufzeiten bei Instanzenskalierung . .	44
5.3	Messung der GPU-Ausführungszeit bei Overdraw-Skalierung	45
5.4	Messung der GPU-Ausführungszeit bei Auflösungsskalierung	46
5.5	Vergleich der Basis-Bandbreitenkosten bei 1080p	48
5.6	Vergleich der Basis-Bandbreitenkosten bei 4K	48
5.7	Messung der GPU-Ausführungszeit bei Lichtanzahlskalierung	50
5.8	Messung der GPU-Ausführungszeit bei Lichtintensitätsskalierung	52
5.9	Messung der GPU-Ausführungszeit bei Lichtquellen-Singularität	53
5.10	Entkopplungsnachweis in steigendem geometrischen Overdraw	54
5.11	Messung der GPU-Shading-Zeit bei variierender Stilverteilung	56
5.12	Vergleich der Stil-Entropie-Verteilungen in HyDra-Framework	56
5.13	Messung der GPU-Shading-Zeit bei stilistischer Kombinatorik	57
5.14	Messung der GPU-Shading-Zeit bei Kuwahara-Radius-Skalierung	58
5.15	Akkumulierte GPU-Ausführungszeit im Makro-Benchmark	61
5.16	Frame-Zeit-Verlauf des Kamera-Flythroughs in Makro-Benchmark	61

6.1	Heuristisches Entscheidungsmodell der Rendering-Architekturen	67
-----	---	----

Tabellenverzeichnis

4.1	Spezifikation des G-Buffer-Layouts im HyDra-Framework.	24
4.2	Shading-Effekte und NPR-Stile in HyDra-Framework	30
A.1	Spezifikation der primären Test-Hardware.	v
A.2	Spezifikation der Software-Umgebung und API-Schnittstellen.	v
A.3	Spezifikation der unabhängigen Variablen, deren formale Definition und Zuweisung zu den Test-Suites.	vi
A.4	Spezifikation der abhängigen Variablen (Metriken), deren formale Definition und CSV-Mapping.	vii
A.5	Spezifikation der Kontrollvariablen und der statischen System-Baseline. .	viii

List of Listings

1	C++ Code zur Berechnung der Style-ID-Verteilung in HyDra-Framework . .	22
2	GLSL-Code des Deferred-Uber Beleuchtungspasses in HyDra-Framework .	27
3	GLSL-Code zur Optimierung des Kuwahara-Filter-Samplings	33
4	C++-Code des EngineState in HyDra-Framework	34
5	C++-Code zum Notfall-Stall-Verfahren in HyDra-Framework	35

Abkürzungsverzeichnis

ALU	Arithmetic Logic Unit	9
API	Application Programming Interface	25
CAD	Computer-Aided Design	2
CPU	Central Processing Unit	19
FBO	Framebuffer Object	6
G-Buffer	Geometry Buffer	2
GPU	Graphics Processing Unit	8
HDR	High Dynamic Range	29
HyDra	Hybrid Deferred Rendering Architecture	1
LOD	Level of Detail	
MRT	Multiple Render Targets	10
MSAA	Multisample Anti-Aliasing	13
MVP	Model-View-Projection	43
NPR	Non-Photorealistic Rendering	1
PCIe	Peripheral Component Interconnect Express	20
PBR	Physically Based Rendering	2
RMW	Read-Modify-Write	1
ROP	Raster Operation Pipeline	8
RT	Render Target	14
SoA	Structure of Arrays	22
SIMD	Single Instruction, Multiple Data	11
SIMT	Single Instruction, Multiple Threads	11
TAA	Temporal Anti-Aliasing	13
UBO	Uniform Buffer Object	24
V-Buffer	Visibility Buffer	13
VAO	Vertex Array Object	19
VBO	Vertex Buffer Object	19
VRAM	Video Random Access Memory	1

1 Einführung

Die stilisierte Darstellung komplexer 3D-Szenen und einzelner Objekte darin mittels NPR ist in der modernen Computergrafik sowohl für technische als auch für unterhaltende Anwendungen von großer Bedeutung. Eine gezielte Stilisierung erhöht die visuelle Lesbarkeit, lenkt den Fokus des Betrachters und ermöglicht künstlerische Abstraktionen (siehe 1.1). Insbesondere die individuelle Praxis großer Spieleproduktionen sieht sich mit der Notwendigkeit konfrontiert, stetig wachsende Datenmengen und Szenenkomplexitäten in Echtzeit bewältigen zu müssen. Die effiziente Bewältigung der resultierenden Performance-Engpässe erfordert hochskalierbare Rendering-Architekturen. Die Gewährleistung dieser Skalierbarkeit erweist sich im Kontext der für NPR oft erforderlichen Stil-Heterogenität jedoch als komplexe Problemstellung und offenbart einen fundamentalen Konflikt innerhalb etablierter Rendering-Paradigmen. [1, Abschn. 1]



Abbildung 1.1: Anwendungsbeispiel für künstlerisches NPR mittels Cel-Shading am Beispiel von *Borderlands 4* (2025) [2].

1.1 Motivation & Problemstellung

NPR-Effekte werden primär mit künstlerischen Umsetzungen und stilisierten Darstellungen in der Unterhaltungsindustrie, wie etwa in Videospielen oder Filmproduktionen, in Verbindung gebracht. Die gezielte Stilisierung ganzer Szenen oder einzelner Objekte zur Schaffung

eines bestimmten künstlerischen Kontextes bildet hierbei einen der Hauptanwendungszwecke von modernem NPR [3, S.651–652]. Darüber hinaus finden diese Techniken jedoch auch im Bereich von technischen Illustrationen, Computer-Aided Design (CAD)-Anwendungen oder medizinischen Visualisierungen weitreichende Anwendung. In diesen Fachbereichen erfüllt die Stilisierung oftmals den funktionalen Zweck, durch eine simplere oder abstraktere Darstellung die allgemeine Lesbarkeit der Szene zu erhöhen oder den Fokus gezielt auf kritische Daten und spezifische Elemente zu lenken [4, Abschn. 2]. Eine derartige funktionale Hervorhebung durch Simplifizierung ist mit gängigem Physically Based Rendering (PBR) in dieser Form oft nicht möglich, da rein fotorealistische Ansätze visuell häufig zu detailreich sind und den Blick auf das Wesentliche verstellen [1, Abschn. 1].

Moderne Echtzeit-Rendering-Pipelines müssen enorme Mengen an 3D-Geometriedaten verarbeiten und simultan hochkomplexe Beleuchtungsszenarien aus zahlreichen Lichtquellen berechnen. Das *Deferred Rendering* entkoppelt die Geometrieverarbeitung strikt von der Lichtauswertung und stellt ein etabliertes Architektur-Paradigma zur effizienten Bewältigung hoher Szenen- und Lichtkomplexität dar, da es in komplexen Szenarien eine drastische Reduzierung des Rechenaufwands ermöglicht. Aufgrund dieses inhärenten Aufbaus weisen klassische Deferred-Pipelines jedoch einen starken architektonischen Nachteil auf, da sie die darstellbare Materialvielfalt stark limitieren. Infolge dieser technischen Restriktionen wurden Deferred-Paradigmen historisch primär für die Auswertung homogener, fotorealistischer Oberflächen (PBR) konzipiert und optimiert [5, Abschn. 1].

Deferred Rendering projiziert die dreidimensionale Szenengeometrie in einen zweidimensionalen Geometry Buffer (G-Buffer). Bei dieser Überführung in den *Screen-Space* geht die geometrische Identität der Objekte, also semantische Informationen wie etwa die Zugehörigkeit zu einem spezifischen 3D-Modell, weitestgehend verloren. Stattdessen werden in klassischen Pipelines lediglich physikalische Oberflächendaten für standardisierte Lichtberechnungen (PBR), wie etwa Albedo, Normalen oder Rauheit, gespeichert. Viele NPR-Effekte erfordern für eine fehlerfreie Ausführung jedoch zwingend einen gewissen Grad an semantischen Informationen [6, Abschn. 2.1 u. 3.3.2]. Die Zuweisung verschiedener Zeichenstile wie *Toon-Shading* oder die exakte Trennung überlappender Objekte durch Umrisslinien benötigen beispielsweise spezifische Material-Identifikatoren (*Shading-Model-IDs*), oder Objekt-IDs [3, Kap. 15.2.3], [7, Folie *Our GBuffer Layout*]. Diese Metadaten fehlen in einem klassischen, auf PBR-optimierten G-Buffer. Dieser Umstand führt dazu, dass NPR in Deferred-Architekturen häufig zu einem rein globalen Post-Processing-Filter über das gesamte Bild verkommt [1, Abschn. 1]. Eine granulare, selektive Stilisierung (z. B. Bauteil A als Skizze, Bauteil B massiv schattiert) wird dadurch extrem erschwert. Bei hoher stilistischer Vielfalt und sehr selektiven Anwendungsfällen wird daher oft auf klassisches *Forward-Rendering* zurückgegriffen. Dieses erweist sich wiederum als stark ineffizient bei hoher Beleuchtungskomplexität [5, Abschn. 1].

Die Kombination aus massenhafter Geometrie, Beleuchtungskomplexität und stilistischer Vielfalt zwingt in der Praxis gängige Rendering-Paradigmen an ihre individuellen hardware-

technischen Leistungsgrenzen. Dabei offenbart sich ein grundlegender architektonischer Konflikt: Architekturen leiden bei der Stilisierung vieler Objekte entweder unter massiven redundanten Zeichenoperationen bei überlappender Geometrie [8, Abschn. 1], oder die Auswertung der diversen Stildaten überlastet die Speicher- und Rechenkapazitäten der Grafikkhardware [9, Abschn. 6]. Trotz der Relevanz dieses Problems mangelt es an systematischen, empirischen Erkenntnissen darüber, unter welchen genauen Bedingungen und Skalierungen diese spezifischen Leistungseinbrüche im Kontext des NPR auftreten. Aus dieser Notwendigkeit leitet sich die direkte Zielsetzung der vorliegenden Arbeit ab.

1.2 Ziel der Arbeit & Forschungsfrage

Das primäre Ziel dieser Arbeit ist die Durchführung einer holistischen, quantitativen Systemanalyse verschiedener Rendering-Architekturen im Kontext der selektiven Stilisierung mittels NPR, mit einem speziellen Fokus auf das in diesem Bereich oft vernachlässigte Deferred-Rendering. Das angestrebte wissenschaftliche Resultat dieser Untersuchung ist die Erstellung eines empirisch hergeleiteten, heuristischen Entscheidungsmodells zur Evaluation elementarer Rendering-Paradigmen. Die untersuchten Basis-Pipelines umfassen dabei ein Single-Pass-Forward-Rendering als Referenz, ein Single-Pass-Deferred-Rendering mit einer monolithischen Shader-Struktur sowie einen Multi-Pass-Deferred-Ansatz mit volumetrischer Lichtauswertung (*Light Volumes*). Das resultierende Modell sowie das zugrunde liegende Evaluations-Framework adressieren einen primären ingenieurstechnischen Nutzen: Sie bieten Entwicklern eine objektive Grundlage, um anhand der Variablen Szenenkomplexität und Stil-Entropie die geeignete Pipeline-Architektur zu evaluieren, noch bevor in der Softwareproduktion architektonische Performance-Engpässe entstehen. Darüber hinaus fungiert die entwickelte Architektur als isolierte Referenzumgebung, um das Basisverhalten und die hardwaretechnischen Leistungsgrenzen dieser fundamentalen Paradigmen unabhängig zu quantifizieren.

Aus diesen Zielen leitet sich die zentrale Forschungsfrage der vorliegenden Arbeit ab, die im Rahmen der Systemanalyse empirisch untersucht und beantwortet wird:

Wie lassen sich Non-Photorealistic Rendering-Pipelines architektonisch so konzipieren und umsetzen, dass eine selektive, objektbasierte Stilisierung unter hoher Entropie ermöglicht wird, und wie verhält sich dabei die Gesamteffizienz der Pipeline im Vergleich zur Szenenkomplexität? Welche Aspekte bilden hier die individuellen Performance-Engpässe und Limitierungen und wie verschieben sich diese zwischen den verschiedenen Rendering-Pipelines?

Das C++/OpenGL-basierte Evaluations-Framework HyDra wird im Rahmen dieser Arbeit als wissenschaftliches Artefakt realisiert, um eine objektive und kontrollierte Testumgebung für die Beantwortung der architektonischen Fragestellungen bereitzustellen. Innerhalb dieses Frameworks werden die drei zuvor genannten Basis-Pipelines isoliert umgesetzt.

Dieser methodische Aufbau ermöglicht die Generierung deterministischer Messreihen unter systematischer Skalierung der Variablen der Geometrie, Beleuchtung und Stil-Entropie, ohne den Overhead proprietärer Engine-Architekturen.

Die Implementierung und Evaluation komplexer Sondereffekte, wie etwa die Verarbeitung semitransparenter Geometrie (Transluzenzen) oder *Anti-Aliasing*, sind nicht Teil dieser Untersuchung, da sie den Rahmen der grundlegenden Analyse der Basis-Rendering-Architekturen sprengen würden.

1.3 Nutzung Generativer KI

Im Rahmen der Erstellung dieser Ausarbeitung wurden generative KI-Werkzeuge verwendet und als unterstützende Hilfsmittel herangezogen, namentlich: *Google-Antigravity* und *Google-Gemini*. Diese Systeme wurden primär zur methodischen Strukturierung einzelner Kapitel, zur Erstellung von UML-Aktivitätsdiagrammen in LaTeX, zur erweiterten Recherche, zur Code-Analyse bei der Fehlerbehandlung sowie für die Generierung von Boilerplate-Code und Python-Skripten für die Visualisierung der Benchmark-Rohdaten eingesetzt. Gemäß den Vorgaben des §14a der Rahmenprüfungsordnung sind die vollständigen, inhaltlich relevanten Chat-Verläufe, bestehend aus den verwendeten Prompts und den generierten Antworten der KI-Modelle, im digitalen Anhang dieser Arbeit dokumentiert.

1.4 Aufbau der Arbeit

Die vorliegende Arbeit gliedert sich wie folgt, um eine systematische Beantwortung der Forschungsfragen zu gewährleisten: Kapitel 2 vermittelt zunächst die architektonischen und hardwaretechnischen Grundlagen sowie relevante technische Begrifflichkeiten, die für das Verständnis der weiteren Untersuchungen erforderlich sind. Kapitel 3 analysiert im Anschluss den aktuellen Stand der Forschung sowie der Industriepraxis und grenzt darauf aufbauend die bestehende Forschungslücke ab. Kapitel 4 dokumentiert die Konzeption und softwaretechnische Realisierung des entwickelten Evaluations-Frameworks sowie etwaige praktische und architektonische Grenzen der verschiedenen Rendering-Architekturen. Kapitel 5 präsentiert die Methodik der durchgeführten Versuchsreihen und wertet die empirischen Messdaten der untersuchten Rendering-Pipelines aus. Kapitel 6 und Kapitel 7 führen die gewonnenen Erkenntnisse abschließend in einem heuristischen Entscheidungsmodell zusammen, diskutieren die Limitationen der Arbeit und schließen diese mit einem Ausblick auf künftige Forschungsmöglichkeiten und Erweiterungen der Evaluationssoftware ab.

2 Grundlagen

In diesem Kapitel werden die theoretischen Grundlagen der untersuchten Rendering-Architekturen und der verwendeten Verfahren definiert und erörtert. Hierzu zählen die technischen Grundlagen der Rendering-Paradigmen sowie die Spezifikation der daraus resultierenden relevanten Hardware-Flaschenhälse.

2.1 Rendering-Paradigmen

Im Folgenden werden die formalen Definitionen und die Gegenüberstellung von klassischen und entkoppelten Render-Pfaden unter Berücksichtigung ihrer inhärenten Skalierbarkeit erörtert.

2.1.1 Forward Rendering

Single-Pass-Forward Rendering zeichnet sich durch das Iterieren über einzelne Geometrien und Bearbeiten von Shading-Operationen pro Objekt aus. Es erfolgt zunächst eine Evaluierung aller Lichter, die ein zu zeichnendes Objekt in einer Szene beeinflussen. In einer weiteren Schleife werden diese Lichter anschließend für das jeweilige Objekt zusammen mit etwaigen anderen Materialeigenschaften verrechnet und angewendet [10, Folie *Forward Rendering – Single Pass*] (vgl. 2.1). Die Beleuchtungsberechnung finden also untrennbar gekoppelt im selben Render-Pass (Fragment-Shader) statt.

Dieser Umstand führt zu einem grundlegenden theoretischen Aufwand des Forward-Renderings, der sich proportional zur Anzahl der Geometrien und der einfallenden Lichtquellen verhält (quadratische Komplexität). [11, Abschn. 1].

$$O(N_{\text{obj}} \cdot N_{\text{light}}) \quad (2.1)$$

Wobei N_{obj} die Objektanzahl und N_{light} die Anzahl der aktiven, Lichtquellen definiert.

Auf Hardware-Ebene manifestiert sich diese Komplexität in der Rasterisierung. Da die Beleuchtungsschleife für jedes erzeugte Fragment ausgeführt wird, skaliert die Rechenlast massiv mit der Pixelfläche der Objekte [5, Abschn. 3.2]. Verdeckt ein nachfolgend

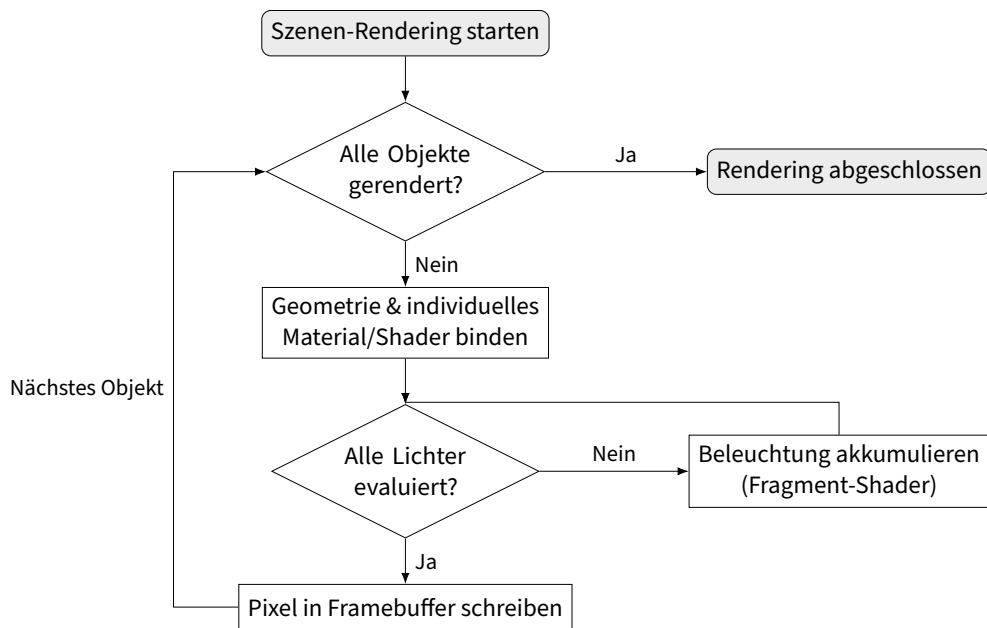


Abbildung 2.1: UML-Aktivitätsdiagramm der Single-Pass-Forward-Rendering-Pipeline. Die direkte Verschachtelung der Objekt- und Lichtevaluierung zeigt die inhärente quadratische Komplexität.

gezeichnetes Objekt ein bereits berechnetes Fragment, war die vorherige, rechenintensive Lichtauswertung obsolet. Dieses ineffiziente Verhalten wird als *Shading-Overdraw* bezeichnet und in Kapitel 2.2.1 detailliert beleuchtet.

Die direkte Kopplung von Geometrie und Material bietet in der Praxis jedoch auch diverse Vorteile. Jedes Objekt kann ohne aufwändige Workarounds seinen eigenen hochspezialisierten Shader binden [5, Abschn. 1]. Dies ist insbesondere vor dem Hintergrund der Evaluationen dieser Arbeit bezüglich selektiver Stilisierungen mittels NPR-Effekten von Relevanz.

2.1.2 Deferred Rendering

Deferred-Rendering zeichnet sich im Gegensatz zu Forward-Rendering durch die eindeutige Trennung von Geometrieverarbeitung und dem Shading der Geometrie aus. Die Geometrien sowie alle damit assoziierten Werte, wie etwa Normalen (Ausrichtungen), Albedos (Farbinformationen) oder Tiefeninformationen, werden in einem eigenständigen, separaten Pass verarbeitet und in einen Zwischenspeicher, dem sogenannten G-Buffer, in Form eines Framebuffer Object (FBO) geschrieben (bildschirmfüllendes Quad). In diesem Pass werden explizit keine Shading-Operationen, wie etwa die Beleuchtung der Objekte, durchgeführt. Erst in einem nachgelagerten Pass werden schließlich die Information aus dem G-Buffer verwendet, um die Beleuchtung und sonstige Materialeigenschaften auf Bildschirm-Ebene (Screen-Space) auszuwerten und zu berechnen (vgl. 2.2). [11, Abschn. 2], [3, Kap. 20.1], [10, Folie *Deferred Rendering*]

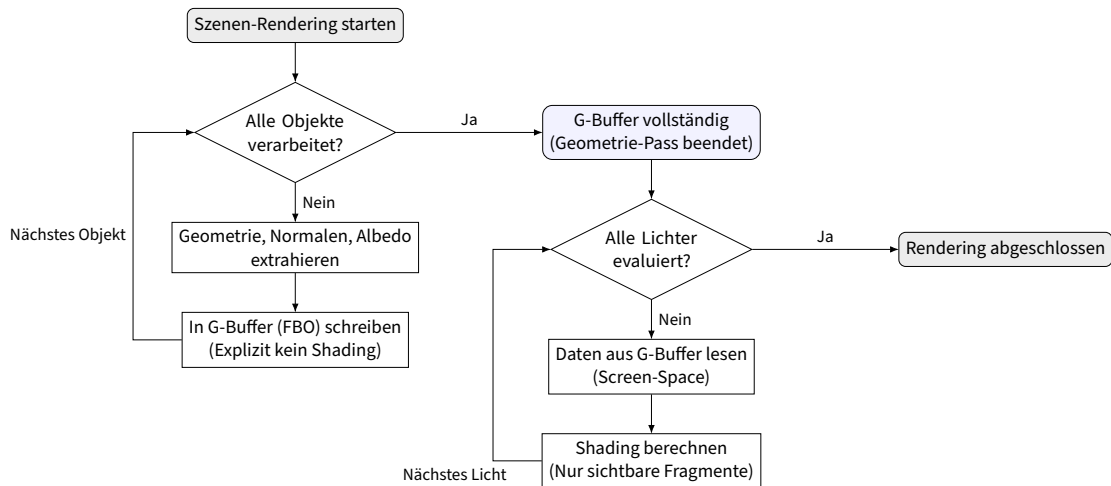


Abbildung 2.2: UML-Aktivitätsdiagramm des Deferred-Rendering-Paradigmas. Die algorithmische Entkopplung von Geometrieverarbeitung (linker Pfad) und Beleuchtungsberechnung (mittlerer Pfad) zeigt die inhärent lineare Komplexität. Shading-Operationen erfolgen ausschließlich im Screen-Space für sichtbare Fragmente.

Algorithmisch bedingt die additive Entkopplung demnach einen linearen, theoretischen Aufwand für Deferred-Rendering-Paradigmen [6, Abschn. 3].

$$O(N_{\text{obj}} + N_{\text{light}}) \quad (2.2)$$

Hierbei definiert N_{obj} die Anzahl der Objekte während der initialen Geometriephase (G-Buffer-Pass) und N_{light} die Anzahl der aktiven Lichtquellen. Die Shading-Operationen werden folglich nur für die tatsächlich sichtbaren Fragmente (Pixel) berechnet.

Werden Deferred-Ansätze nur in einem einzigen, dem Geometrie-Pass nachgelagerten, Pass umgesetzt, kommt es bei der Umsetzung unterschiedlicher Materialeigenschaften und möglichen Stilisierungen zwangsweise zu einer monolithischen Shader-Struktur im finalen Fragment-Shader und sogenannten *Uber-Shadern* [3, S. 881], [11, Abschn. 2.6], [9, Abschn. 6]. Für die in dieser Arbeit implementierten Single-Pass Deferred-Pipeline (im weiteren Verlauf *Deferred-Uber* genannt) ist dieser Umstand vor dem Hintergrund der selektiven stilisierung mit NPR-Effekten besonders bedeutend.

Ein erweiterter und gängiger Ansatz der Implementierung von Deferred-Pipelines stellt eine Multi-Pass Shading Architektur unter der verwendung von *Light-Volumes* dar [12, Abschn. 2], [13, Folie "Conventional Deferred Shading"]. In diesem Ansatz werden die Lichtquellen um spezielle Proxy-Geometrien erweitert, die jeweils den Wirkungsbereich dieser Lichtquellen darstellen [11, Abschn. 2.2]. Für diesen Ansatz muss das eigentliche Shading in zwei separate Pässe aufgeteilt werden. Nach dem initialen Geometrie-Pass, in dem der G-Buffer ohne die entsprechenden Proxy-Geometrien beschrieben wurde, werden die Light-Volumes in einem separaten Lighting-Pass rasterisiert [11, Abschn. 2], [3, S. 884–886]. Innerhalb dieses Prozesses werden ausschließlich die Beleuchtungseinflüsse für die

betroffenen Fragmente durch das Auslesen des G-Buffers berechnet und in ein separates FBO („Lighting-Buffer“) geschrieben [6, Abschn. 3.1]. Die Einflüsse etwaiger überlappender Light-Volumes werden durch automatisierte additive *Blending*-Operationen auf den speziell dafür ausgelegten Raster Operation Pipelines (ROPs) der Graphics Processing Unit (GPU) akkumuliert [14, Kap. 11.2.4.10] (vergleiche Kap. 2.2.2). In einem finalen *Resolve*-Pass werden schließlich die Basis-Materialeigenschaften (wie etwa Farbinformationen) aus dem G-Buffer mit der akkumulierten Beleuchtung kombiniert und um finale Shading- oder Post-Processing-Effekte ergänzt [11, Abschn. 2.3] (siehe Abb. 2.3).

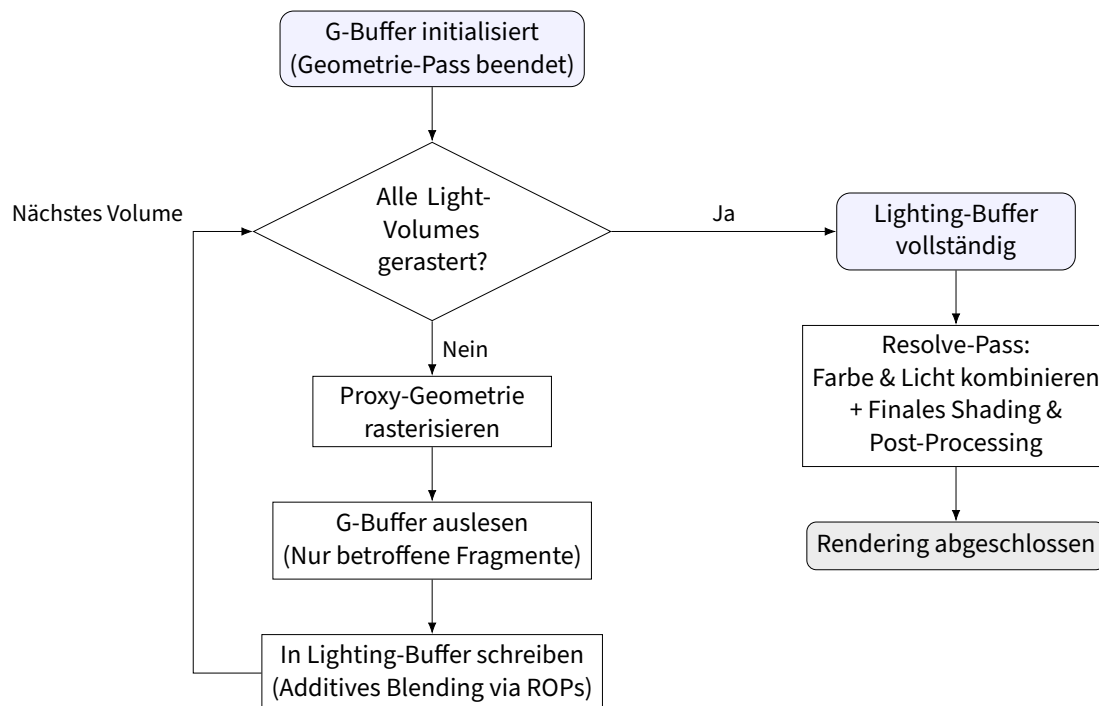


Abbildung 2.3: UML-Aktivitätsdiagramm der Multi-Pass-Architektur für Deferred-Rendering mittels Light-Volumes. Im Lighting-Pass (linker Pfad) werden Proxy-Geometrien gerastert und überlappende Einflüsse per Hardware-Blending in einem separaten FBO akkumuliert. Der Resolve-Pass (rechter Pfad) kombiniert diese Daten anschließend mit den initialen Materialinformationen.

2.2 Hardware & Performance

Die Evaluation der in dieser Arbeit betrachteten Architekturen und Paradigmen erfordert die Einordnung und Kontextualisierung einiger zentraler Phänomene und Flaschenhälse auf Hardware-Ebene der GPU. Im Folgenden werden die drei für diese Arbeit relevantesten Phänomene erörtert.

2.2.1 Overdraw

Wie in Kapitel 2.1.1 skizziert leiden vor allem klassische Forward-Rendering-Pipelines unter dem Phänomen des *Fragment-Overdraws* [11, Abschn. 1.5]. Dieses Phänomen beschreibt die Eigenschaft GPU-Seitig unnötige Shading-Operationen auf Objekten auszuführen, die im späteren Verlauf von anderen Objekten teilweise oder gänzlich verdeckt und im Framebuffer überschrieben werden [3, S. 801–802], [15, S. 55]. Dies führt zu einem massiven Konsum der verfügbaren Füllrate (*fill rate*) und resultiert in modernen Architekturen primär in Leistungsengpässen durch limitierte Speicherbandbreite und unnötige Fragment-Shader-Auslastung [3, S. 788 und 801–802].

Die Kompensation von Overdraw erfolgt in erster Linie durch zwei vorherrschende Prinzipien: das automatisierte *Early-Z* auf GPU-Hardwareebene und *Z-Prepass*-Algorithmen. Wie Gregory [14, Kap. 11.2.4.8] beschreiben sind Early-Z-Tests direkt auf der Hardware moderner GPUs verankerte Tiefen-Tests auf Fragmentebene, um diese zu verwerfen, sollte bereits ein übergelagerter Pixel an dieser Stelle im Framebuffer existieren. Die Pixel-Shader-Ebene (Fragment-Shader) wird somit für das entsprechende Fragment übersprungen.

Eine weitere gängige Praxis zur Eliminierung von Overdraw ist die Implementierung eines Z-Prepass-Algorithmus. Ein klassischer Z-Prepass-Algorithmus ist in zwei Phasen unterteilt. In der initialen Phase erfolgt die Erstellung eines *Z-Buffers*. Im Rahmen dieses Prozesses werden opake Geometrien von vorne nach hinten (Front-to-Back) gerendert, wobei ausschließlich die Tiefenwerte der Fragmente in den Z-Buffer geschrieben werden. In der zweiten Phase erfolgt schließlich das vollständige Shading der sichtbaren Fragmente, das sich am erstellten Z-Buffer orientiert. [14, Kap. 11.2, Abschn. *z-Prepass to the Rescue*] Gemäß dieser Prämisse durchlaufen sämtliche Objekte für dieses Verfahren die Geometrie-Pipeline (Vertex-Shader) zweimal, was eine zusätzliche Belastung der Performance zur Folge hat.

2.2.2 Speicherbandbreite

Eine signifikante Einschränkung in der Architektur von GPUs stellt die Speicherbandbreite (Memory Bandwidth) dar. Sie definiert die maximale Übertragungsgeschwindigkeit von Daten zwischen dem dedizierten VRAM und den eigentlichen Rechenkernen der GPU [3, S. 30 u. S. 791]. Der Datenbus zwischen VRAM und GPU weist eine harte Kapazitätsgrenze auf, die in Gigabyte pro Sekunde (GB/s) gemessen wird. Jeder Render-Vorgang erfordert einen Datenaustausch über diesen Bus, beispielsweise während des Auslesens von Texturen und Geometrie oder dem Schreiben von finalen Farbwerten in den Framebuffer [15, S. 56]. Moderne GPUs sind mit leistungsstarken Arithmetic Logic Units (ALUs) ausgestattet. Diese Einheiten ermöglichen die Ausführung von Berechnungen, wie etwa Fragment-Shading, mit einer hohen Geschwindigkeit [3, Kap. 23.2]. Übersteigt der Bedarf an Lese- und Schreibzugriffen die physikalische Kapazität der Speicherbandbreite, stößt die Architektur an eine Grenze, auch wenn die ALUs theoretisch in der Lage wären, die reine Rechenlast zu

bearbeiten [3, S. 791]. In diesem Zustand limitiert die Datenübertragungsrate des Speichers die Gesamtleistung: Die ALUs verbleiben in Wartezuständen (*Stalls*), da der Speicherbus die benötigten Daten nicht schnell genug übertragen kann [3, S. 31 u. S. 791–792].

Deferred-Rendering-Architekturen zeichnen sich durch eine besonders hohe Auslastung der Speicherbandbreite aus. Die Notwendigkeit, teils extrem große Datenmengen (Albedo, Normalen, Tiefe etc.) über Multiple Render Targets (MRT) in den G-Buffer schreiben zu müssen (vgl. Kap. 2.1.2), belastet die Speicherbandbreite deutlich mehr als klassisches Forward-Rendering [16, S. 32], [11, Abschn. 2.1]. Bei besonders großen Datenmengen innerhalb eines G-Buffers spricht man auch von *Fat G-Buffern* [16, S. 71].

Nach dem Aufbau des G-Buffers im Geometrie-Pass manifestiert sich die Bandbreitenlimitierung ebenfalls im nachfolgenden Beleuchtungs-Pass, sollten Light-Volumes verwendet werden. Wie in Kapitel 2.1.2 dargelegt, erfordert die Überlappung mehrerer Light-Volumes den Einsatz von additivem Hardware-Blending. Der Vorgang des Blendings erzeugt für jedes berechnete Fragment RMW-Zyklen. Im Rahmen dieses Prozesses lädt die Hardware (GPU) den bisherigen Farbwert des Pixels aus dem VRAM (Read), addiert die neue Lichtintensität in der ALU (Modify) und schreibt das Resultat schließlich in den Speicher (Write) [3, Kap. 18.2.5], [14, S. 676]. Bei einer hohen lokalen Tiefenkomplexität der Lichtquellen (wenn sich mehrere Light Volumes über demselben Bildschirmbereich überschneiden), multipliziert sich diese Last [13, Folie 9]. Der permanente RMW-Zyklus führt in diesem Szenario zu einer Sättigung des Datenbusses und resultiert, wie zuvor beschrieben, in weiteren ALU-Stalls.

2.2.3 Registerdruck & Warp-Divergence

Neben der limitierten Speicherbandbreite stellt der sogenannte Registerdruck (*Register Pressure*) einen weiteren Leistungsengpass in der GPU-Architektur dar [3, S. 801 u. 904]. Jede GPU verfügt über extrem schnellen lokalen Speicher in Form von Shader-Registern [14, Kap. 11.2.5.1, Abschn. *Shader Registers*]. Diese Register bilden die schnellste Speicherstufe der Hardware, sind jedoch in ihrer Kapazität pro Rechenkern streng limitiert [3, S. 791]. Wenn ein Shader kompiliert, wird die maximale Anzahl an Registern, die ein einzelner Thread zur Laufzeit benötigt, ermittelt. Bei der Ausführung wird für jeden Thread diese Anzahl an benötigten Registern reserviert [17, Abschn. *The Logical Pipeline*, Punkt 8], [3, S. 32]. Das Phänomen des Registerdrucks beschreibt den Zustand, der eintritt, wenn ein Shader-Programm (wie beispielsweise ein rechenintensiver Fragment-Shader mit vielen Materialien oder NPR-Stilen) eine so hohe Anzahl an Registern anfordert, dass diese stark beansprucht oder vollständig erschöpft werden und damit die Anzahl maximal gleichzeitig ausführbaren Threads limitiert [3, S. 32 u. 904].

Ergänzend zur Limitierung aktiver Threads durch den Registerdruck existiert mit der sogenannten *Warp-Divergence* (oft auch als *thread divergence* bezeichnet) ein weiterer Flaschenhals bezüglich der Threadverarbeitung auf der GPU [3, S. 32]. Die Verarbeitung von Threads

durch die GPU erfolgt nicht isoliert, sondern gebündelt in festen Gruppen. Sogenannten *Warps* (oder *Wavefronts* in AMD-Architekturen) [3, S. 31]. Die Abarbeitung dieser Warps erfolgt entsprechend dem Single Instruction, Multiple Data (SIMD) beziehungsweise Single Instruction, Multiple Threads (SIMT)-Paradigma. In der Praxis bedeutet dies, dass für alle Threads innerhalb eines Warps in jedem einzelnen Taktzyklus exakt dieselbe Instruktion ausgeführt wird. Diese Instruktion wird somit auf alle individuellen Datensätze der jeweiligen Threads im Warp angewendet, wie beispielsweise auf die ausgelesenen Texturwerte von räumlich benachbarten Pixeln [3, Kap. 23.2]. Unter der Prämisse, dass der Shader-Code linear und uniform verläuft (alle Threads eines Warps können mit den gleichen Instruktionen abgearbeitet werden), ist die GPU in der Lage, mit maximaler Parallelität zu arbeiten und Shading-Berechnungen signifikant zu optimieren. Ein architektonischer Bottleneck in Form von Warp-Divergence entsteht dann, wenn dieser hardwareseitig erzwungene Gleichschritt durch dynamische Verzweigungen im Code aufgebrochen wird. Treten dynamische Verzweigungen, beispielsweise durch `if/else`- oder `switch`-Bedingungen auf, evaluieren Threads desselben Warps unterschiedliche Codepfade und damit unterschiedliche Instruktionen [3, Kap. 3.1]. Die GPU wird dadurch gezwungen, diese abweichenden Pfade innerhalb des Warps strikt sequenziell zu berechnen. Die parallele Architektur verliert an dieser Stelle ihre Effizienz, da Threads, die auf die Berechnung ihres Pfades warten müssen, in dieser Zeit ungenutzt pausieren [17, Abschn. *The Logical Pipeline*, Punkt 6], [3, S. 32].

3 Stand der Technik und verwandte Arbeiten

Dieses Kapitel analysiert den aktuellen Forschungsstand zu Forward- und Deferred-Rendering-Architekturen sowie moderne Industriepraktiken im Bereich hybrider Architekturen und selektiver Stilisierung. Durch die Betrachtung bestehender Verfahren im Kontext des NPR werden deren konzeptionelle und leistungstechnische Limitierungen aufgezeigt. Die Identifikation dieser Schwachstellen sowie der Mangel an empirischen Vergleichsdaten dienen der Definition der zugrundeliegenden Forschungslücke, legitimieren die in dieser Arbeit durchgeführte holistische Systemanalyse und ermöglichen die wissenschaftliche Einordnung des entwickelten Evaluations-Frameworks.

3.1 Klassische Rendering-Architekturen und deren Implikationen für das NPR

Deferred-Rendering-Paradigmen stellen sowohl in der akademischen Forschung als auch in der industriellen Praxis etablierte Standards dar [12, S. 2]. Moderne Echtzeit-Rendering-Pipelines werden maßgeblich von komplexen Beleuchtungsanforderungen getrieben, wodurch das effiziente Handhaben von Deferred-Architekturen in den Fokus rückt. Gleichzeitig werden im Bereich des Forward-Renderings durch diverse Optimierungsverfahren die klassischen Schwächen dieses Ansatzes zunehmend relativiert [5], [18]. Die Wahl des Rendering-Ansatzes ist in der Praxis stark kontextabhängig und auf die spezifischen Leistungs- und Qualitätsanforderungen des Zielbildes ausgerichtet. Folglich existieren sowohl Deferred- als auch Forward-Implementierungen in modernen Engines selten in ihrer reinen Form [15, S. 56]. Der wissenschaftliche Diskurs fokussiert sich daher primär auf hybride Lösungsansätze und gezielte Reduktionen architektonischer Engpässe.

Der primäre architektonische Nachteil klassischer Forward-Rendering-Ansätze liegt, wie in Abschnitt 2.1.1 dargelegt, im multiplikativen Aufwand von $O(N_{\text{obj}} \cdot N_{\text{light}})$ bei hoher Geometrie- und Beleuchtungskomplexität. Zusätzlich zu dieser Komplexität ergibt sich eine kombinatorische Explosion der Shader-Permutationen, sobald diverse Materialeigenschaften und Lichttypen in einer Szene verrechnet werden müssen. Dieser Limitierung liegt die inhärente Eigenschaft von Forward-Pipelines zugrunde, bei hoher Szenen-Entropie zwangsweise eine gewisse Menge an Overdraw zu erzeugen [8, S. 1]. Ansätze wie ein Z-Prepass zur frühzeitigen Verwerfung verdeckter Fragmente gehören in Forward-Pipelines zwar zum Standard [15, S. 57], reichen bei hoher Objekt- und Lichtkomplexität jedoch nicht

aus, um die Effizienz von Deferred-Ansätzen zu erreichen [11, S. 4]. Modernere Techniken wie *Clustered Forward* oder *Forward+* adressieren dieses Problem, indem Lichtquellen in 2D-Kacheln oder 3D-Clustern räumlich sortiert werden [5], [18]. Dies ermöglicht eine Effizienz, die Deferred-Ansätzen nahekommt, während der primäre Vorteil der Forward-Architektur, die hohe Flexibilität bei der Materialauswertung, erhalten bleibt.

Deferred-Rendering-Pipelines haben sich in Umgebungen mit hoher geometrischer Komplexität und anspruchsvollen Beleuchtungsszenarien weitestgehend durchgesetzt [12, Abschn. 2], weisen jedoch ebenfalls diverse architektonische Limitierungen auf. Der signifikanteste Flaschenhals ist die hohe Auslastung der Speicherbandbreite (Memory Bandwidth), welche durch den permanenten Schreib- und Lesezugriff auf den G-Buffer induziert wird [15, S. 55]. Ein moderner Lösungsansatz ist die Implementierung entkoppelter Architekturen wie des Visibility Buffer (V-Buffer) [15], welcher als Shading-Grundlage für hochkomplexe Geometrie-Pipelines wie der *Nanite*-Architektur in der Unreal Engine 5 dient [19, Folie *Visibility Buffer*]. Im Gegensatz zum traditionellen G-Buffer speichert der V-Buffer pro Pixel lediglich minimale geometrische Metadaten (wie Primitive- und Instanz-IDs) sowie Tiefenwerte. Die speicherintensive Materialauswertung wird vollständig in einen nachgelagerten Pass verschoben. Neben der Bandbreitenproblematik existieren in klassischen Deferred-Architekturen zwei weitere Nachteile: die fehlende native Kompatibilität mit hardwarebasiertem Multisample Anti-Aliasing (MSAA) sowie eine grundlegende Einschränkung in der Darstellung transluzenter Oberflächen. Da die Beleuchtung im Screen-Space berechnet wird, sind die für MSAA erforderlichen Geometriedaten zu diesem Zeitpunkt verworfen. Stattdessen müssen Verfahren wie Temporal Anti-Aliasing (TAA) eingesetzt werden [11], [20, S. 5]. Die Evaluierung semitransparenter Geometrie ist im reinen Deferred-Pfad nicht trivial lösbar, da ein Standard G-Buffer pro Pixel nur eine diskrete Tiefenschicht vorhält. Dieses Defizit wird in der industriellen Praxis standardmäßig über einen nachgelagerten Forward-Pass für transluzente Objekte gelöst [15, S. 56].

Hinsichtlich des NPR unterscheidet die aktuelle Literatur primär zwischen *Object-Space*- und *Screen-Space*-Methoden [1, S. 148]. Object-Space-Ansätze sind in der Regel in Forward-Pipelines verankert und berechnen Stilisierungen direkt auf der Geometrie, was eine granulare, objektbasierte Anpassung ermöglicht. Bei hoher Szenenkomplexität weisen diese Verfahren jedoch eine unzureichende Skalierbarkeit auf, was primär auf den Fragment-Overdraw bei rechenintensiven Abstraktionseffekten zurückzuführen ist [8, S. 1]. Die moderne Rendering-Forschung weicht zur Umgehung dieser Performance-Limits oft auf Screen-Space-Methoden (Post-Processing) aus, was architektonisch stark den Paradigmen des Deferred-Renderings entspricht. Wie Magdics et al. [1, S. 148] beschreiben, ist diese Screen-Space-Verarbeitung zwar sehr effizient, löst jedoch den direkten Bezug zum individuellen 3D-Objekt auf, wodurch etwa die selektive Stilisierung stark erschwert wird. Die Nutzung von Deferred-Rendering-Architekturen ermöglicht die Rekonstruktion des räumlichen Kontextes im Post-Processing. Wie Thaler [11, Abschn. 3.2] anmerkt, basiert die Lösung dieses Problems auf einem zweistufigen Ansatz: Im ersten Schritt werden alle räumlichen 3D-Informationen (Tiefen, Normalen) im G-Buffer gesammelt, um diese Metadaten im zweiten

Schritt für globale, Screen-Space-basierte NPR-Effekte nutzbar zu machen. Dies ermöglicht es, Stilisierungen im Screen-Space auch geometriebewusst durchzuführen. Klassische Deferred-Pipelines weisen in diesem Kontext jedoch eine fundamentale Schwäche bezüglich der Materialvielfalt auf [5, S. 1]. Das Auswerten zahlreicher spezifischer Materialien oder Stilisierungen in einem vereinheitlichten Screen-Space-Pass ist nur unter starken Performance-Einbußen durch *Dynamic Branching* auf der GPU realisierbar [11, Abschn. 2.6]. In der Konsequenz bleiben performante Stilisierungen bei reinem Deferred-Rendering meist auf globale Post-Processing-Filter beschränkt [1, S. 148].

Zusammenfassend lässt sich festhalten, dass die aktuelle Forschung zwar diverse architektonische Lösungen für hohe Geometrie- und Beleuchtungskomplexität sowohl in Forward- als auch in Deferred-Architekturen bietet, die spezifische Schnittmenge mit selektivem NPR in Deferred-Pipelines jedoch theoretischen Limitierungen unterliegt. Die Analyse der angewandten Industriepraxis sowie der Architektur moderner Game-Engines zeigt auf, wie diese theoretischen Hürden in der Praxis umgangen werden.

3.2 Rendering-Pipelines und Stilisierung in der Industriepraxis

Der theoretische Konflikt zwischen der für Deferred Shading charakteristischen Homogenität des Beleuchtungspasses und der für NPR oft erforderlichen Heterogenität individueller Stilisierungen zwingt die Industrie, zu architektonischen Kompromissen. Der naive Ansatz, alle für komplexe NPR-Modelle benötigten Parameter separat in den G-Buffer zu schreiben, führt unweigerlich zu einem sogenannten *Fat G-Buffer* [16, S. 71]. In modernen AAA-Produktionen der Spieleindustrie werden teilweise bis zu acht Render Targets (RTs) gleichzeitig beschrieben, was starke Bandbreiten-Engpässe (Memory Bandwidth Bottlenecks) verursachen kann [3, S. 884]. Folglich existiert in der Industriepraxis kein universeller, beziehungsweise einheitlicher, Deferred-NPR-Algorithmus. Stattdessen greifen Entwickler auf hoch spezialisierte, kompromissbehaftete Architekturen zurück, um selektive Stilisierungen innerhalb gegebener Hardware-Grenzen zu realisieren.

Ein primärer Lösungsansatz hierfür ist das *G-Buffer Packing* in Kombination mit „Shading-Model-IDs“. Die Wiederverwendung oder Neudefinition von Datenkanälen, die für das klassische PBR vorgesehen sind, reduziert den Bedarf an Speicherbandbreite. Ein exemplarisches Vorgehen zeigt das auf Toon-Rendering fokussierte Spiel *Hi-Fi RUSH*: Hierbei werden regulär für PBR-Daten vorgesehene G-Buffer-Kanäle gezielt umgewidmet, um diese stattdessen mit spezifischen NPR-Parametern und ID-Masken zu belegen (siehe Abbildung 3.1).

Ein ähnliches Verfahren zeigt die Architektur von *Uncharted 4*, in der eine komprimierte „Material-ID“ im G-Buffer mitgeführt wird, um im nachgelagerten Lighting-Pass zwischen

Hi-Fi RUSH G-Buffer Layout

	GBufferA R8G8B8A8		GBufferC R8G8B8A8		GBufferB R8G8B8A8
R	World Normal X	R	BaseColor R	R	Metallic (Fix at 0)
G	World Normal Y	G	Base Color G	G	Specular (Customize)
B	World Normal Z	B	Base Color B	B	Roughness
A	PerObjectGBufferData	A	GBufferAO (Customize)	A	ShadingModelID+SelectiveOutputMask

* Angepasste GBufferAO- und Specular-Kanäle für Toon-Rendering

Abbildung 3.1: Speicherlayout der Hi-Fi Rush G-Buffer. Eigene Darstellung in Anlehnung an [7, Folie „Our G-Buffer Layout“].

verschiedenen Shader-Features zu differenzieren [21, Folie „Deferred Pipeline“]. Überträgt man diesen Ansatz auf das NPR, um eine selektive Stilisierung direkt im regulären Lighting-Pass zu verankern, führt dies in der Praxis unweigerlich zu monolithischen Shader-Strukturen (Über-Shadern) [21, Folie „Problems“]. Eine steigende Stil- und Materialvielfalt induziert innerhalb eines solchen Shaders einen massiven Registerdruck (*Register Pressure*) und provoziert durch die notwendigen Fallunterscheidungen zeitgleich ineffiziente *Warp Divergence* (vgl. Kap. 2.2.3). Diese architektonischen Flaschenhälse limitieren die Gesamteffizienz der Rendering-Pipeline bei hoher Szenen-Entropie auf der GPU signifikant [9, Abschn. 6].

Moderne Engines implementieren hybride Architekturen, die eine grundlegende Deferred-Pipeline um einen nachgelagerten Forward-Pass erweitern, um den strukturellen Limitierungen des G-Buffers hinsichtlich der Materialvielfalt zu begegnen. Shading-Operationen, die hochkomplexe Materialbrechungen oder Transparenzen erfordern, werden in einem oder mehreren, an den Deferred-Pass anschließenden, Forward-Rendering-Schritten berechnet und im finalen Framebuffer kombiniert [3, S. 887]. Da dieser Forward-Pass jedoch die ursprünglichen Schwächen bezüglich der Beleuchtungskomplexität erbt, haben sich Techniken wie *Clustered Forward* [18] und *Forward+* [5] für diesen Hybridschritt etabliert. Durch räumliches Frustum- oder Tile-basiertes *Light-Culling* reduzieren sie die Anzahl der pro Fragment zu berechnenden Lichtquellen drastisch. Die Etablierung dieser Ansätze als Industriestandard zeigt sich beispielhaft an der *id Tech 6* Engine: Die Architektur nutzt hybrides Clustered Deferred und Forward Shading um komplexe Effekte und transparente Objekte mit der opaken Basisgeometrie zusammenzuführen, während die Lichtauswertung durch die räumliche 3D-Cluster-Zuweisung performant skaliert [22].

Eine weitere gängige Industriepraxis bildet das selektive Screen-Space Post-Processing. Hierbei wird die Stilisierung nicht auf Materialebene berechnet, sondern als bildbasierter 2D-Filter in der Nachbearbeitung angebracht [3, S. 661]. Die Industrie implementiert Verfahren wie das *Stencil-Masking*, um bei globalen Filtern dennoch eine selektive, objektbasierte Zuweisung zu gewährleisten [16, S. 81]. In klassischen Deferred-Pipelines ist der *Stencil-Buffer* (*Schablonenspeicher*) hardwareseitig meist direkt an den *Depth-Buffer* (*Tiefenpuffer*). gekoppelt [10, Folie „G-Buffer: Our approach“]. Anstatt Identifikationsnummern aufwendig als Farbinformationen in den regulären G-Buffer zu schreiben, markieren die zu stilisierenden Objekte ihre Pixel während des ersten Render-Durchlaufs (Geometrie-Pass)

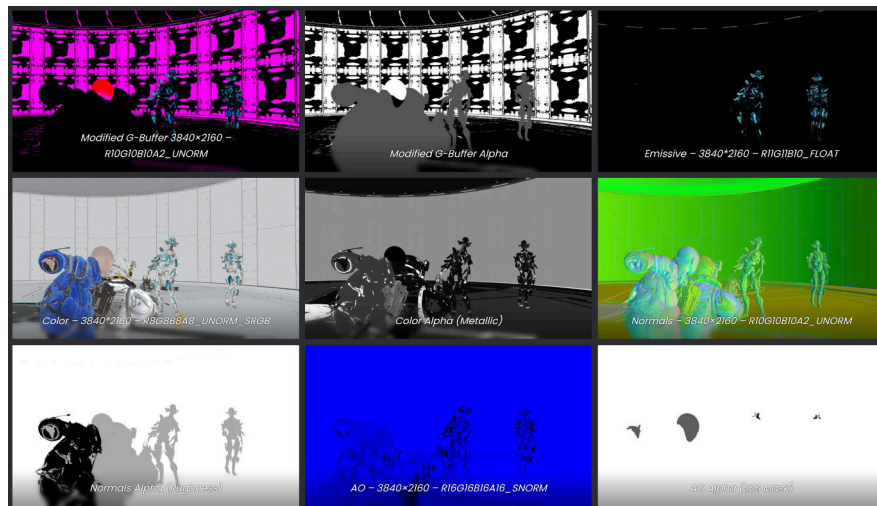
mit einem spezifischen Referenzwert direkt im Stencil-Buffer. Wird im Anschluss der Post-Processing-Filter über das fertige Bild gelegt, fungiert dieser Stencil-Wert als Filterkriterium. Die Grafikkarte prüft durch einen fest integrierten Hardware-Test, ob das aktuell berechnete Pixel zu einem markierten Objekt gehört. Ist dies nicht der Fall, wird das Fragment sofort verworfen. Der Shader-Code für den Post-Processing-Effekt kommt somit nur genau dort zur Ausführung, wo er auch tatsächlich benötigt wird. Die *CryEngine 3* demonstriert diese Praxis anschaulich: Sie nutzt *Stencil-Masking* gezielt, um rechenintensive Post-Processing-Algorithmen wie die Kantenerkennung (Edge-Detection) performant nur auf bestimmte Objekte im Vordergrund anzuwenden, während der Rest der Szene unberührt bleibt [16, S. 81].

Die Betrachtung der Industriepraxis offenbart, dass die Umsetzung selektiver, objektbasierter Stilisierungen in Echtzeit-Engines gegenwärtig aus einem hochkomplexen Patchwork aus G-Buffer-Manipulation, Uber-Shader-Branching und hybriden Forward-Overlays besteht. Dass moderne Rendering-Pipelines hierbei zunehmend fragmentieren, veranschaulicht die Analyse der aktuellen AAA-Produktion *PRAGMATA* (2026), deren selektives Post-Processing durch die Aufspaltung in zahlreiche isolierte Render-Pässe und eigens generierte Masken realisiert wird [23, Abschn. „Frame“] (vgl. Abbildung 3.2). Solche architektonischen Kompromisse sind zwar für spezifische, isolierte Anwendungsfälle praktikabel, lassen sich aber nur schwer auf Szenen mit extrem hoher Material- und Stil-Entropie unter Verwendung weniger leistungsfähiger Hardware skalieren. Aus dieser Diskrepanz zwischen hoch-spezialisierten Industrie-Workarounds und der Notwendigkeit einer skalierbaren Lösung ergibt sich die Forschungslücke der vorliegenden Arbeit, welche im folgenden systematisch eingegrenzt wird.



(a) Finales Rendering einer Spielszene.

Abbildung 3.2: Visualisierung der Pipeline-Fragmentierung in modernen AAA-Produktionen am Beispiel von *PRAGMATA* (2026) zur Veranschaulichung spezialisierter Industrie-Workarounds [23].



(b) Frame-Analyse des korrespondierenden G-Buffers und der isolierten Render-Pässe.

Abbildung 3.2: Visualisierung der Pipeline-Fragmentierung in modernen AAA-Produktionen am Beispiel von *PRAGMATA* (2026) zur Veranschaulichung spezialisierter Industrie-Workarounds [23] (Fortsetzung).

3.3 Limitierungen bestehender Ansätze und Identifikation der Forschungslücke

Wie in den Kapiteln 3.1 und 3.2 dargelegt, bieten sowohl die Forschung als auch die Industriepraxis diverse, hochspezialisierte Lösungsansätze für architekturbedingte Restriktionen klassischer Forward- und Deferred-Pipelines, insbesondere im Kontext der selektiven Stilisierung mittels NPR. Da eine vollumfängliche Analyse sämtlicher industriespezifischer Workarounds den Untersuchungsumfang dieser Arbeit überschreiten würde, adressiert die vorliegende Untersuchung die bestehende Forschungslücke durch eine empirische Rückbesinnung auf die architektonischen Fundamente. Die Evaluation der systematischen Verschiebung von Performance-Engpässen erfolgt über den Vergleich primärer Rendering-Paradigmen: Single-Pass-Forward-Rendering, Single-Pass-Deferred-Rendering mit G-Buffer-Packing über *Style-IDs* und Uber-Shader sowie Deferred-Rendering mit Light-Volumes als Referenz für Multi-Pass-Architekturen. Ziel ist die Erarbeitung einer architektonischen Basis für ein fundiertes empirisches Entscheidungsmodell.

Globale Screen-Space-Filter stellen in der Praxis einen einfachen und etablierten Standard dar [1, S. 147]. Im spezifischen Kontext der selektiven, objektbasierten Stilisierung, vornehmlich innerhalb der weit verbreiteten Deferred-Rendering-Paradigmen, stoßen gängige Praktiken jedoch in Abhängigkeit von der intendierten visuellen Stilistik an grundlegende architektonische Grenzen. Industrie-Workarounds wie Stencil-Maskierungen, dediziertes G-Buffer-Packing oder der Einsatz hybrider Pipelines versprechen hierbei zwar Abhilfe, sind jedoch aufgrund ihrer hochgradig spezialisierten Implementierung kaum hinsichtlich ihrer zugrundeliegenden Skalierungsmuster dokumentiert. Es bleibt folglich unzureichend

geklärt, welche Szenenkonfigurationen welche spezifischen Lösungsansätze erfordern und an welchen Stellen sich die primären Performance-Engpässe manifestieren.

Theoretische Performance-Engpässe, wie etwa Bandbreitenlimitierungen in G-Buffern oder Registerdruck in Uber-Shadern, sind in der Literatur zwar dokumentiert [3, S. 881], [11, Abschn. 2.6], [9, Abschn. 6]; wie Akenine-Möller et al. [3, S. 905] jedoch anmerken, verschieben sich diese Limitierungen in der Praxis sehr dynamisch in Abhängigkeit von der Szenenkomplexität. Der aktuellen Literatur mangelt es folglich an einer isolierten, komparativen und empirischen Systemanalyse, welche diese Engpässe spezifisch im Kontext des NPR quantifiziert.

Die Gewährleistung valider empirischer Messungen erfordert eine strikte Limitierung des Untersuchungsraums. Komplexe Sondereffekte oder Optimierungen wie Transparenzen, moderne Clustered-Techniken oder Anti-Aliasing-Ansätze werden für diese fundamentale Basis-Analyse bewusst exkludiert. Der Fokus liegt isoliert auf der Wechselwirkung zwischen opaker Geometrie, dynamischer Beleuchtung und materialbasierter NPR-Shader-Verarbeitung.

Die Schließung dieser Forschungslücke erfordert ein isoliertes Messinstrument, das die zugrundeliegenden Rendering-Architekturen anhand ihrer reinen Basis-Paradigmen und frei von proprietärem Engine-Overhead evaluiert. Das im Rahmen der vorliegenden Arbeit entwickelte Evaluations-Framework HyDra dient der methodischen Umsetzung dieses Vorhabens (siehe Abb. 3.3). Durch die Erhebung granularer Rohdaten unter skalierender Stil-Entropie und Szenenkomplexität soll das in Kapitel 1.2 skizzierte heuristische Entscheidungsmodell abgeleitet werden. Die technische Konzeption sowie die architektonischen Spezifika dieses Frameworks bilden den Schwerpunkt des nachfolgenden Kapitels.

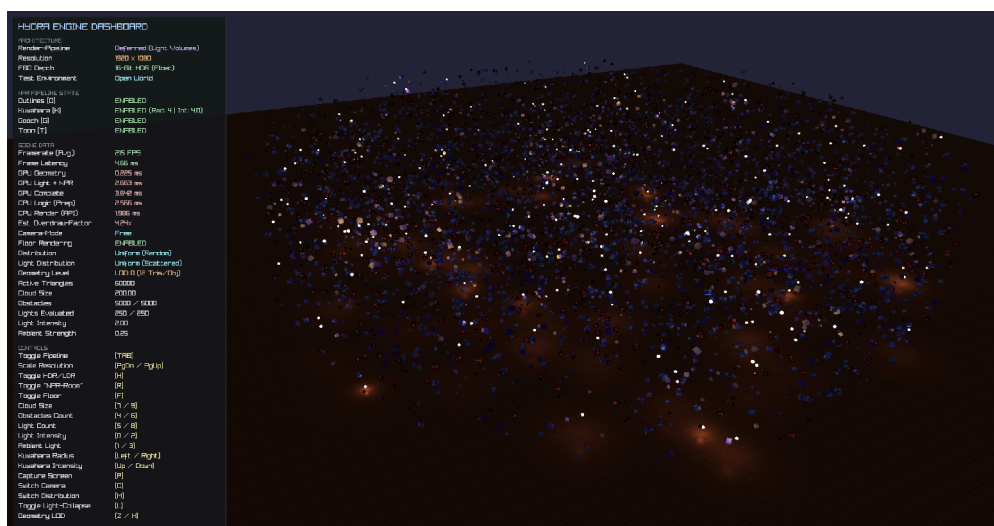


Abbildung 3.3: Screenshot aus HyDra-Framework.

4 Konzeption und Realisierung

Im folgenden Kapitel wird der Aufbau und die technische Umsetzung des in C++, und OpenGL (unter Verwendung der *raylib*-Bibliothek [24]–[26]) implementierten Test-Frameworks HyDra beschrieben. Es werden die Struktur der Daten, die Implementierungen der untersuchten Rendering-Pipelines, die Implementierungen der verwendeten NPR-Effekte, die individuellen architektonischen Limitationen sowie die Umsetzung der Leistungsmessungskomponenten (Benchmark und Telemetrie-Orchestrierung) erläutert.

Der Aufbau des HyDra-Frameworks folgt im Wesentlichen den Kernparadigmen der maximalen Vergleichbarkeit und des Determinismus. Das Programm wird so strukturiert, dass die drei zu untersuchenden Rendering-Pipelines unter identischen Bedingungen getestet werden können.

4.1 Systemarchitektur und Lebenszyklus

Die Architektur des HyDra-Frameworks ist durch die Entkopplung der logischen Szenenverwaltung durch die Central Processing Unit (CPU) vom eigentlichen Rendering (GPU-Dispatch) gekennzeichnet. Es wird garantiert, dass alle drei Rendering-Pipelines exakt denselben deterministischen Zustand als Eingabedaten erhalten. Ziel ist es, etwaigen individuellen CPU-Overhead möglichst zu eliminieren. Für die Deferred-Pipelines wird zudem ein semantischer G-Buffer aufbereitet, wofür im Rahmen der Szeneverwaltung zusätzlich die Vergabe von objektspezifischen *Style-IDs* erfolgt. Ein weiterer Bestandteil dieser vorbereitenden CPU-Logik ist das implementierte *View-Frustum-Culling*. Es fungiert als Filter, der die sichtbare Geometrie evaluiert, bevor die Grafikkarte involviert wird [14, Kap. 11.2.7, Abschn. *Frustum Culling*].

4.1.1 Frame-Lebenszyklus

Wie in Abbildung 4.1 dargestellt, durchläuft das Framework vor dem Eintritt in die Haupt-Rendering-Schleife ein initiales Setup. Diese Phase erfüllt im Wesentlichen zwei architektonische Aufgaben: Zum einen die Speicherallokation (VRAM) der benötigten FBOs, des G-Buffers sowie der Vertex Array Object (VAO)s und Vertex Buffer Object (VBO)s; andererseits das Laden, Kompilieren und Verlinken aller Shader-Programme inklusive des

Uniform-Mappings. Die in dieser einmaligen Initialisierungsphase definierte Basiskonfiguration kann zur Laufzeit partiell rekonfiguriert werden (etwa durch dynamische Skalierung der Auflösung oder Anpassung der Bitrate der G-Buffer-FBOs). Der Kontrollfluss wird erst an die zyklische Ausführung des Main-Loops übergeben, wenn der initiale Aufbau des VRAM und der Render-Pipelines vollständig abgeschlossen ist.

Die Initialisierung der für die spätere Hardware-Performance-Evaluation relevanten Timer erfolgt zu Beginn der Haupt-Rendering-Schleife. Gleichzeitig wird eine explizite Limitierung der Frame-Taktung (`AwaitCapacity()`) erzwungen, um zu verhindern, dass die CPU der asynchronen GPU-Abarbeitung zeitlich zu weit vorausläuft. Ein derartiger Rückstand der Grafikkarte würde die Latenz- und Profiler-Messungen systematisch verfälschen (vgl. Kap. 4.4.2).

Im direkten Anschluss steuert ein dezidiertes `InputController`-Modul die Modifikation des Szenenzustands. Diese Komponente ermöglicht eine manuelle Interaktion zur Laufzeit, um spezifische Performance-Metriken, visuelle Artefakte oder systemische Limitierungen der einzelnen Pipelines interaktiv untersuchen zu können.

Vor der eigentlichen Datenübertragung an die GPU wird die Geometrie einem CPU-seitigen logischen Pass unterzogen. Ein *View-Frustum-Culling* dient hierbei als Filter, um Geometrien, die sich außerhalb des sichtbaren Bereichs der Kamera befinden, von der weiteren Verarbeitung auszuschließen [14, Kap. 11.2.7, Abschn. *Frustum Culling*]. Für die verbleibenden, sichtbaren Entitäten erfolgt anschließend der Peripheral Component Interconnect Express (PCIe)-Speichertransfer (`r1UpdateVertexBuffer()`) der Modell-Transformationsmatrizen und Style-IDs.

Nachdem die Datenpräparation auf der CPU abgeschlossen ist, wird die Ausführung der selektierten Rendering-Pipeline orchestriert. In dieser Phase werden die notwendigen Zustandsänderungen durch das Szeneverwaltungs-Modul (`SceneManager`) vorgenommen und die instanziierten Draw-Calls an die GPU übermittelt. Die Grafikkarte übernimmt daraufhin, asynchron zum Main-Thread, die Rasterisierung und Ausführung der definierten Shader-Logik [27, S. 494]. Die exakte Implementierung der Rendering-Pipelines wird in den nachfolgenden Kapiteln erläutert.

Den Abschluss eines Frame-Zyklus bildet die Präsentation des finalen Bildes auf dem Bildschirm. Zusätzlich werden die asynchronen Hardware-Timer der GPU ausgelesen (`QueryOldestResult()`). Dieser *Drain*-Prozess erfolgt explizit nicht-blockierend, um die fortlaufende Ausführung der GPU für nachfolgende Messungen nicht durch künstliche CPU-Stalls zu unterbrechen (vgl. Kap. 4.4.2).

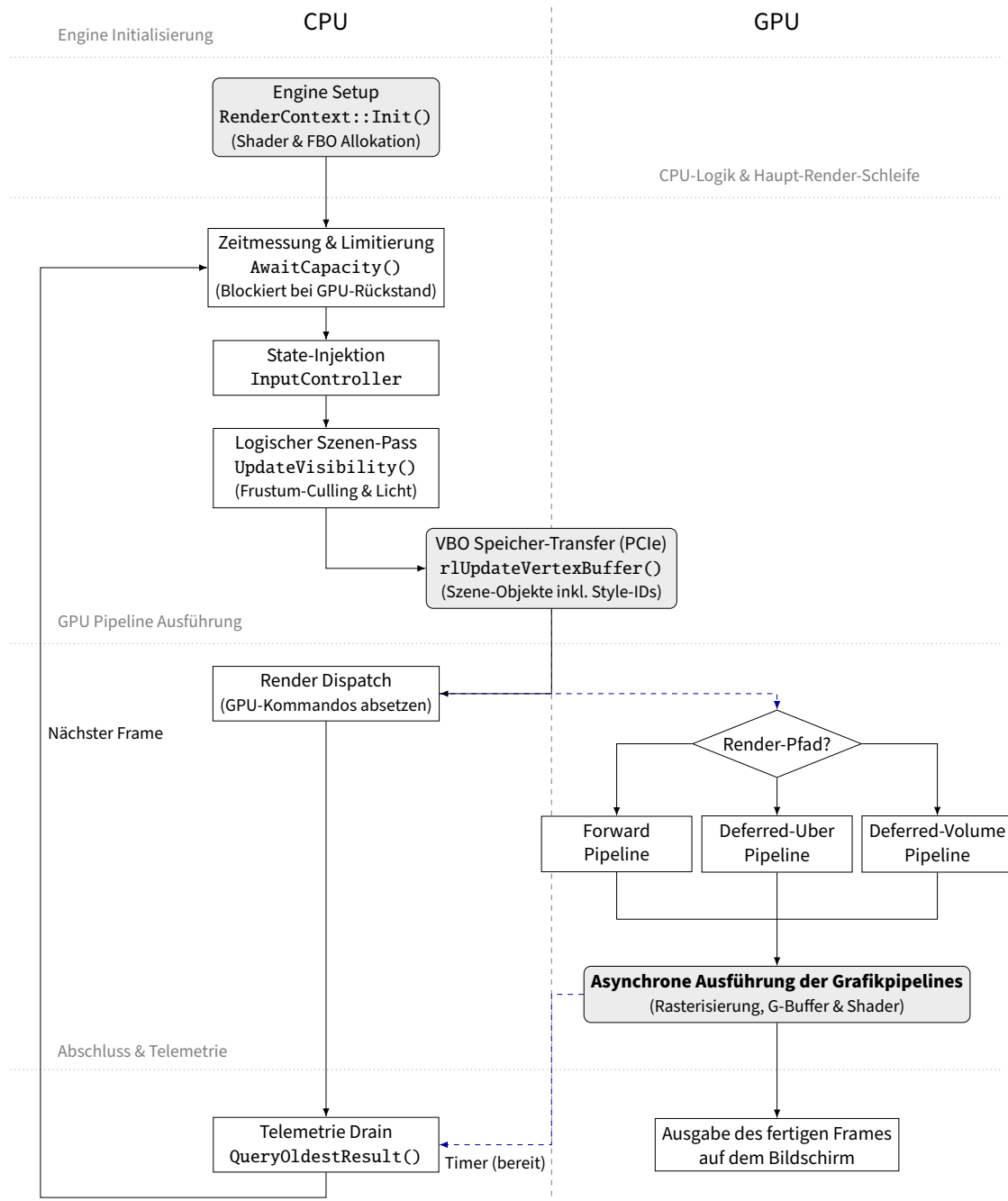


Abbildung 4.1: UML-Aktivitätsdiagramm des Frame-Lebenszyklus innerhalb des Hydra-Frameworks. Die Architektur gliedert sich in eine initiale Setup-Phase und den nachfolgenden Main-Loop. Der Kontrollfluss veranschaulicht die explizite CPU-Limitierung zur Wahrung der Messgenauigkeit, die CPU-seitige Datenpräparation sowie das Dispatching an eine von drei wählbaren Rendering-Pipelines. Abgeschlossen wird der Frame durch die asynchrone GPU-Ausführung und eine nicht-blockierende Telemetrie-Extraktion.

4.1.2 Szenenverwaltung

Die inhaltliche Verwaltung der dargestellten Szene wird im HyDra-Framework von einem dedizierten Szenenmanager-Modul (SceneManager) orchestriert. Dieses verwaltet den Aufbau der Geometrie, insbesondere die Punktlicht-Quellen und die beleuchteten Geometrien. Neben einem optionalen, großflächigen „Boden-Quad“ werden alle Geometrien und Punktlichtquellen innerhalb einer zylinder-Förmigen Objektwolke dargestellt siehe 3.3. Die Geometrien werden prozedural mithilfe eines festgelegten Seeds erstellt. Damit wird sichergestellt, dass jeder Programmaufruf oder jede Änderung durch den Input-Controller zu vollständiger Szeneparität hinsichtlich der Geometrieverteilung unter den gegebenen Parametern Objektanzahl, Punktlichtanzahl, Lichtintensität und Objektdichte (Radius der Objektwolke) führt.

Die Datenhaltung der Szenegeometrien und Punktlicht-Quellen mitsamt ihrer assoziierten Style-IDs und Transformationsmatrizen erfolgt in Arrays, die mit der Maximalanzahl der erstellbaren Objekte vorallokiert werden. Jeder Geometrie wird ein Index zugewiesen. Anhand dieser Indizes werden für jedes Objekt drei Arrays für die Style-ID, die räumliche Position und die bereits vorab berechnete Transformationsmatrix, nach dem Structure of Arrays (SoA)-Muster [3, S. 793], gespeichert.

Hervorzuheben ist hier die Logik bezüglich der Stilentropie-Steuerung, die zwischen gestreuten (hohe Entropie) und geordneten (niedrige Entropie) Verteilungen wechselt. Die Zuweisung der Stil-ID erfolgt in Listing 1 und beschreibt die Gruppierung/Sortierung innerhalb des HyDra-Frameworks in der zylindrischen Objektwolke mit anteiliger Verteilung der Stile.

```
1  float styleId;
2  if (state.useClusteredStyles) {
3      float normAngle = std::fmod(angle, 2.0f * PI);
4      if (normAngle < 0.0f)
5          normAngle += 2.0f * PI;
6      // Divide the 2PI circle by the current number of active styles
7      const float slice = (2.0f * PI) / numStyles;
8      const int clusterIndex =
9          std::clamp(static_cast<int>(normAngle / slice), 0,
10                  ↪ static_cast<int>(numStyles - 1));
11      styleId = activeStyles[clusterIndex];
12  } else {
13      // Distribute using the current pool size
14      styleId = activeStyles[i % static_cast<int>(numStyles)];
15  }
```

Listing 1: C++ Code zur Berechnung der styleId-Verteilung basierend auf dem Cluster-Status in HyDra-Framework.

Die Prüfung der Sichtbarkeit erfolgt mittels eines klassischen Schnitttests der Objekte gegen die sechs Ebenen des Kamera-View-Frustums [14, S. 688, Abschn. *Frustum Culling*]. Darüber hinaus wird die Größe (der Radius) der Light-Volume-Proxies für die Deferred-Volume-Pipeline durch die Lichtintensität beeinflusst [3, S. 891]. Im Rahmen des View-Frustum-Cullings werden diese ebenfalls berücksichtigt, um die Darstellung von Lichtern, deren Wirkungsbereich vollständig außerhalb des Sichtfeldes liegt, zu vermeiden und somit keine unnötigen Beleuchtungsberechnungen für spätere Performance-Evaluationen zu verursachen.

Das Resultat des Szeneaufbaus ist abhängig von der gewählten Rendering-Pipeline. Während die Deferred-Pipelines die sichtbaren Instanzen als zusammenhängende, lineare Arrays für die Weiterverarbeitung erhalten, erzwingt das Forward-Rendering eine Aufspaltung der Daten. Der SceneManager durchläuft hierfür eine Iteration über die Style-IDs der sichtbaren Objekte und ordnet diese in pipelinespezifische Queues (z. B. `fwdTransformsGooch`, `fwdTransformsToon`) an. Dieser Sortiervorgang auf der CPU ist eine architektonische Notwendigkeit des Forward-Ansatzes, um kostenintensive Shader-State-Changes auf der GPU zu bündeln [3, S. 795 u. S. 886].

4.1.3 Datenfluss der Stilisierung und G-Buffer Layout

Die verschiedenen verwendeten Shading-Effekte werden für jedes Objekt über Style-IDs definiert. Die genaue Definition dieser Style-IDs erfolgt über dynamische Vertex-Buffer (VAOs und VBO). Schließlich werden die Style-IDs als instanziierte Vertex-Attribute für jede Geometrie in den Vertex-Shader übertragen. Die Style-IDs werden nicht pro Vertex, sondern einmal pro Instanz im Vertex-Shader ausgelesen, um sie effizient an den Fragment-Shader weiterzureichen, der die G-BufferTextur erstellt [3, S. 797-798]. In der Forward Pipeline erfolgt die Mitlieferung der Style-IDs pro verarbeiteter Geometrie direkt in den Shader-Aufrufen (Draw-Calls).

Wie in Tabelle 4.1 aufgezeigt wird für das Render-Target RT0 (Albedo) ein 8-Bit-Format (R8G8B8A8) genutzt. Für den Normalen-Buffer (RT1) wird hingegen ein 16-Bit-Format (R16G16B16A16) verwendet. Diese erhöhte Präzision ist eine architektonische Notwendigkeit, da geringere Bittiefen bei den nachfolgenden Screen-Space-Lichtberechnungen unzureichend sind [16, Folie 37]. Die genauen visuellen Auswirkungen (Banding-Artefakte), die diesen Kompromiss zugunsten einer höheren Speicherbandbreite erzwingen, werden in der Evaluation der Deferred-Volume-Pipeline in Kapitel 4.2.3 im Detail dargelegt.

Der semantische Kern der hybriden Stilisierung liegt in der Persistierung der durchgereichten Instanz-Metadaten. Wie in Tabelle 4.1 ersichtlich, wird die komprimierte Style-ID in den ungenutzten Alpha-Kanal des Normalen-RT encodiert. Mithilfe dieses Mappings stehen den nachgelagerten Deferred-Shadern sämtliche benötigten Metadaten pro Fragment zur Verfügung. Dadurch kann die GPU im Shading dynamisch und selektiv den korrekten NPR-Algorithmus auswählen.

Tabelle 4.1: Spezifikation des G-Buffer-Layouts im Hydra-Framework.

Render Target	Format	Kanal R	Kanal G	Kanal B	Alpha
<i>RT0 (Albedo)</i>	R8G8B8A8	BaseColor R	BaseColor G	BaseColor B	BaseColor A
<i>RT1 (Normal)</i>	R16G16B16A16	World Normal X	World Normal Y	World Normal Z	Style-ID
<i>Depth</i>	8-Bit Grayscale	Linear Depth	-	-	-

4.2 Pipeline-Architekturen und systematische Restriktionen

In Kapitel 2.1, wurden die theoretischen Paradigmen der grundlegenden Rendering-Pipelines etabliert. In diesem Abschnitt werden die tatsächlichen architektonischen Restriktionen der implementierten Renderpfade innerhalb des Hydra-Frameworks analysiert. Der Fokus liegt demnach nicht auf der grundlegenden Funktionsweise, sondern auf den spezifischen Modifikationen zur Integration selektiver Stilisierungsprozesse und den daraus resultierenden Limitationen. Im Rahmen der vorliegenden Untersuchung werden die systemisch bedingten absoluten Skalierungsgrenzen und spezifischen visuellen Artefakte (Drawbacks) der jeweiligen Architekturen analysiert, um das theoretische Fundament für die quantitative Performance-Evaluation in Kapitel 5 zu legen.

4.2.1 Forward-Rendering-Pipeline

Die Umsetzung der Forward-Pipeline innerhalb des Hydra-Frameworks folgt der klassischen Standardimplementierung, wie in Kapitel 2.1.1 dargelegt und stellt eine vergleichende „Baseline“ dar. Die Forward-Pipeline wird entsprechend ohne zusätzliche Optimierungen wie einen Z-Prepass oder anderweitige Variationen implementiert.

Wie in Kapitel 4.4.1 dargelegt, erfolgt die selektive Stilisierung in der Forward-Pipeline unmittelbar während der Drawcalls der in Batches instanziierten und von der CPU (dem SceneManager) aufbereiteten Geometrie-Arrays. Demzufolge wird jedem Objekt der entsprechende Fragment-Shader sowie der damit assoziierte NPR-Stil direkt zugewiesen.

Dieser Ansatz weist allerdings, vor allem in Hinblick auf die selektive Stilisierung, diverse Schwachstellen und Limitationen auf. Eine wesentliche Schwachstelle ist die Begrenzung in Form von vorherrschenden Uniform-Buffer-Limits. In der für das Framework gewählten Grafik-API OpenGL 3.3 erfolgt die Übergabe der Lichtdaten-Arrays standardmäßig über Uniform Buffer Objects (UBOs), um den CPU-seitigen Overhead einzelner Uniform-Zuweisungen pro Lichtquelle zu minimieren [3, S. 798]. Die maximale physische Speichergröße, die ein einzelner Uniform-Block umfassen darf, wird treiberseitig durch das

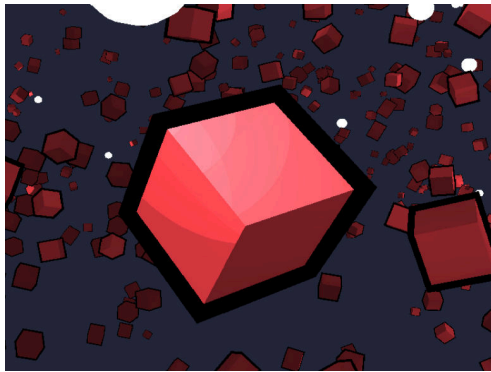
hardwareabhängige Limit `GL_MAX_UNIFORM_BLOCK_SIZE` definiert. Während dieser Wert auf moderner Desktop-Hardware variiert, schreibt die OpenGL-3.3-Spezifikation herstellerübergreifend eine garantierte Mindestgröße von 16 KiB vor [28, S. 321, Tab. 6.45].

Wird jede der Punktlichtquellen im Shader durch eine vereinfachte Struktur aus einer Positions- und einer Farb-/Intensitätskomponente mit einer Gesamtgröße von 32 Bytes abgebildet, lässt sich die theoretisch maximale Lichtanzahl N_{Licht} innerhalb des garantierten Application Programming Interface (API)-Mindestlimits über folgende Relation bestimmen:

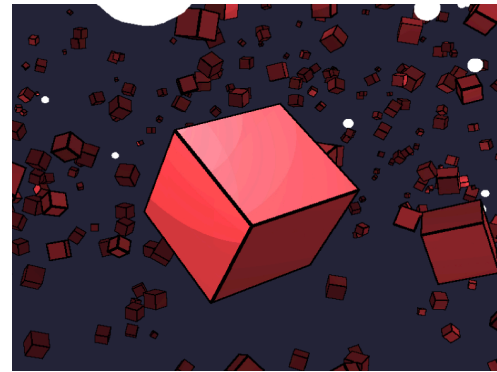
$$N_{\text{Licht}} = \frac{\text{GL_MAX_UNIFORM_BLOCK_SIZE}}{\text{Größe pro Lichtquelle}} = \frac{16.384 \text{ Bytes}}{32 \text{ Bytes}} = 512 \quad (4.1)$$

Auf Hardware-Ebene existiert diese Restriktion, weil Uniform-Daten in dedizierten, extrem schnellen On-Chip-Registern (Constant Registers) der GPU gehalten werden müssen [3, S. 35]. Dieses fundamentale API-Limit begründet, warum die Single-Pass-Architekturen des Frameworks (Forward und Deferred-Uber) auf eine Kapazitätsgrenze von maximal 500 Punktlichtern beschränkt sind, um eine vollständige Plattformkompatibilität ohne Treiber-Abstürze zu gewährleisten.

Eine weitere grundlegende Einschränkung der naiven Forward-Pipeline bezüglich selektiver Stilisierung ist, dass sich traditionelle Screen-Space- oder Post-Processing-Effekte wie ein Kuwahara-Filter oder Sobel-Outlines nicht selektiv bzw. partiell anwenden lassen. Für die Generierung dieser Effekte ist präzises Wissen bezüglich der finalen Szene sowie der Umgebungsgeometrie erforderlich [1, Abschn. 5]. Im Rahmen des implementierten Forward-Renderings erfolgt die Anwendung der Shader direkt und isoliert auf die einzelnen Geometrien innerhalb der Draw-Calls, wodurch keine Umgebungsbezogenen Daten (semantische Objektidentität) des finalen Bildes während des Shadings auf der GPU vorhanden sind. Eine selektive Realisierung solcher Effekte ist in naiven Single-Pass Forward-Pipelines also nicht möglich. In Bezug auf den im HyDra-Framework implementierten Effekt zur Darstellung von Konturen kann in der Forward-Pipeline, anstelle der in den Deferred-Pipelines verwendeten Sobel-Outlines (Screen-Space-Methode) das bedingt gleichwertige Inverted-Hull-Verfahren (Object-Space-Methode) zur Kantenzeichnung verwendet werden (siehe Abb. 4.2). Diese nur äußerst bedingte visuelle Parität muss für etwaige vergleichende Evaluationen berücksichtigt werden.



(a) Inverted-Hull + Toon-Shading in Forward



(b) Sobel-Outlines + Toon-Shading in Deferred

Abbildung 4.2: Toon-Shading und Vergleich von Kontureffekten in Forward- und Deferred-Pipelines mittels Inverted-Hull- und Sobel-Outline-Verfahren in Hydra-Framework.

4.2.2 Deferred-Uber Rendering Pipeline

Im Hydra-Framework wird zur Referenz eines einfachen Deferred-Ansatzes eine Deferred-Pipeline in einer Single-Pass-Architektur (bezogen auf den Lighting-Pass) mittels Uber-Shader-Struktur implementiert (vgl. Kap. 2.1.2). Dieser Ansatz wird gewählt, um Deferred Shading in seiner grundlegendsten Form, vergleichbar mit dem naiven Single-Pass-Forward-Ansatz, zu untersuchen und die Einflüsse etwaiger Optimierungsverfahren möglichst gering zu halten. Deferred-Architekturen mit einem einzigen Lighting-Pass finden in der Praxis, ähnlich wie naive Forward-Ansätze, allerdings nur selten Anwendung (vgl. Kap. 3.1).

Aufgrund der Charakteristik des Deferred-Uber-Ansatzes, bei dem nur ein Lighting-Pass erfolgt, unterliegt auch diese Pipeline den in Kapitel 4.2.1 beschriebenen Restriktionen der Grafik-API und Hardware in Bezug auf die Uniform-Buffer-Limits und ist somit durch eine Maximalanzahl von 500 gleichzeitigen Punktlichtquellen gekennzeichnet (vgl. 4.1).

Listing 2 skizziert die Kontrollstruktur des monolithischen Uber-Shaders. Auf Hardware-Ebene zwingen diese `if-else`-Zweige die GPU bei unterschiedlicher Fragment-Stilisierung in eine ineffiziente, sequenzielle Abarbeitung. Weisen benachbarte Fragmente innerhalb desselben Warps unterschiedliche `Style-IDs` auf, müssen alle betroffenen Code-Pfade nacheinander durchlaufen werden [3, S. 32] (vgl. Kap. 2.2.3). Die performancetechnischen Auswirkungen werden in der empirischen Evaluation in Kapitel 5 (insbesondere in Abschnitt 5.5.1) quantitativ untersucht.

Eine weitere Einschränkung der Deferred-Uber-Pipeline ist, dass bei der selektiven Darstellung klassischer Post-Process-Effekte (Bildraum-Filter), wie dem im Hydra-Framework verwendeten Kuwahara-Filter, auf *Pre-Lit*-Varianten der Effekte zurückgegriffen werden muss. Der Begriff *Pre-Lit* beschreibt hier die Anwendung des Bildraum-Filters auf die unbeleuchtete Geometrie, bevor die reguläre Beleuchtungsberechnung (hier *Blinn-Phong* [14, Kap. 11.1.3.2]) erfolgt. Da der Kuwahara-Filter für die Bestimmung von Mittelwert und

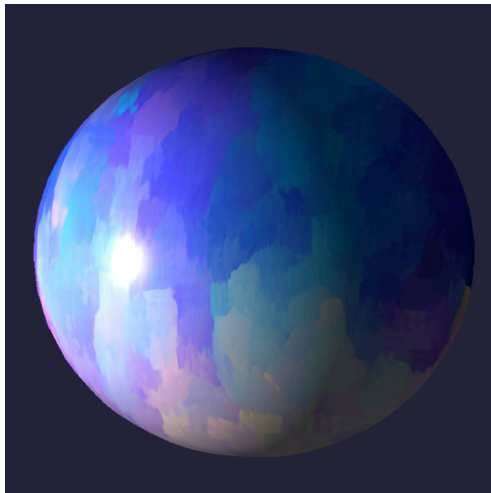
```

1 // Extract Normal and StyleID (Decode from 8-bit "Alpha"-channel)
2 int styleID = int(round(normalData.a * 255.0));
3 // Masking check (for light sources and sky box)
4 if (styleID == 0) {
5     // ...
6     return;
7 }
8 if (styleID == 2) {
9     // GOOCH SHADING ...
10
11 } else if (styleID == 3) {
12     // SELECTIVE SOBEL OUTLINING & TOON ...
13
14     if (isEdge) {
15         finalColor = vec4(0.0, 0.0, 0.0, 1.0);
16     } else {
17         // TOON SHADING ...
18     }
19 } else if (styleID == 4) {
20     // KUWAHARA FILTER (Pre-Lit) ...
21 } else {
22     // styleID == 1 or Fallback for Debug
23     useBlinnFallback = true;
24 }
25 // BASELINE BLINN-PHONG FALLBACK
26 if (useBlinnFallback) {
27     // ...
28 }

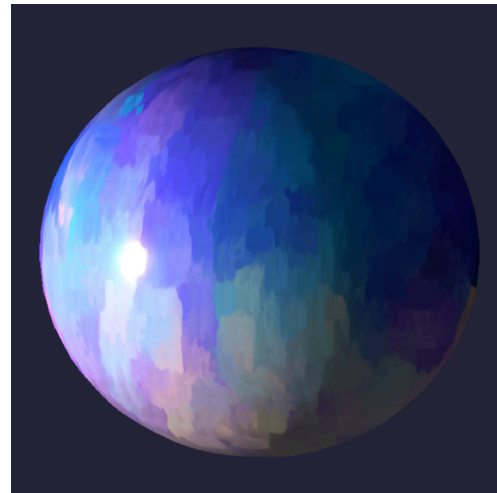
```

Listing 2: GLSL-Code des Deferred-Über Beleuchtungspasses in HyDra-Framework

Varianz die Farbinformationen der Nachbarpixel innerhalb eines definierten Radius benötigt (Filter-Kernel [29, Abschn. 2]), müsste für eine *Post-Lit*-Variante (Filterung des fertig beleuchteten Bildes) das finale Beleuchtungsergebnis der gesamten Nachbarschaft vorliegen. In einem Single-Pass-Über-Shader würde dies erfordern, die globale Lichtschleife für jedes einzelne Kernel-Sample redundant zu evaluieren. Bei einer hohen Anzahl an Lichtquellen führt dies zu einer kombinatorischen Explosion der algorithmischen Komplexität von $O(\text{Filter-Samples} \times \text{Lichtanzahl})$ pro Pixel, was eine Echtzeit-Ausführung ausschließt. Abbildung 4.3 zeigt den stilistischen Unterschied von Pre- und Post-Lit Verfahren im HyDra-Framework anhand des implementierten Kuwahara-Filters.



(a) Pre-Lit Kuwahara in Deferred-Uber



(b) Post-Lit Kuwahara in Deferred-Volume

Abbildung 4.3: Visueller Vergleich von Pre-Lit und Post-Lit Kuwahara in Hydra-Framework.

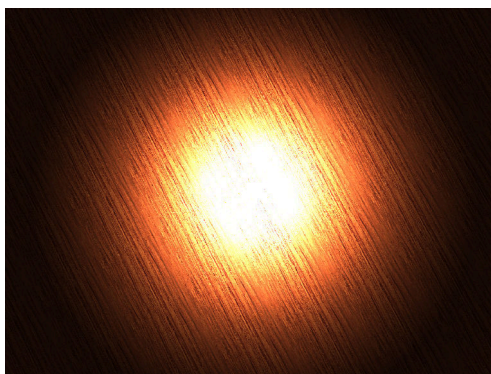
4.2.3 Deferred-Volume Rendering Pipeline

Zusätzlich zur Deferred-Uber-Pipeline wurde im Hydra-Framework eine weitere Deferred-Pipeline implementiert, um die ebenfalls weit verbreiteten Multi-Pass-Rendering-Verfahren (bezogen auf den Lighting-Pass) zu repräsentieren und vergleichend heranzuziehen. Dieses Paradigma reflektiert den in der Praxis wesentlich verbreiteteren Ansatz in Deferred-Pipelines [13, Folie 5], ohne dabei auf fortgeschrittene Optimierungsverfahren zurückzugreifen. Zu diesem Zweck wird die *Deferred-Volume-Pipeline* implementiert. Deferred Rendering mit Light-Volumes zeichnet sich, wie in Kapitel 2.1.2 dargelegt, durch das Zeichnen von Proxy-Geometrien (hier in Form einer Kugel) um die entsprechenden Punktlichtquellen aus. Der vorliegende Ansatz teilt den in der Deferred-Uber-Pipeline verwendeten Uber-Shader in zwei ausgelagerte Pässe auf. Dies erfolgt, indem die Beleuchtungslogik in isolierte, iterative Draw-Calls verpackt wird.

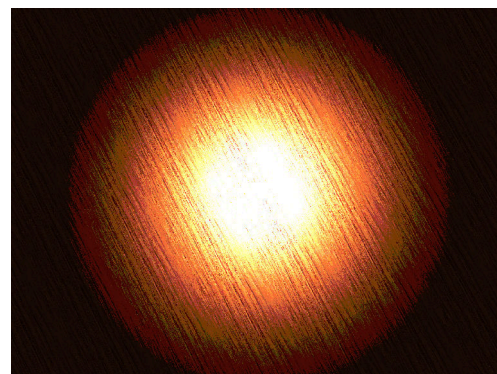
Die Charakteristische-Last der Deferred-Volume-Pipeline zeigt sich hier in einem anderen Bereich und wird, durch das additive Blending der Light-Volumes, im Framebuffer(ROPs) und der VRAM-Bandbreite verortet [13, Folie 9], [14, S. 676]. Die genauen performancetechnischen Auswirkungen werden in der empirischen Evaluation in Kapitel 5 (insbesondere in Abschnitt 5.4.2) quantitativ analysiert.

Ein signifikanter architektonischer Unterschied zwischen der monolithischen Deferred-Uber-Pipeline und der Deferred-Volume-Pipeline äußert sich in der Anfälligkeit für visuelle Quantisierungsartefakte (*Color Banding*) bei der Verwendung von Bildformaten mit niedriger Bit-Tiefe (8-Bit pro Kanal) innerhalb der G-Buffer. In Ansätzen mit einem einzigen Lighting-Pass erfolgt die Akkumulation der Lichtbeiträge innerhalb einer Schleife auf Treiberebene in lokalen GPU-Registern mit nativer 32-Bit-Fließkommapräzision [14, Kap. 11.2.5.1, Abschn. *Shader Registers*]. Im Gegensatz dazu entkoppelt die Deferred-Volume-Pipeline die Lichtberechnung in diskrete Pässe, deren Ergebnisse mittels additiven

Hardware-Blending (*Blending Stage*) im Framebuffer aufsummiert werden (vgl. Kap. 2.1.2). Wird hierbei ein 8-Bit-Buffer verwendet, unterliegt das Zwischenergebnis nach der Evaluierung jedes einzelnen Lichtvolumens einer fortlaufenden Quantisierung sowie einer harten Rundung auf den Wertebereich $[0, 1]$ (*Clamping*) [28, Kap. 4.1.7]. Die wiederholte Rundung der Lichtbeiträge führt zu einer Akkumulation von Rundungsfehlern, die sich visuell als stufenförmige Banding-Artefakte darstellen (siehe Abb. 4.4). Diese Artefakte sind besonders in künstlerischen und abstrahierenden Effekten, wie sie im NPR Anwendung finden, von Signifikanz (siehe Abbildung 4.5). Die Deferred-Volume-Pipeline ist folglich zwingend auf High Dynamic Range (HDR) mit mindestens 16-Bit-Fließkommapräzision zur Kompensation dieses Präzisionsverlusts und Vermeidung von Artefakten wie etwa Farbsättigung durch Clamping angewiesen. Dieser Umstand erhöht die VRAM-Belastung von Deferred-Volume-Ansätzen im Vergleich zu anderen Architekturen.

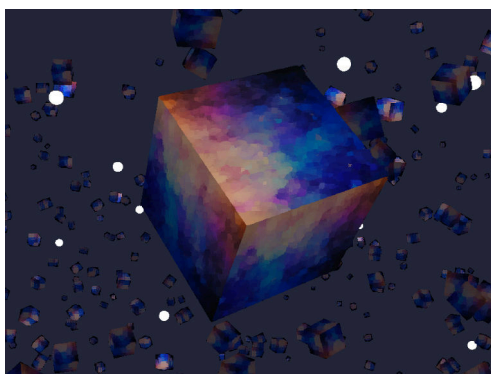


(a) 16-Bit Fließkommapräzision (HDR)

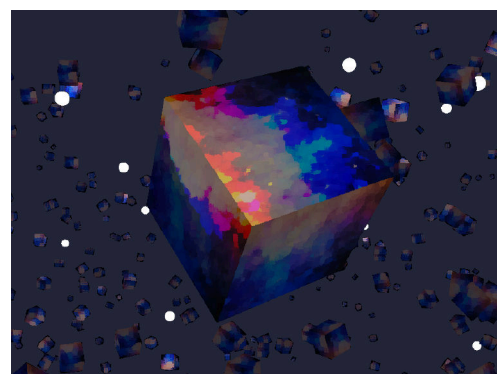


(b) 8-Bit Fixpunktpräzision (LDR)

Abbildung 4.4: Visueller Vergleich der Lichtakkumulation zur Isolation von Quantisierungsartefakten in Deferred-Volume-Pipeline in Hydra-Framework.



(a) Verlustfreie Akkumulation (16-Bit HDR)



(b) Farb-Clamping und Posterisation (8-Bit LDR)

Abbildung 4.5: Auswirkung des Framebuffer-Präzisionsformats auf die Integrität des Kuwahara-Filters in Deferred-Volume-Pipeline in Hydra-Framework.

Im Gegensatz zur Deferred-Über-Architektur bietet die Deferred-Volume-Architektur den Vorteil, dass Bildraumeffekte, ohne den Zwang Pre-Lit-Verfahren anzuwenden, mit Post-Lit-Verfahren aufbereitet werden können (vgl. Abb. 4.3). Für die spätere Performance-Analyse (siehe Kap. 5.5.2) ist jedoch zu bemerken, dass diese Post-Lit-Evaluierungen mit einem leicht

erhöhten algorithmischen Aufwand verbunden sind. Dieser begründet sich im Resolve-Pass der Deferred-Volume-Pipeline durch die multiplen Texturzugriffe pro Kernel-Sample; konkret das simultane Abfragen von G-Buffer-Identitätsdaten und dem akkumulierten Lichtpuffer. Im Pre-Lit-Verfahren der Deferred-Uber-Pipeline müssen dagegen nur die Albedo-Daten (Farbinformationen) der Nachbarpixel ausgelesen werden.

4.3 NPR-Payloads

Für die Evaluation der zugrunde liegenden Rendering-Pipelines werden im HyDra-Framework neben einer Baseline (Blinn-Phong-Shading [14, Kap. 11.1.3.2]) verschiedene NPR-Effekte implementiert. Im Folgenden werden die Charakteristika der verwendeten Stile sowie ihre grundlegende algorithmische Komplexität erläutert, um sie für die in Kapitel 5 durchgeführten Evaluationen zu kontextualisieren. Eine detaillierte Beschreibung der Implementierung wird in dieser Arbeit nicht vorgenommen, da für alle NPR-Effekte und das Baseline-Shading die jeweils etablierten Standardlösungen implementiert wurden. Die implementierten Effekte fungieren demnach lediglich als reine Payloads (Lastgeneratoren) für die Evaluation der selektiven Stilisierung unter Anwendung der verschiedenen Rendering-Verfahren.

Für das HyDra-Framework sind die in Tabelle 4.2 beschriebenen Effekte bzw. Stile verwendet worden.

Tabelle 4.2: Ausführung der verwendeten Shading-Effekte und NPR-Stile in HyDra-Framework.

Effekt	Algorithmische Charakteristika	Semantische Charakteristika
Kuwahara [29], [30]	Iterative Kernel-Texture-Fetches in 4 Quadranten zur Varianz- und Mittelwertberechnung. Radiusabhängige Schleifen.	Erzeugt eine stilisierte, male- rische Abstraktion (Painterly- Look) durch Homogenisierung lokaler Farbflächen (siehe Abb. 4.3).
Toon [31]	Skalarprodukte ($N \cdot L$) und Quan- tisierung (Banding) der diffusen Beleuchtung über Floor/Step- Funktionen.	Reduziert kontinuierliche Schat- tierungsverläufe auf diskrete Helligkeitsstufen; Charakteris- tisch für die Ästhetik klassi- scher Cel-Animationen (siehe Abb. 4.2).

Weiter auf der nächsten Seite...

Tabelle 4.2 – Fortsetzung

Effekt	Algorithmische Charakteristika	Semantische Charakteristika
Gooch [4]	Lineare Interpolation zwischen definierten kalt- und warm-Farben basierend auf dem Skalarprodukt ($N \cdot L$).	Verbessert die visuelle Wahrnehmung von Geometriedetails durch Temperaturkontraste; Vorrangig verwendet für technische Illustrationen (siehe Abb. 4.6).
Sobel [32, Kap. 3.3]	Diskrete 3×3 Kernel-Texture-Fetches auf Depth- und Normal-Buffern im G-Buffer zur Gradienten- und Kantenerkennung.	Extrahiert und betont Geometrie- sowie Tiefendiskontinuitäten zur klaren visuellen Segmentierung von Objekten im Bildraum (siehe Abb. 4.2b).
Inverted-Hull [3, Kap. 15.2.2]	Verschieben der Oberfläche entlang der Normalen bei gleichzeitiger Umkehrung der gerenderten Flächen.	Generiert eine konsistente, geschlossene Außenkontur im Objektraum durch das vergrößerte Rendern der maskierten Rückseiten (siehe Abb. 4.2a).

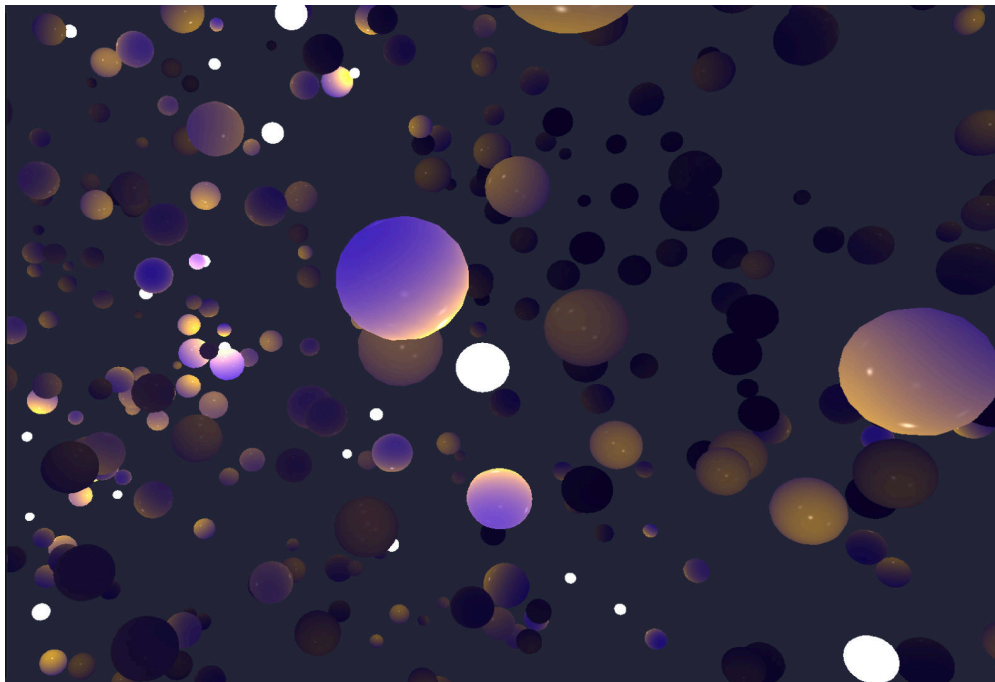


Abbildung 4.6: Gooch-Shading in HyDra-Framework.

Ein inhärentes Problem der selektiven Kuwahara-Filterung in Deferred Pipelines äußert sich bei Objekten mit sehr geringer projizierter Bildschirmfläche. Da bei bildschirmba-

sierten Operatoren die Arbeit auf einer festen Pixel-Nachbarschaft erfolgt, kommt es bei einer Verkleinerung des Objekts (z. B. durch zunehmende Kameraentfernung) oder an den Objekträndern zu einem Verlust oder der inhaltlichen Verfälschung der notwendigen diskreten Abtastpunkte innerhalb der Objektgrenzen [29, Abschn. 5] (siehe Abb. 4.7). Innerhalb des HyDra-Frameworks werden zur Umgehung dieser Effekte jene Samples verworfen, deren Style-ID nicht mit dem Zentrum übereinstimmt, um eine Verrechnung benachbarter Objekte mit unterschiedlicher Stilisierung zu verhindern. Diese werden durch einen konstanten Fallback-Wert in Form der unfiltrierten Basisfarbe des gezeichneten Pixels ersetzt (vgl. Listing 3). Diese Modifikation dient der Gewährleistung einer stilistischen Konsistenz des Kuwahara-Filters auch bei selektiver Anwendung des Effektes.

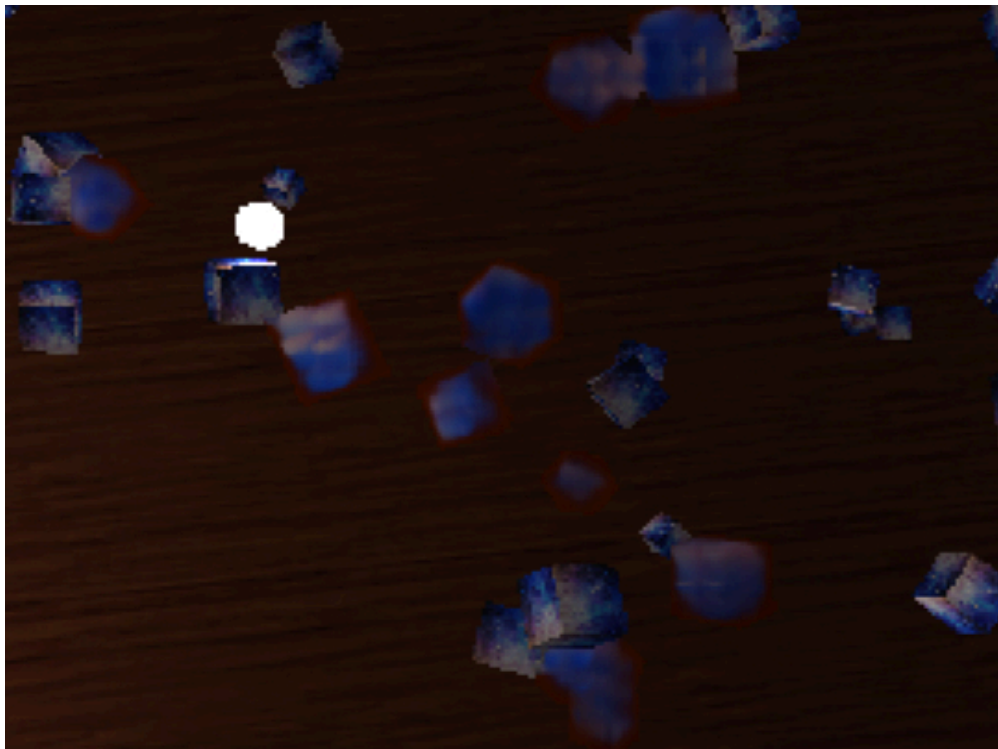


Abbildung 4.7: Kollabieren der selektiven Kuwahara-Bildraumfilterung bei weit entfernten und kleinen Geometrien in HyDra-Framework (vergrößerte Darstellung).

Wie Papari et al. [30, Abschn. II-B] darlegen, leidet die klassische Kuwahara-Filterstruktur unter einer inhärenten mathematischen Instabilität, sobald mehrere Subregionen identische Standardabweichungen aufweisen. Bei kleinen oder weit entfernten Geometrien wird dieser Zustand erzwungen. Da das Objekt nicht mehr genügend Pixel besitzt, um den Kernel zu füllen, dominieren die konstanten Fallback-Werte der Identitäts-Maskierung die Quadranten. Die Varianz der abgetasteten Nachbarpixel sinkt hierbei gegen null, wodurch der Algorithmus die mathematische Basis verliert, eine eindeutige Region zu isolieren.

```

1 // Fetch neighbor's spatial and identity data
2 vec4 neighborNormalData = texture(gNormalTex, sampleUV);
3 int neighborStyleID = int(round(neighborNormalData.a * 255.0));
4 vec3 col;
5 // Validate Neighbor
6 if (neighborStyleID != styleID) {
7     // Invalid: Clamp to the fully lit center pixel
8     col = centerCol;
9 } else {
10    // Valid: Sample actual neighbor lit color + ambient
11    vec3 sampledLit = texture(litSceneTex, sampleUV).rgb;
12    vec3 sampledAlbedo = texture(texture0, sampleUV).rgb;
13    vec3 sampledAmbient = sampledAlbedo * ambientLightStrength;
14    col = sampledLit + sampledAmbient;
15 }

```

Listing 3: GLSL Code zur Optimierung von Kuwahara-Filter-Sampling zur Mitigierung von Kanten-Bleeding Effekten in der Deferred-Volume-Pipeline in Hydra-Framework.

4.4 Telemetrie & Benchmark-Orchestrierung

Nachdem die architektonischen Limitationen und Charakteristika der implementierten Rendering-Pipelines dargelegt wurden, müssen für die detaillierten Performancemessungen in Kapitel 5 die verwendeten Telemetrie- und Benchmark-Orchestrierungsmodule definiert werden, um den Versuchsmethodik und produzierten Ergebnisse zu kontextualisieren.

4.4.1 Szenen- & Zustandsparität

Ein objektiver Vergleich und die Belastbarkeit empirischer Performancedaten der verschiedenen Rendering-Pipelines in der Echtzeitgrafik erfordern die technische Garantie, dass jede Pipeline pro Frame exakt denselben Workload verarbeitet. Wenn eine Architektur durch abweichende Frustum-Culling-Berechnungen oder abweichende Licht-Allokationen weniger Last auf die GPU überträgt, verlieren die Untersuchungen ihre Validität.

Die Systemarchitektur des Hydra-Frameworks basiert zu diesem Zweck auf einem zentralen EngineState (vgl. Listing 4). Dieser geteilte Zustand fungiert als zentrale Quelle für das Verhalten und den Gesamtaufbau der Szene sowie der verwendeten Rendering-Pipelines der gesamten Laufzeitumgebung. Das BenchmarkOrchestrator-Modul wendet vor dem Start und während jeder Testreihe sämtliche Variablenänderungen (wie Instanzenanzahl, Detailgrad, Filter-Radien etc.) exakt einmal auf den globalen Zustand an und variiert im Testverlauf lediglich die untersuchte unabhängige Variable (vgl. Tab. A.3). Wie in Kapitel 4.4.1 dargelegt nutzen alle untersuchten Rendering-Pipelines über die Variablen des

EngineStates hinaus die gleiche Logik bezüglich Sichtbarkeitsprüfung und Geometrieverwaltung unter Verwendung des SceneManager-Moduls. Die Entkopplung von Datenhaltung und Render-Ausführung zielt demnach darauf ab, Divergenzen auf Szenen- und Geometrie-Verwaltungsebene zwischen den verschiedenen Pipelines zu vermeiden.

```

1  // Centralized state container for the current frame
2  struct EngineState {
3      RenderPath activeRenderPath = RenderPath::Forward;
4      // Core Metrics
5      bool use16BitHDR           = Config::DefaultState::Use16BitHDR;
6      int  activeObstacleCount   = Config::DefaultState::ActiveObstacleCount;
7      int  activeLightCount      = Config::DefaultState::ActiveLightCount;
8      float lightIntensity       = Config::EngineSettings::LightIntensity;
9      float ambientLightStrength =
10         ↪ Config::EngineSettings::AmbientLightStrength;
11      int  renderWidth           = Config::EngineSettings::ScreenWidth;
12      int  renderHeight          = Config::EngineSettings::ScreenHeight;
13      // NPR Toggles
14      bool enableOutlines = Config::DefaultState::EnableOutlines;
15      bool enableKuwahara = Config::DefaultState::EnableKuwahara;
16      bool enableGooch    = Config::DefaultState::EnableGooch;
17      bool enableToon      = Config::DefaultState::EnableToon;
18      // Spatial & Stylistic Modifiers
19      int  kuwaharaRadius    = Config::DefaultState::KuwaharaRadius;
20      float kuwaharaIntensity = Config::DefaultState::KuwaharaIntensity;
21      float objectSphereRadius = Config::DefaultState::ObjectSphereRadius;
22      int  currentLodIndex   = Config::DefaultState::CurrentLodIndex;
23      // Architecture & Distribution Flags
24      bool useClusteredStyles = Config::DefaultState::UseClusteredStyles;
25      bool useLightSingularity = Config::DefaultState::UseLightSingularity;
26      bool useNprRoom          = Config::DefaultState::UseNprRoom;
27      bool renderFloor         = Config::DefaultState::RenderFloor;
28      // System Flags
29      bool requestScreenshot = false;
30  };

```

Listing 4: C++ Code des EngineState. Fungiert als Verwaltungselement des zentralen Engine-Zustands innerhalb des Hydra-Frameworks.

4.4.2 Asynchrone Hardware-Telemetrie

Die Erfassung von Ausführungszeiten auf der Host-Seite (CPU) ist für die Bewertung der reinen GPU-Leistung unzureichend, da moderne Grafiktreiber hochgradig asynchron arbeiten [3, S. 1005]. Die CPU sendet Render-Kommandos an eine Warteschlange und setzt die Ausführung unmittelbar fort, noch bevor die GPU das erste Polygon verarbeitet [17, Abschn. *The logical pipeline* 1–2].

Das HyDra-Framework injiziert über `glQueryCounter` asynchrone Zeitstempel direkt in den GPU-Befehlsstrom. Diese *OpenGL Query Objects* ermöglichen eine hardwareseitige Zeiterfassung, um die Ausführungsdauer der einzelnen Render-Pässe isoliert zu messen [28, Kap. 2.14]. Da das unmittelbare Auslesen dieser Markierungen im selben Frame einen kritischen Pipeline-Stall erzeugen würde [3, S. 794], implementiert die `GpuProfiler`-Klasse ein asynchrones Ring-Buffer-Verfahren [27, Kap. 34.2.5], um die Grafikkarten-Abläufe im Normalfall nicht zu blockieren (*Non-Blocking*). Durch die strikte Begrenzung der maximal im Flug befindlichen Frame-Abfragen auf `GpuProfiler::MAX_IN_FLIGHT = 16` verhindert der `BenchmarkOrchestrator` zudem bei hoher GPU-Auslastung ein Überschreiben noch nicht ausgelesener Hardware-Abfragen der GPU durch die CPU. Der kontrollierte „Notfall-Stall“ stoppt die CPU-Befehlsübertragung und stellt somit sicher, dass jeder gerenderte Frame ohne Datenverlust lückenlos und chronologisch in die Telemetrie einfließt (vgl. Listing 5).

```

1  // Process resolved frames
2  while (!pendingFrames.empty()) {
3      const auto &[...] = pendingFrames.front();
4      ...
5      // Check if hardware queries are naturally ready
6      ...
7      const bool allQueriesReady = (geomReady && lightReady && masterReady);
8      const bool emergencyStallRequired =
9          (pendingFrames.size() >= GpuProfiler::MAX_IN_FLIGHT);
10     // Wait for all profilers to report, or force wait to clear capacity
11     if (allQueriesReady || emergencyStallRequired) {
12         double geomMs = 0.0, lightMs = 0.0, totalGpuMs = 0.0;
13         // Pass 'forceWait' down so thread blocks and waits for queries to
14         //    ↪ resolve before extracting
15         if (isDeferred)
16             geomProfiler.TryGetOldestResult(geomMs, emergencyStallRequired);
17         lightProfiler.TryGetOldestResult(lightMs, emergencyStallRequired);
18         masterGpuProfiler.TryGetOldestResult(totalGpuMs,
19             ↪ emergencyStallRequired);
20         telemetryWriter.AppendRow(...);
21         pendingFrames.pop();
22     } else {
23         break; // GPU data for the oldest frame is not ready; Pause writes
24     }
25 }

```

Listing 5: C++ Code zu Ring-Buffer und Notfall-Stall Verfahren während der Benchmark-Orchestrierung in HyDra-Framework

4.4.3 Engine-Determinismus und Frame-Locking

Für die Makro-Benchmark-Synthese (siehe Kap. 5.6) wird im Gegensatz zu den Evaluationen mit statischen Blickwinkeln und unbewegter Geometrie eine Kamerafahrt durch die Szene in mehreren Durchgängen angewandt. Da die untersuchten Rendering-Pipelines innerhalb des HyDra-Frameworks nacheinander geprüft werden, muss eine klare Metrik für den Messrahmen dieser Versuche festgelegt werden. Eine einfache, zeitlich getaktete Kamerafahrt erweist sich an dieser Stelle als unzureichend, da zu erwarten ist, dass die in dieser Arbeit untersuchten Rendering-Pipelines signifikante Unterschiede in den absoluten Ausführungszeiten aufweisen. Eine an Systemzeit gekoppelte Kameraführung würde folglich zwangsläufig zu räumlichen Divergenzen zwischen den Testdurchläufen führen, da die Kamera ihre Durchläufe an zunehmend verschiedenen räumlichen Punkten beenden und beginnen würde. Eine Framezeit gekoppelte Kameraführung hätte einen vergleichbaren Effekt. In jedem einzelnen Frame muss exakt derselbe Szeneninhalte an die Pipeline übergeben werden (räumliche Parität), um valide Vergleichsdaten während einer dynamischen Kamerafahrt zu generieren.

Vor diesem Hintergrund erzwingt das HyDra-Framework absoluten Determinismus während dieser Bewegungssequenzen durch ein striktes „Frame-Locking“. Der Fortschritt der Simulation wird ausschließlich durch einen diskreten Frame-Index gesteuert. Für jede Benchmark-Sequenz aus 5.6 wird ein linearer Interpolationskoeffizient t berechnet.

$$t = \frac{\text{currentFrame}}{\text{targetFrameCount}} \quad (4.2)$$

Dieser Koeffizient bestimmt die exakte Positionierung und Orientierung der Kamera entlang eines vordefinierten Pfades. Der `targetFrameCount` beschreibt hier die gesamte Menge an Frames, die pro Durchlauf/Messung aufgezeichnet werden soll. Diese softwaretechnische Maßnahme führt dazu, dass unabhängig von der tatsächlichen Render-Dauer eines Frames stets mathematisch identische Szeneninhalte berechnet werden.

Das Zusammenwirken dieser deterministischen Bewegungssequenzen und der anschließenden asynchronen Telemetrie-Erfassung bildet den vollständigen Lebenszyklus eines Benchmark-Frames. Die Architektur dieses Prozesses innerhalb des HyDra-Frameworks wird in Abbildung 4.8 zusammenfassend dargestellt.

Die technische Konzeption der Rendering-Pipelines sowie die Etablierung einer präzisen Telemetrie-Architektur, die in diesem Kapitel dargestellt wurden, bilden das strukturelle Fundament der vorliegenden Evaluationssoftware. Im folgenden Kapitel wird diese Architektur als isolierte Testumgebung genutzt, um die übrigen Hardware-Performance orientierten Forschungsfragen anhand systematischer empirischer Versuche quantitativ zu beantworten.

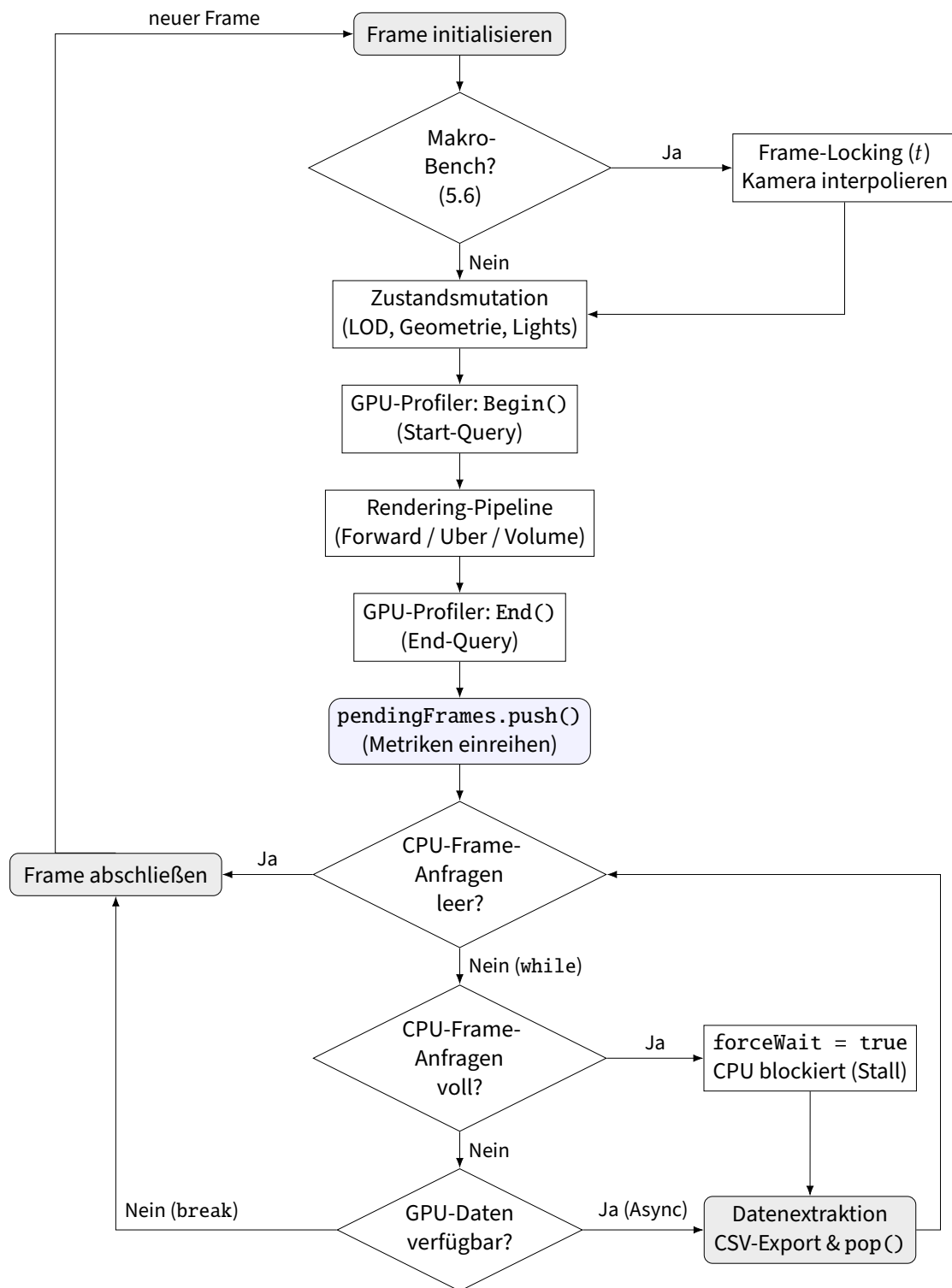


Abbildung 4.8: UML-Aktivitätsdiagramm der Architektur des Benchmark-Lebenszyklus in Hydra-Framework. Der Control-Flow differenziert zwischen statischen Messungen und Frame-Locking (Suite 5.6) sowie zwischen asynchroner Telemetrie-Extraktion und erzwungenem CPU-Stalls bei Überlauf des Ring-Buffers.

5 Evaluationen

Nach der Definition der softwaretechnischen Realisierung und dem Aufzeigen erster Grenzen und Eigenheiten der untersuchten Rendering-Paradigmen in Kapitel 4 werden die implementierten Architekturen isolierten Stresstests unterzogen. Das primäre Ziel dieses Kapitels liegt in der objektiven Quantifizierung der zuvor theoretisch beschriebenen Performance-Engpässe unter systematischen Skalierung von Szenenkomplexität und Stil-Entropie. Der erste Abschnitt definiert die methodischen Rahmenbedingungen sowie die technischen Spezifikationen des Versuchsaufbaus, um eine belastbare Beantwortung der Forschungsfragen zu gewährleisten. Darauf aufbauend erfolgt die schrittweise Datenauswertung anhand spezifischer Kontrollgruppen und der gezielten Untersuchung vermuteter Performance-Engpässe, bevor ein abschließender Makro-Benchmark die gewonnenen Erkenntnisse ganzheitlich evaluiert.

5.1 Evaluationsmethodik & Versuchsaufbau

Das in Kapitel 4 konzipierte Test-Framework fungiert im Folgenden als Messwerkzeug für die empirische Analyse der darin implementierten Rendering-Pipelines. Für die objektive und isolierte Betrachtung der verschiedenen postulierten Performance-Engpässe definiert der vorliegende Abschnitt die methodischen Grundlagen. Dies umfasst die Formulierung von Hypothesen, die exakte Spezifikation des Test-Systems und des Variablen-Mappings sowie die Orchestrierung des Versuchsablaufs.

5.1.1 Zielsetzung und Hypothesen der Untersuchung

Wie in Kapitel 2.1 dargelegt, weisen die theoretischen Architekturparadigmen des Forward-, Deferred-Uber- und Deferred-Volume-Renderings spezifische Leistungsmerkmale auf. Das primäre Ziel der nachfolgenden Testreihen besteht in der isolierten Quantifizierung dieser technologischen Trade-offs. Es sollen die individuellen Kippunkte auf Hardware-Ebene lokalisiert und evaluiert werden. Dadurch soll vermieden werden, eine Pipeline den anderen gegenüber als inhärent überlegen zu präsentieren.

Auf Basis der in Kapitel 2 dargelegten theoretischen Faktenlage werden für die implementierten Rendering-Pipelines spezifische Leistungshypothesen formuliert. Diese Annahmen

gilt es im Folgenden durch gezielte Evaluationen objektiv zu verifizieren oder zu falsifizieren. Die empirische Untersuchung beleuchtet dabei gleichzeitig die hardwaretechnischen Grenzen und strukturellen Schwächen des verwendeten Evaluations-Frameworks HyDra.

Für die als Referenz gewählte Single-Pass-Forward-Rendering-Pipeline werden zwei primäre Leistungsengpässe postuliert. Auf GPU-Ebene wird bei steigender Szenen- und Beleuchtungskomplexität erwartet, dass die Kombination aus geometrischem Fragment-Overdraw und der multiplikativen algorithmischen Komplexität der direkten Beleuchtungsberechnung von $O(\text{Fragmente} \cdot \text{Lichter})$ (vgl. 2.1) zu signifikanten Leistungseinbußen führt (Hypothese H1.1). Hinsichtlich der CPU-Auslastung wird angenommen, dass das CPU-seitige Filtern und Zuordnen von Lichtquellen pro Objekt sowie das hochfrequente Binden objektspezifischer Licht-Uniforms einen signifikanten API-Overhead provoziert, der die CPU-Zuweisungszeit bei anwachsender Instanzenanzahl nicht-linear skalieren lässt (Hypothese H1.2) [5, Abschn. 1].

Für die Deferred-Pipeline mit einfachem Beleuchtungs-Pass (Single-Pass) und monolithischem Über-Shader wird anhand von Kapitel 2.1.2 und 2.2.3 postuliert, dass der signifikanteste Leistungsengpass bei einer hohen räumlichen Stil-Entropie auftritt (Hypothese H2). Wie in Kapitel 2.2.3 theoretisch hergeleitet, erzwingen feingranular gestreute NPR-Effekte divergente Code-Zweige (Warp-Divergence & Registerdruck). Diese kombinierten Hardware-Flaschenhälse zwingen die Architektur zu einer sequenziellen Abarbeitung der Pixel und lassen eine messbare Degradation der Shading-Performance bei stark vermischten Effekten erwarten.

Für die Deferred-Pipeline auf Basis von Light-Volumes (Deferred-Volume) postuliert Hypothese H3, anhand von Kapitel 2.1.2 und 2.2.2, dass diese Architektur eine hohe Resilienz gegenüber räumlicher Stil-Entropie aufweist und die Branching-Problematik des Über-Shaders umgeht. Der primäre Leistungsengpass dieser Methode wird stattdessen in der Sättigung der Speicherbandbreite vermutet (vgl. Kap. 2.2.2).

Im vorliegenden Kapitel werden die untersuchten Limitierungen ausschließlich auf Hardware-Leistungs-Ebene betrachtet. Spezifische harte Implementierungsgrenzen der Pipelines (wie etwa die Uniform-Buffer-Limitierungen der Forward- und Deferred-Über-Architekturen) sowie etwaige künstlerische Qualitätsengpässe fließen nicht in die Messreihen ein. Diese Aspekte wurden bereits in Kapitel 4.2 verortet und evaluiert. Da weiterhin im Rahmen dieser Makro-Evaluation keine herstellereinspezifischen Hardware-Profiler wie *NVIDIA Nsight* eingesetzt werden, um Mikrometriken wie exakte Cache-Hit-Raten oder Thread-Maskierungen auf Siliziumebene auszulesen, werden die architektonischen Engpässe deduktiv aus den gemessenen Rendering-Laufzeiten abgeleitet. Die benannten Ursachen sind daher stark korrelierende, theoretische Erklärungsmodelle für das beobachtete Performance-Verhalten.

5.1.2 Systemkonfiguration und Variablen-Mapping

Im Folgenden werden die Referenzdaten der Testumgebung und des verwendeten Software-Stacks beschrieben. Es werden die für die Testreihen angewendeten unabhängigen Variablen, abhängigen Variablen (Metriken) sowie die entsprechenden Kontrollvariablen dargelegt und definiert. Die referenzierten detaillierten tabellen sind hierzu im Anhang unter A zu finden.

Die in Tabelle A.1 aufgezeigten Hardware-Spezifikationen bestimmen die absoluten physikalischen Leistungsgrenzen des Systems auf dem dies vorliegenden Versuche durchgeführt wurden. Die in Tabelle A.2 dokumentierten Software-Schnittstellen und Treiber-Versionen kontextualisieren die CPU-seitige Code-Basis sowie den grundlegenden API-Overhead.

Nach der Definition der Hardware- und Software-Spezifikationen erfordert das Evaluation-Design eine exakte Abgrenzung der unabhängigen Variablen. Diese Parameter manipuliert das Framework während der Testreihen systematisch. Es sollen Leistungsgrenzen der Hardware isoliert provoziert werden. Tabelle A.3 fasst das entsprechende Variablen-Mapping zusammen.

Im direkten Gegenzug zu den manipulierten Parametern erfasst das Telemetrie-Modul des Frameworks eine Reihe quantitativer Messgrößen. Diese abhängigen Variablen bilden die empirische Datengrundlage der Evaluationen. Die Aggregation der Daten pro Frame sowie deren tabellarische Aufarbeitung dienen der isolierten Beurteilung der Hardware- und Softwareleistung für die spätere statistische Auswertung. Tabelle A.4 beschreibt die extrahierten Leistungs- und Szenenmetriken. Die GPU-Laufzeiten werden dabei über asynchrone Hardware-Queries ermittelt, um ein Blockieren der CPU-Pipeline zu verhindern.

Den Abschluss des Variablen-Mappings bilden die Kontrollvariablen. Diese Parameter definieren die statische Baseline des Frameworks. Diese Werte bleiben konstant, um reproduzierbare Bedingungen für alle Messreihen zu gewährleisten, es sei denn, die jeweilige Test-Suite überschreibt sie explizit als unabhängige Variable. Tabelle A.5 fasst die Standardkonfiguration der Kontrollvariablen zusammen.

5.1.3 Versuchsablauf

Vor der aktiven Datenaufzeichnung durchläuft das System eine obligatorische Warm-up-Phase von zehn Sekunden. Dieser Prozess dient dazu, die Hardware-Caches und Textur-Caches aufzubauen [3, Kap. 18.4, Abschn. *Memory Issues*], [33]. Hierdurch werden inkonsistente Frametimes, die durch asynchrone Ladevorgänge, späte Speicherallokationen oder aufgeschobene Statusänderungen des Grafiktreibers in den ersten Frames entstehen [3, Kap. 18.4.2, Abschn. *State Changes*], für die Messung exkludiert. Während der Wechsel zwischen Rendering-Pipelines oder beim Voranschreiten abhängiger Schritt-Variablen wird ebenfalls eine Warm-up-Pause von zehn Sekunden erzwungen.

Nach der Warm-up-Phase startet die Sampling-Phase mit einer festgelegten Dauer von 1000 Frames. Die Kamera wird eingefroren (Ausnahme: Evaluation 5.6). Das Telemetrie-Modul fragt nun jede der definierten abhängigen Variablen pro Frame ab und akkumuliert die Rohdaten in dedizierten Arrays zur späteren Exportierung.

Die final präsentierten Graphen und Messergebnisse basieren auf arithmetischen Mittelwerten der Sampling-Phase. Identifikation und Bereinigung unvorhersehbarer Micro-Stutterer (*Frame-Rate Spikes*) erfolgt durch die Standardabweichung der Frame-Zeiten [14, Abschn. 8.5.2.2]. Für jede Parameter-Konfiguration bildet die Menge der erfassten Frame-Metriken den initialen Datensatz X . Aus diesem werden gruppenbasiert der empirische Mittelwert μ sowie die Standardabweichung σ berechnet. Ein individueller Messwert $x_i \in X$ wird ausschließlich dann in die finale Kurvengenerierung einbezogen, sofern die folgende Bedingung erfüllt ist:

$$\mu - 3\sigma \leq x_i \leq \mu + 3\sigma \quad (5.1)$$

Datenpunkte außerhalb dieses Intervalls klassifiziert der Algorithmus als statistische Ausreißer und verwirft diese für die finale Auswertung. Dieser Schritt garantiert, dass die später präsentierten Messdaten das rein architektonische und algorithmische Skalierungsverhalten der Rendering-Pipelines abbilden.

5.2 Geometrische Dichte und Overdraw

Im Rahmen des nachfolgenden Kapitels werden Benchmarks und Erkenntnisse präsentiert, die dazu dienen, geometriebedingte Performance-Bottlenecks innerhalb der untersuchten Rendering-Pipelines zu evaluieren. Der Fokus liegt primär auf der Isolation der *Hardware-Rasterize* und der CPU-Draw-Call-Aufwände.

5.2.1 Level of Detail (Mikro-Geometrie)

Das Ziel dieser Testreihe besteht in der Isolation des Hardware-Rasterizers zur Evaluierung der geometrischen Basis-Last. Konkret soll das absolute Limit der Vertex-Verarbeitung (*Triangle Throughput*) identifiziert und der Einfluss des G-Buffer-Speicherdurchsatzes isoliert bewertet werden. Zu diesem Zweck wird die Pipeline einem reinen Geometrie-Stresstest ohne Beleuchtungseinflüsse ($N_{\text{Light}} = 0$) ausgesetzt, um zu prüfen, ob die massiven Speicherschreibvorgänge der Deferred-Pipelines bei hoher Dreiecks-Dichte im Vergleich zur Forward-Pipeline einen signifikanten Bandbreiten-Engpass erzeugen [15, S. 55].

Versuchs-Setup

Unabhängige Variable:	Geometrischer Detailgrad ($LOD \in \{0, \dots, 4\}$)
Skalierung (Dreiecke):	24.000 bis 16,38 Mio. Dreiecke
Kontrollvariablen:	$N_{Inst} = 2.000$, $N_{Light} = 0$, $C_{Floor} = \text{Deaktiviert}$, $Culling = 0\%$, Statische Kamera
Zielgrößen:	GPU-Gesamtlaufzeit (t_{GPU}), Dispatch-Laufzeit (t_{CPU})

Wie aus den Ergebnissen in Abbildung 5.1 zu erkennen ist, verhalten sich die verschiedenen Rendering-Pipelines nahezu identisch und skalieren auch bei erhöhtem Detailgrad (LOD) äquivalent, sowohl in Bezug auf die GPU- als auch die CPU-Auslastung. Weiterhin ist zu beobachten, dass sich die GPU-Gesamtlaufzeit (t_{GPU}) nahezu linear zur absoluten Dreiecksanzahl ($O(N)$) verhält, wohingegen die Dispatch-Laufzeit der CPU weitestgehend konstant ($O(1)$) bleibt.

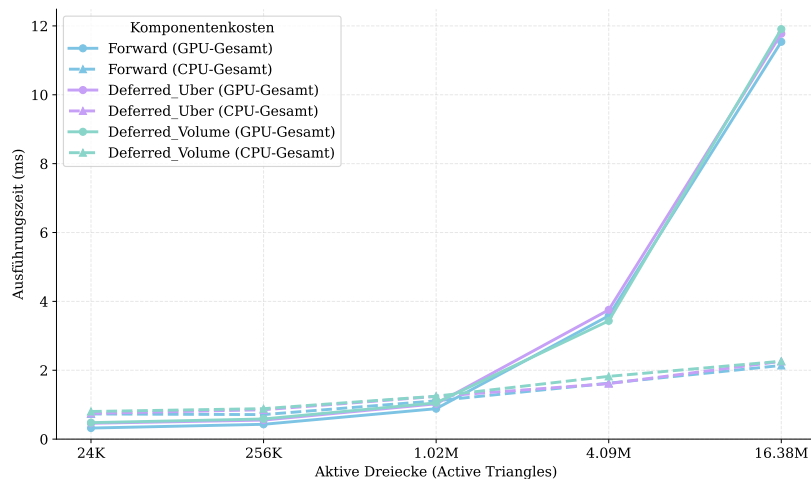


Abbildung 5.1: Messung der GPU- und CPU-Laufzeiten bei einer LOD-Skalierung von 24.000 bis 16,38 Mio. Dreiecken und einer konstanten Objektanzahl von 2.000.

Die ermittelte Laufzeit-Parität zwischen den Pipelines widerlegt die Hypothese der hohen Speicher- und Bandbreitenkosten für reine Geometrieszenarien [3, S. 905]. Das Füllen der breiteren Deferred-Buffer erzeugt gegenüber dem Forward-Buffer keinen limitierenden Bandbreiten-Overhead auf dem VRAM-Bus.

Es sei jedoch methodisch angemerkt, dass eine rein geometrische Skalierung auf konstantem Raum nur bedingt geeignet ist, um die VRAM-Speicherbandbreite zu sättigen. Die extrem hohe Dreiecksdichte bei $LOD = 4$ (16,38 Mio. Dreiecke auf 2.000 Instanzen bei 1080p) führt dazu, dass die Größe der verarbeiteten Dreiecke den Subpixel-Bereich erreicht oder sogar unterschreitet [8, S. 1]. Die Hardware stößt bei der Transformation dieser winzigen Dreiecke an ihre Kapazitätsgrenze, noch bevor eine signifikante Anzahl von Fragmenten erzeugt werden kann, die den VRAM-Bus überhaupt belasten würden [3, Kap. 18.2.3], [19, Folie *Tiny triangles*].

5.2.2 Objektanzahl

Das Ziel der folgenden Testreihe besteht in der Isolation der CPU-Architektur. In der vorliegenden Untersuchung werden die Draw-Call-Generierung sowie die Ausführungszeit des logischen Szenen-Passes (Frustum Culling, Sichtbarkeitsprüfung und Matrix-Updates) durch eine schrittweise Skalierung der Geometrie-Instanzen evaluiert. Die Durchführung des Tests erfolgt ohne Beleuchtungseinflüsse ($N_{\text{Light}} = 0$) und bei minimalem geometrischen Detailgrad ($LOD = 0$, 12-Dreieck-Würfel), um potenzielle GPU-Engpässe zu eliminieren und eine isolierte Betrachtung der Host-Seite (CPU) zu gewährleisten. Es wird primär die Hypothese geprüft, ob der CPU-Overhead das Gesamtsystem limitiert, bevor die GPU vollständig ausgelastet ist, und inwiefern architektonische Unterschiede zwischen den Rendering-Pipelines im Dispatch-Prozess auf der Host-Seite (CPU) existieren.

Versuchs-Setup

Unabhängige Variable:	Instanzen-Anzahl ($N_{\text{Inst}} \in \{1.000, \dots, 50.000\}$)
Dynamische Variable:	Kamera-Distanz (Skaliert proportional zu N_{Inst} zur Vermeidung von Culling-Verzerrungen)
Kontrollvariablen:	$LOD = 0$, $N_{\text{Light}} = 0$, $C_{\text{Floor}} = \text{Deaktiviert}$
Zielgrößen:	Logik-Laufzeit (t_{Logic}), Dispatch-Laufzeit (t_{Dispatch})

Die Ergebnisse in Abbildung 5.2 zeigen, dass sowohl die Laufzeiten des logischen Szenen-Passes (t_{Logic}) als auch die Dispatch-Laufzeiten (t_{Dispatch}) mit der Instanzenanzahl korrelieren, wobei die jeweiligen Steigungsraten signifikant divergieren. Der Anstieg von t_{Dispatch} verläuft zwar linear, verbleibt jedoch selbst bei einer Volllast von 50.000 Instanzen mit ca. 7.8 ms über alle Rendering-Pipelines hinweg stabil und effizient. Im Gegensatz dazu weist t_{Logic} ein deutlich steileres Anstiegsverhalten auf, wobei sich die Forward-Pipeline bei steigender Last zunehmend von den Deferred-Pipelines entkoppelt: In der Traversierung des logischen Szenen-Passes (State-Changes/Culling) generiert die Forward-Architektur eine messbar höhere CPU-Last als die Deferred-Äquivalente.

Die Analyse ergibt, dass die reine API-Befehlsübermittlung an die Grafikkarte bei keinem der untersuchten Rendering-Paradigmen einen primären Engpass darstellt. Darüber hinaus kann beobachtet werden, dass die umfangreichen mathematischen Operationen (Frustum Culling, Model-View-Projection (MVP)-Matrix-Updates) die Datenverarbeitung auf der CPU hier weitaus größer belasten. Das Forward Rendering weist in diesem Kontext eine geringere Skalierbarkeit auf, da es pro Objekt auf der CPU eine Lichtliste evaluieren und binden muss [3, S. 891]. Wie Thaler [11, S. 1] beschreibt, müssen bei Single-Pass-Forward-Architekturen für jedes Objekt zunächst alle beeinflussenden Lichtquellen gesucht werden. Dieser Suchlauf kostet kontinuierlich CPU-Zeit, auch wenn er bei $N_{\text{Light}} = 0$ für alle $N_{\text{Inst}} = 50.000$ Objekte vollständig ins Leere läuft. Das schlechtere Skalierungsverhalten der Forward-Pipeline bei Verarbeitung des logischen Szenen-Passes auf der CPU verifiziert somit Hypothese H1.2.

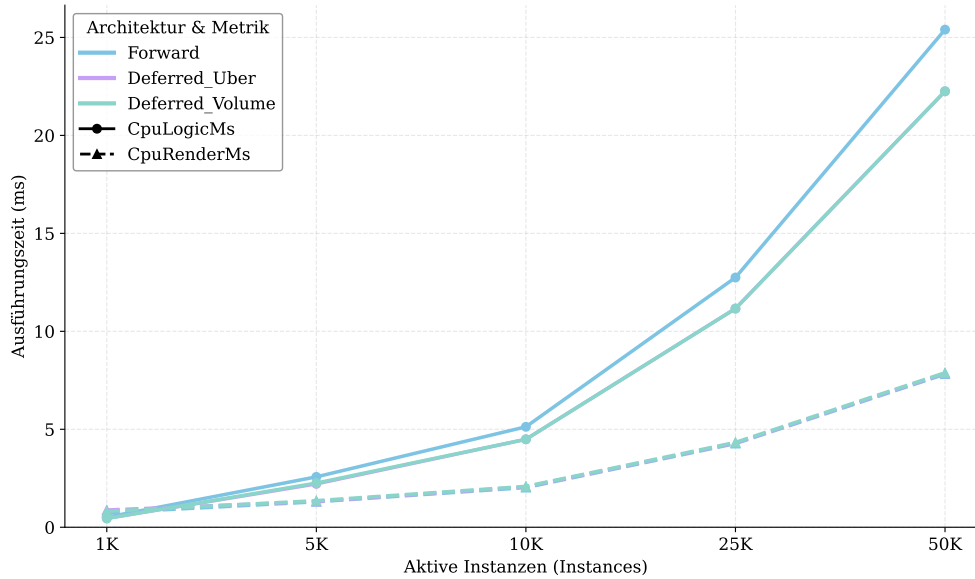


Abbildung 5.2: Messung von Logik-Laufzeit (t_{Logic}) und Dispatch-Laufzeit (t_{Dispatch}) bei einer Skalierung von 1.000 bis 50.000 Instanzen.

Dies stellt gleichzeitig eine architektonische Limitation des genutzten Frameworks dar, da eine alternative Implementierung des Forward Renderings in mehreren Pässen (ein Render-Pass pro Lichtquelle) diese spezifische Suchlogik umgangen hätte (Multi-Pass-Forward) [11, S. 1].

5.2.3 Overdraw-Dichte

Der folgende Versuch zielt darauf ab, den Einfluss extremer geometrischer Tiefenkomplexität (Overdraw) auf die Speicherbandbreite der GPU zu evaluieren. Zu diesem Zweck wird N_{Inst} auf engstem Raum ($r_{\text{Spawn}} = 30,0$) signifikant erhöht, um eine substantielle Z-Achsen-Stapelung zu erzwingen. Aufgrund der Tatsache, dass Deferred Rendering beim Beschreiben des G-Buffers signifikant mehr VRAM-Bandbreite benötigt als Forward Rendering, wird eine Prüfung durchgeführt, ob ein hoher Overdraw-Faktor (O_{Draw}) die Deferred-Pipelines im Vergleich unverhältnismäßig stark ausbremst (Bandbreiten-Sättigung durch massives Überschreiben von Pixeln).

Versuchs-Setup

Unabhängige Variable:	Instanzen-Anzahl ($N_{\text{Inst}} \in \{2.000, \dots, 32.000\}$)
Metrik (X-Achse):	Theoretischer Overdraw-Faktor ($(\approx 1,1 \text{ bis } 18,1)$)
Kontrollvariablen:	Fixer Radius ($r_{\text{Spawn}} = 30,0$), LOD = 1, $N_{\text{Light}} = 0$
Zielgröße:	GPU-Gesamtlaufzeit (t_{GPU})

Obwohl alle analysierten Rendering-Pipelines einen messbaren Anstieg der GPU-Ausführungszeit aufweisen, sind, wie in Abbildung 5.3 zu beobachten, die Abweichungen untereinander

minimal. Des Weiteren ist festzustellen, dass alle Rendering-Pipelines nahezu linear skalieren.

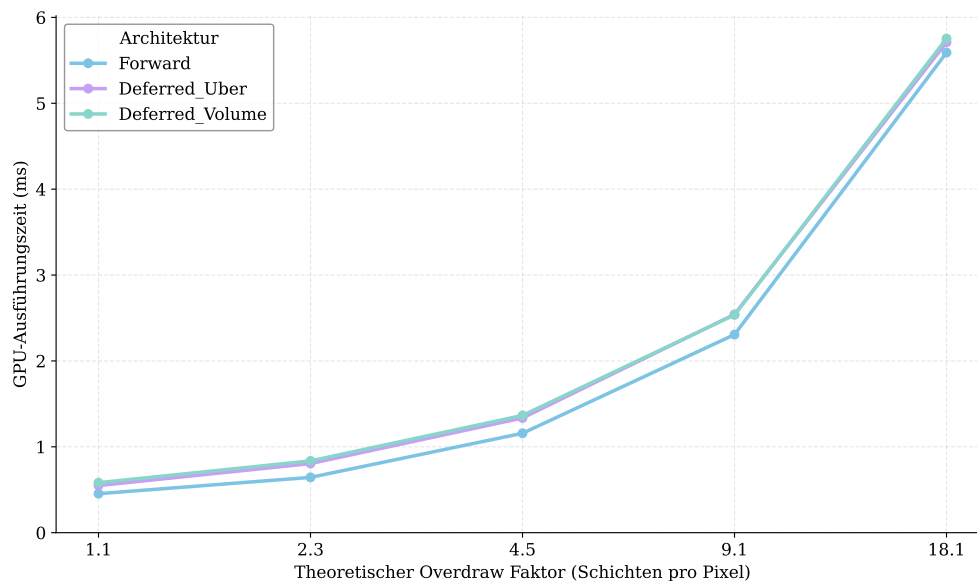


Abbildung 5.3: Messung der GPU-Ausführungszeit in Abhängigkeit vom theoretischen Overdraw-Faktor (1,1 bis 18,1 Schichten pro Pixel) bei einer Skalierung von 2.000 bis 32.000 Instanzen.

Die nahezu deckungsgleichen Kurven demonstrieren, dass die umfangreichen G-Buffer-Schreibvorgänge selbst bei einem äußerst hohen Overdraw keine VRAM-Blockade induzieren. Da opake Geometrie gerendert wird, testet die GPU-Hardware die Tiefe bereits vor dem eigentlichen Fragment-Shader. Es ist zu konstatieren, dass verdeckte Pixel-Fragmente mittels *Early-Z* (vgl. Kapitel 2.2.1) auf Siliziumebene verworfen werden, bevor kostspielige Schreiboperationen in den Framebuffer oder G-Buffer stattfinden. [3, S. 801]

5.3 Bandbreite und Auflösung

In dem nachfolgenden Kapitel werden Benchmarks und Erkenntnisse präsentiert, die dazu dienen, die bandbreitenspezifischen Limitierungen auszuloten. Im Rahmen der Analyse werden die API und die Speicherkosten des G-Buffers sowie der zunehmende ALU-Aufwand des Forward-Renderings betrachtet.

5.3.1 Auflösung

Der folgende Versuch zielt darauf ab, den Einfluss der Framebuffer-Auflösung (und der damit verbundenen Pixel-Anzahl) auf die VRAM-Füllrate und den Shading-Aufwand zu untersuchen. Sowohl die Geometrie als auch die Lichtverhältnisse bleiben auf einen Standard

von $N_{\text{Light}} = 200$ Lichtern und $N_{\text{Inst}} = 5.000$ Objekt-Instanzen festgelegt. Im Rahmen der Evaluierung werden ausschließlich die Auflösung und damit gerenderten Pixel analysiert. Da Deferred Rendering die Beleuchtung strikt im 2D-Screen-Space berechnet, wird erwartet, dass die Kosten an die Pixelanzahl gekoppelt sind [34, Abschn. 9.2].

Versuchs-Setup

Unabhängige Variable:	Basis-Auflösung ($R_{\text{Base}} \in \{480\text{p}, 720\text{p}, 1080\text{p}, 1440\text{p}, 4\text{K}\}$)
Skalierung (Pixel):	409.920 bis 8.294.400
Kontrollvariablen:	$N_{\text{Inst}} = 5.000$, $LOD = 1$, $N_{\text{Light}} = 200$
Zielgrößen:	GPU-Gesamtlaufzeit (t_{GPU})

Bei niedrigen Auflösungen (480p bis 720p) weisen die Architekturen eine relativ geringe Variabilität auf ($t_{\text{GPU}} < 4$ ms). Die Forward-Pipeline weist jedoch eine zunehmende Abnahme der Skalierbarkeit auf. Unter 4K-Auflösung werden Forward-Renderzeiten von nahezu 17 ms erreicht, während die Deferred-Pipelines mit etwa 12 bis 13 ms deutlich effizienter sind (siehe Abbildung 5.4).

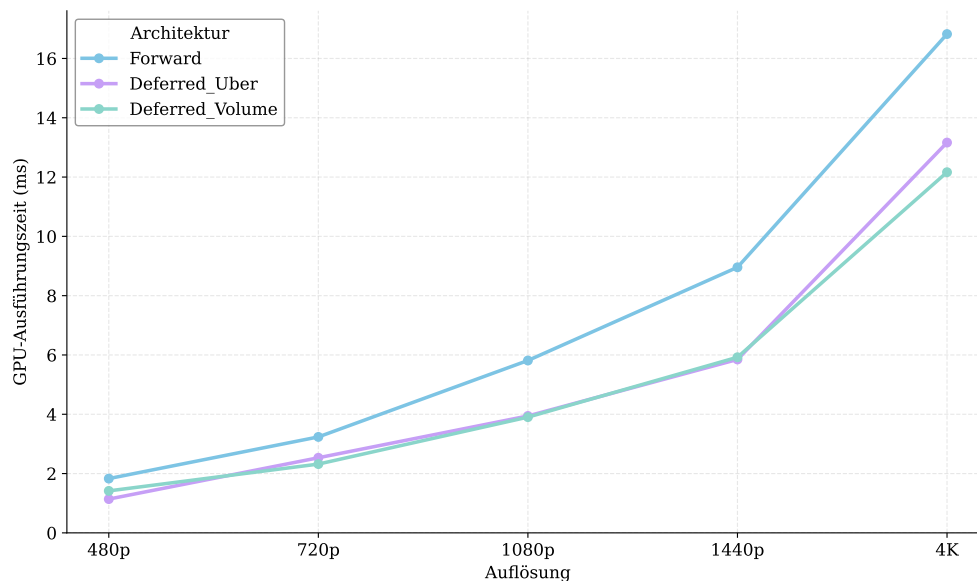


Abbildung 5.4: Messung der GPU-Ausführungszeit in Abhängigkeit von der Framebuffer-Auflösung (480p bis 4K) unter einer statischen Last von 200 Lichtquellen.

Die starken Einbußen von Forward gegenüber den Deferred-Pipelines wird im Zusammenhang mit *Shading-Overdraw* vermutet. Da in diesem Setup $N_{\text{Light}} = 200$ Lichter berechnet werden, führt die Forward-Pipeline komplexe Beleuchtungsmathematik auf Fragmenten aus, die später von anderen Objekten verdeckt werden (vgl. Kap. 2.2.1). Bei einer Anzahl von 8,3 Millionen Pixeln (4K) multipliziert sich diese redundante Rechenlast wesentlich stärker als bei den entkoppelten Deferred-Pipelines (ein ALU-Limit wird schneller erreicht) [12, Abschn. 1]. Diese Vervielfachung der redundanten Rechenlast bei steigender Pixelanzahl stützt die Annahme aus Hypothese H1.1.

5.3.2 Bandbreiten

Ziel der folgenden Evaluation ist die Isolierung der absoluten Speicherbandbreite (Fillrate) und des Basis-Overheads der Rendering-Pipelines. Der Versuchsaufbau gründet auf einem simplen binären Test. Die Szene ist in diesem Fall vollständig leer ($N_{\text{Inst}} = 0$, $N_{\text{Light}} = 0$). Es erfolgt lediglich eine Aktivierung bzw. Deaktivierung eines vollflächigen Boden-Quads (in 1080p und 4K Auflösung). Im Rahmen dessen werden die Bandbreiten-Nachteile während des Schreibens und Lesens der Framebuffer ausgelotet.

Versuchs-Setup

Unabhängige Variablen:	Sichtbarkeit des Bodens ($C_{\text{Floor}} \in \{\text{Deaktiviert}, \text{Aktiviert}\}$), Render-Auflösung ($R \in \{1080p, 4K\}$)
Metrik (Zustände):	0 sichtbare Fragmente vs. 100 % Screen-Space-Abdeckung (≈ 2 Mio. vs. $\approx 8,3$ Mio. Pixel)
Kontrollvariablen:	$N_{\text{Inst}} = 0$, $N_{\text{Light}} = 0$
Zielgröße:	GPU-Gesamtlaufzeit (t_{GPU})

Bei Betrachtung der Abbildung 5.5 lassen sich zwei grundlegende Aspekte feststellen. Die Basislast der Deferred-Pipelines scheint im Vergleich zu der Forward-Pipeline geringfügig höher zu sein. Ansonsten kann keine signifikante prozentuale Steigerung der GPU-Aufwände zwischen den Pipelines in der 1080p Messung festgestellt werden. Bei der Initialisierung des bildschirmfüllenden Boden-Quads zeigen sowohl die Forward- als auch die beiden Deferred-Pipelines eine annähernd identische Skalierung, wobei sich die t_{GPU} -Zeit in allen Fällen um etwa 64 % erhöht. Abbildung 5.6 zeigt jedoch, dass die Deferred-Volume-Pipeline bei einer Auflösung von 4K im Vergleich zu den anderen Single-Pass-Pipelines signifikant schlechter skaliert.

Die Gegenüberstellung der Messreihen bei 1080p und 4K lässt auf einen physischen Hardware-Engpass schließen. Bei einer Auflösung von 1080p ist die VRAM-Bandbreite noch ausreichend, um den Lese- und Schreibverkehr von circa zwei Millionen Pixeln ohne Verzögerungen abzuwickeln. Bei einer Vervierfachung der Datenlast, wie sie bei 4K-Auflösungen erreicht wird, kommt es jedoch zur Sättigung des Speicherbusses. Wäre die Ursache ein reines Rechenproblem der Shader (ALU-Limit), müssten alle Pipelines bei 4K proportional gleichmäßig skalieren. Die signifikant stärkere Einbuße der Deferred-Volume-Pipeline bei der 4K-Auflösung (240 %) im Vergleich zu den Forward- und Uber-Shader-Ansätzen (135 %) lässt sich primär auf den hardwarenahen Overhead des additiven Blendings (vgl. Kap. 2.2.2) zurückführen und liefert ein starkes Indiz für die Hypothese H3. Deferred-Uber und Forward rechnen das finale Beleuchtungsergebnis in einem einzigen Durchlauf aus (Single-Pass-Architektur) und überschreiben den Zielpuffer direkt (nur eine Replace-Operation). Die Multi-Pass-Architektur der Volume-Pipeline hingegen erfordert zwingend den kontinuierlichen Einsatz von additivem Blending zur Akkumulation der Lichtbeiträge [13, S. 9]. Akenine-Möller et al. [3, S. 803] weisen darauf hin, dass derartige Blending-Operationen

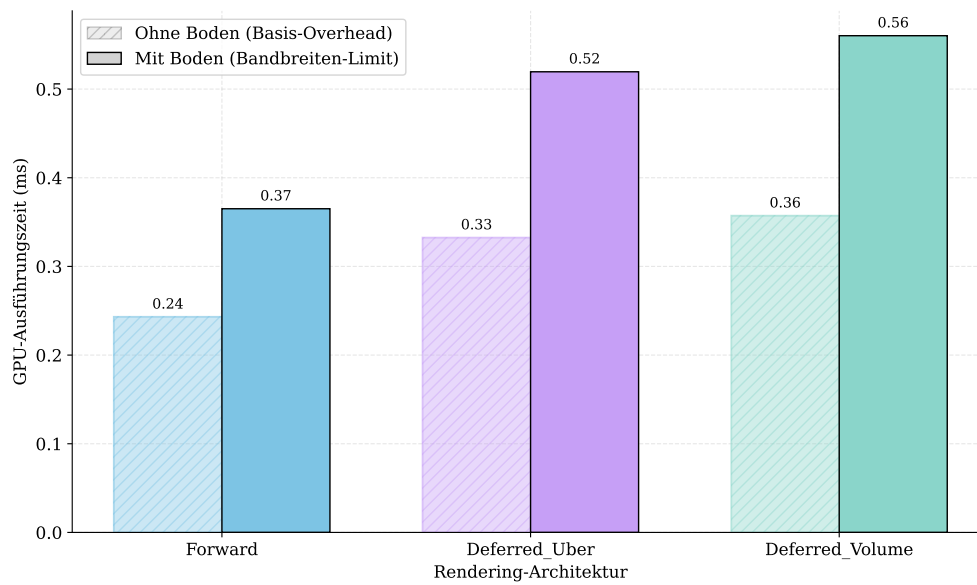


Abbildung 5.5: Vergleich der GPU-Ausführungszeit zwischen keiner Geometrielast (Basis-Overhead) und vollflächiger Screen-Space-Abdeckung bei 1080p Auflösung ohne Beleuchtung.

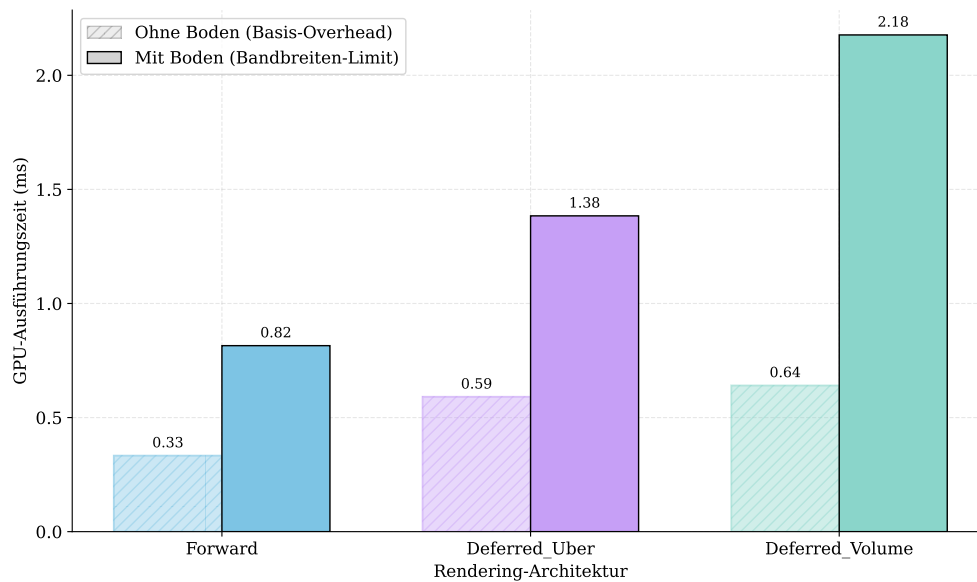


Abbildung 5.6: Vergleich der GPU-Ausführungszeit zwischen keiner Geometrielast (Basis-Overhead) und vollflächiger Screen-Space-Abdeckung bei 4K Auflösung ohne Beleuchtung.

auf dem VRAM-Bus im Vergleich zu einfachen Schreibvorgängen mit signifikant höheren Kosten verbunden sind. In Kombination mit dem simultanen Auslesen der breiten G-Buffer resultiert dieser zusätzliche Lese-Schreib-Zyklus bei 8,3 Millionen Pixeln (4K) selbst bei simplen Ambient-Auswertungen ($N_{\text{Light}} = 0$) in einer Sättigung der Speicherbandbreite.

Trotz dieser starken theoretischen Indizien muss an dieser Stelle eine methodische Einschränkung des entwickelten Frameworks eingeräumt werden. Da die Testumgebung primär auf die Messung von Ausführungszeiten ausgelegt ist, fehlen native Schnittstellen zur direkten Auslesung spezifischer Hardware-Counter der GPU, wie beispielsweise der expliziten Bus-Auslastung. In dieser Evaluation im Besonderen kann die tatsächliche Ursache für den Leistungsabfall der Deferred-Volume-Pipeline somit nicht zweifelsfrei geklärt werden. Die präzise Evaluierung von Bandbreiten- und Bus-Limitierungen verbleibt daher als eine architektonische Schwäche der vorliegenden Testarchitektur.

5.4 Beleuchtung

Im nachfolgenden Abschnitt werden Benchmarks und Erkenntnisse präsentiert, die dazu dienen, die beleuchtungsspezifischen Limitierungen zu ermitteln. Da Deferred Rendering im Bereich der Beleuchtungsberechnung signifikante Vorteile verspricht, sollen im Folgenden die genauen architektonischen Crossover-Points ermittelt werden, ab denen die Deferred-Pipelines die Forward-Pipelines übertreffen. Auf Hardware-Ebene werden im Rahmen der vorliegenden Untersuchung die ALU- und ROP-Limits mithilfe der nachfolgenden Benchmarks evaluiert.

5.4.1 Lichtanzahl

Ziel dieser Evaluation ist die Untersuchung der Skalierbarkeit von Punktlichtern. Dazu wird der klassische Ansatz der UBO-Iteration (Forward und Deferred-Uber) mit der Rasterisierung von Proxy-Geometrie (Deferred-Volume) verglichen. Die Anzahl der Lichtquellen N_{Light} wird dabei logarithmisch von 10 auf bis zu 5.000 skaliert. Wie in Unterkapitel Kapitel 4.2.1 beschrieben, sind sowohl die Forward-Pipeline als auch die Deferred-Uber-Pipeline hinsichtlich der maximal nutzbaren Lichtquellen durch die UBO-Limits auf insgesamt 500 Punktlichter limitiert. Im Rahmen der Untersuchung der allgemeinen Skalierung erfolgt zudem eine Evaluation der Deferred-Volume-Pipeline mit bis zu 5.000 Lichtquellen. Darüber hinaus zeichnet sich das szenische Setup durch eine minimale Überlappung der Lichtquellen und deren Radien aus um Blending-Operationen möglichst gering zu halten.

Versuchs-Setup

Unabhängige Variable:	Anzahl aktiver Lichtquellen ($N_{\text{Light}} \in \{10, \dots, 5.000\}$)
Kontrollvariablen:	Umgebungslicht deaktiviert ($I_{\text{Ambient}} = 0,0$), Licht-Radius reduziert ($r_{\text{Light}} = 1,0$)
Architektonische Limits:	Forward und Deferred-Uber limitiert auf max. 500 Lichter (UBO-Limit)
Zielgröße:	GPU-Gesamtlaufzeit (t_{GPU})

Die Ergebnisse, dargestellt in Abbildung 5.7, zeigen eine nahezu identische Verteilung der Pipelines bei einer Lichtanzahl von ≤ 50 Punktlichtern. Bei geringer Last nimmt die Forward-Pipeline eine leicht überlegene Position ein. Bei einer Last von etwa 100 Lichtquellen lässt sich allerdings ein Kipppunkt zugunsten beider Deferred-Pipelines erkennen. Obwohl die Deferred-Uber-Pipeline eine signifikant geringere Skalierbarkeit aufweist als die Deferred-Volume-Pipeline, übertrifft diese ab dem 100-Lichter-Kipppunkt die Forward-Pipeline zunehmend, wenn auch nur marginal. Bei einer Anzahl von 250 Lichtquellen zeigt sich ein signifikanter Einbruch der Leistung von Forward, während Deferred-Volume eine extreme Effizienz und flache Skalierung aufweist. Erst bei einer Last von 5.000 Lichtquellen überschreitet die Deferred-Volume-Pipeline die Rechenzeit von Forward bei einer zehnmal geringeren Last von 500 Lichtern.

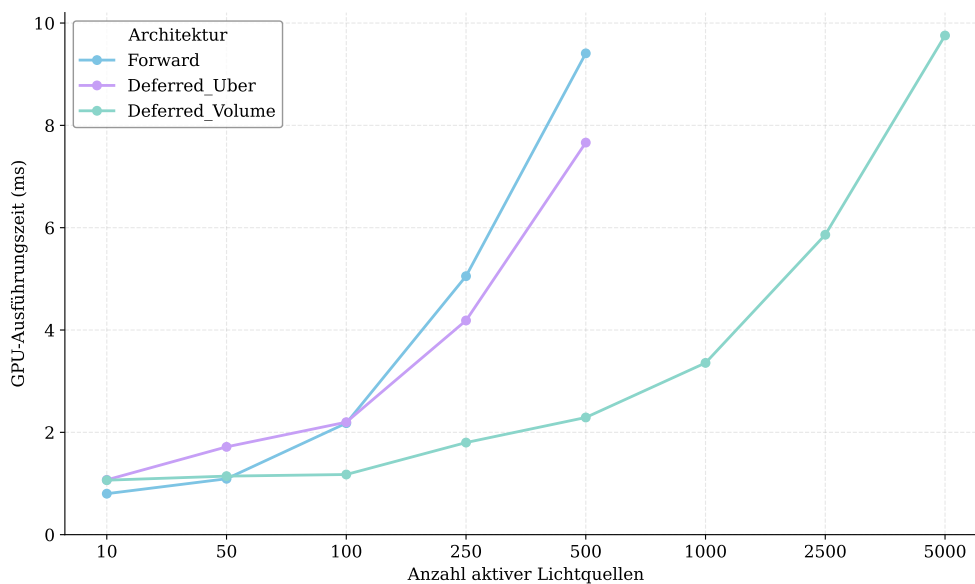


Abbildung 5.7: Messung der GPU-Ausführungszeit in Abhängigkeit von der Anzahl aktiver Lichtquellen (10 bis 5.000).

Aus den Beobachtungen kann abgeleitet werden, dass die Deferred-Pipelines bei steigender Lichtanzahl grundlegend besser skalieren als einfache Forward-Ansätze. Diese Erkenntnis validiert die in der Literatur beschriebene Überlegenheit von Deferred-Rendering im Bereich massiver Lichtevaluationen und bestätigt den primären Existenzgrund dieses Paradigmas im Allgemeinen [11, S. 1]. Obwohl die Deferred-Uber-Pipeline den gleichen Loop

zur Beleuchtungsberechnung ausführt wie der Forward-Ansatz, wird durch das Entkoppeln der Geometrie der Shading-Overdraw effektiv vermieden. Der Einsatz des G-Buffers gewährleistet, dass die Schleife für die Lichtberechnung lediglich für die auf dem Bildschirm tatsächlich sichtbaren Pixel ausgeführt wird. Die signifikante Überlegenheit des Deferred-Volume-Ansatzes lässt sich auf die Effizienz des lokal begrenzten Evaluationsbereiches der Licht-Volumes zurückführen. Anstelle eines volumenunabhängigen Vollbild-Passes wird seitens der GPU lediglich eine Rasterisierung der Licht-Proxy-Geometrien vorgenommen. Die ALU-Last beschränkt sich somit exakt auf die Screen-Space-Projektion, in der das Lichtvolumen die Geometrie tatsächlich beeinflusst [3, S. 886]. Bei einer extremen Last von 5.000 Lichtquellen lässt sich zudem die Hypothese aufstellen, dass das Limit der Deferred-Volume-Pipeline primär durch die VRAM-Bandbreite der ROPs aufgrund des massiven *Read-Modify-Write*-Zyklus des additiven Blendings erreicht wird (siehe Kapitel 5.3.2). Diese Beobachtung verifiziert den algorithmischen Kern von Hypothese H1.1: Die multiplikative Komplexität der direkten Beleuchtungsberechnung zwingt die Forward-Architektur bei steigender Lichtkomplexität unweigerlich in ein massives Performance-Defizit.

5.4.2 Lichtintensität & Overlap

Ziel der folgenden Untersuchungen ist die Evaluation der Pipelines unter Bedingungen extremer räumlicher Dichte der Lichtquellen und des damit verbundenen massiven *Light Overdraws*. Es werden zwei komplementäre Szenarien evaluiert: Test A skaliert den physikalischen Radius von 250 Lichtquellen, um die Bildschirmabdeckung künstlich in die Höhe zu treiben, während Test B bis zu 1.000 Lichtquellen auf exakt dieselbe 3D-Koordinate (Singularität) zwingt, um eine maximale Überlagerung pro Pixel zu garantieren. Dieses Versuchsdesign verspricht die gezielte Provokation und Isolierung des *Read-Modify-Write*-Bandbreitenlimits der Rasterisierungseinheiten (ROPs). Es soll aufgezeigt werden, an welchem Punkt die kontinuierliche Speicher-Akkumulation bei überlappenden Lichtvolumina die reine ALU-Rechenlast der Shader-Schleifen als primärer Performance-Flaschenhals ablöst.

Versuchs-Setup

Test A: Skalierung der Lichtintensität (Radius)

Unabhängige Variable: Licht-Radius-Multiplikator ($r_{\text{Light}} \in \{0,5, \dots, 20,0\}$)

Kontrollvariablen: $N_{\text{Light}} = 250$ (fixiert), Umgebungslicht deaktiviert ($I_{\text{Ambient}} = 0,0$)

Test B: Licht-Singularität (Räumliche Ballung)

Unabhängige Variable: Anzahl aktiver Lichtquellen ($N_{\text{Light}} \in \{10, \dots, 1.000\}$)

Kontrollvariablen: Licht-Singularität aktiviert ($S_{\text{Light}} = \text{Aktiviert}$), $I_{\text{Ambient}} = 0,0$

Zielgröße: GPU-Gesamtlaufzeit (t_{GPU})

Die Analyse der Abbildung 5.8 zeigt, dass die Deferred-Volume-Pipeline bei einem kleinen Radius von 0,5 und dem damit verbundenen minimalen Volume-Overlap den signifikant schnellsten Ansatz darstellt. Wird der Radius-Multiplikator auf 20,0 erhöht und damit die Screen-Space-Abdeckung drastisch gesteigert, führt dies zu einer massiven Zunahme der Laufzeit der Deferred-Volume-Pipeline auf über 8 ms, während die Werte für Forward und Deferred-Uber nahezu konstant bleiben. In Abbildung 5.9 wird ein identisches Bild in forcierter Form dargestellt. Die Deferred-Volume-Pipeline bricht bei einer Skalierung auf bis zu 1.000 Lichtquellen stark ein und erreicht eine GPU-Laufzeit von nahezu 37 ms. Die Deferred-Uber-Pipeline und der Forward-Ansatz zeigen demgegenüber bei einer Auslastung von bis zu 500 Lichtquellen eine signifikant höhere Effizienz und verbleiben stabil unterhalb der 12 ms-Marke.

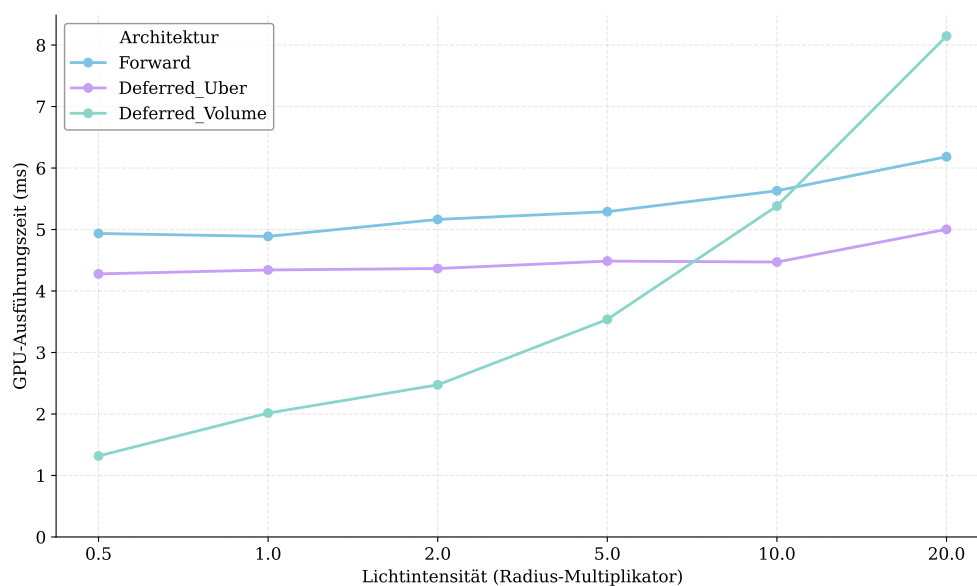


Abbildung 5.8: Messung der GPU-Ausführungszeit in Abhängigkeit von der Lichtintensität respektive dem Radius-Multiplikator (0,5 bis 20,0) bei einer konstanten Anzahl von 250 Lichtquellen.

Diese Untersuchungen forcieren die in den Kapiteln 5.3.2 und 5.4.1 beobachteten Phänomene im Zusammenhang mit den ROP- und Bandbreitenlimits der Deferred-Volume-Pipeline und verifizieren Hypothese H3 abschließend: Sie verorten die größte architektonische Limitierung des Deferred-Volume-Ansatzes in Szenarien mit hochgradig überlappenden Einflussbereichen verschiedener Lichtquellen. Wenn hunderte Licht-Volumina dieselbe Pixelfläche abdecken, zwingt die Deferred-Volume-Pipeline die GPU, für diese einzelnen Pixel hunderte Male den RMW-Zyklus für das additive Blending (ROPs) auf dem Speicherbus auszuführen [13, Folie 9], [14, S. 676] (vgl. Kap. 2.2.2). Der VRAM-Speicherbus kollabiert in der Folge unter der schieren Frequenz dieser redundanten Lese- und Schreibzugriffe.

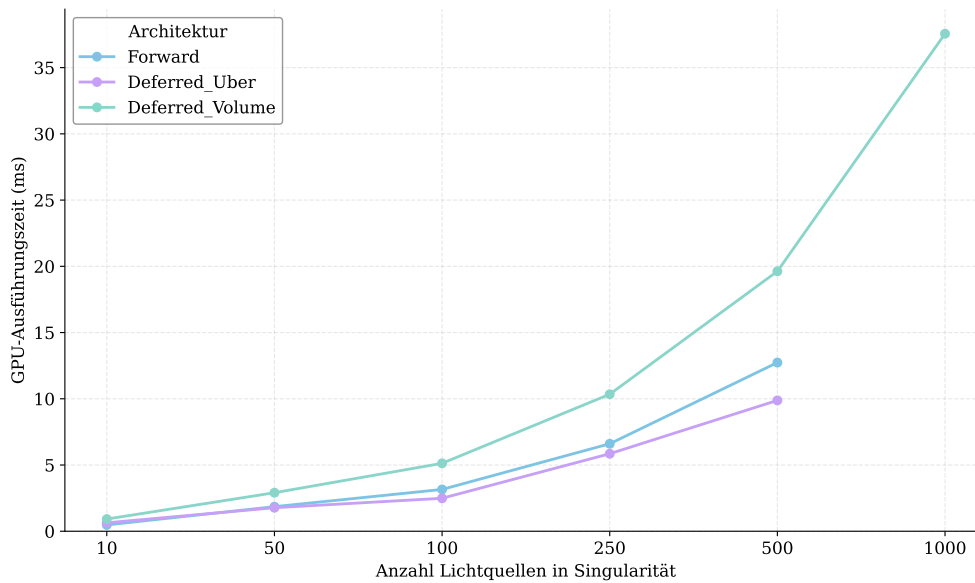


Abbildung 5.9: Messung der GPU-Ausführungszeit in Abhängigkeit von der Anzahl der Lichtquellen (10 bis 1000) bei einer Positionierung aller Lichtquellen in einer räumlichen Singularität.

5.4.3 Shading Overdraw

Wie in Unterkapitel 5.2.3 bereits für die reine Geometrie evaluiert, wird im Folgenden erneut der Einfluss des Overdraws untersucht. In diesem Versuchsaufbau wird die Tiefenkomplexität allerdings unter Zuhilfenahme einer konstanten Beleuchtungslast überprüft. Der Aufbau ist hierbei nahezu identisch zu 5.2.3, mit der Ausnahme von 100 zusätzlich aktiven Lichtquellen. Die Anzahl der Lichtquellen ist dabei so gewählt, dass sie den in Kapitel 5.4.2 aufgezeigten *Light-Volume-Overlap* innerhalb der Deferred-Volume-Pipeline möglichst geringfügig provoziert. Für die Messergebnisse wird erwartet, dass der Forward-Ansatz durch klassischen Shading-Overdraw an Effizienz verliert, da die aufwendige Beleuchtungsrechnung hier untrennbar mit der Geometrieauswertung verknüpft ist (vgl. Kap. 2.1.1). Somit sollte sich der Berechnungsaufwand theoretisch mit jedem überzeichneten Fragment vervielfachen (vgl. Kap. 2.2.1).

Versuchs-Setup	
Unabhängige Variable:	Instanzanzahl ($N_{\text{Inst}} \in \{2.000, \dots, 32.000\}$)
Kontrollvariablen:	$N_{\text{Light}} = 100$, Spawn-Radius reduziert ($r_{\text{Spawn}} = 30,0$), Boden deaktiviert ($C_{\text{Floor}} = \text{Deaktiviert}$)
Metrik (X-Achse):	Theoretischer Overdraw-Faktor ($\approx 1,1$ bis $18,1$)
Zielgröße:	GPU-Laufzeit (t_{GPU}), aufgeschlüsselt nach Komponenten

Abbildung 5.10 zeigt, dass der vorgelagerte G-Buffer-Pass der Deferred-Pipelines in diesem Kontext den primären weitestgehend linearen Skalierungsfaktor bei einer ansteigenden

Instanzdichte für die beiden Deferred-Pipelines darstellt. In der Betrachtung der Forward-Pipeline-Renderzeit ist ein Anstieg der kombinierten Zeit zu verzeichnen (von 1.11 ms auf 5.98 ms), der jedoch nicht so stark ausfällt, wie es die ursprüngliche Annahme vermuten ließe. Der Engine Overhead weist zudem bei sämtlichen Pipelines eine hohe Konstanz auf.

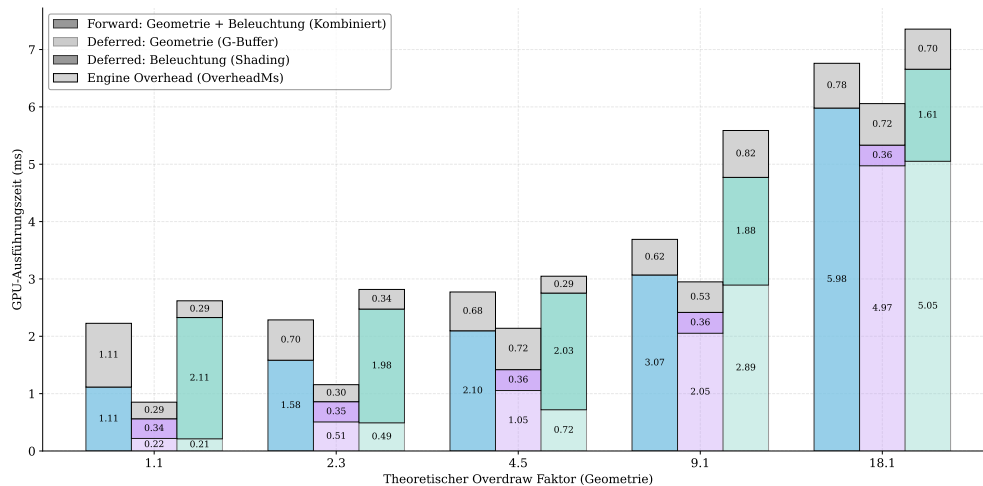


Abbildung 5.10: Aufschlüsselung der GPU-Ausführungszeit nach einzelnen Pipeline-Komponenten in Abhängigkeit vom theoretischen Overdraw-Faktor bei einer konstanten Last von 100 Lichtquellen.

Die Ergebnisse dieser Evaluation widerlegen die weit verbreitete Annahme, dass der Overdraw, insbesondere in Kombination mit Beleuchtung, in einfachen Forward-Pipelines einen der größten Engpässe darstellt [11, Abschn. 1]. Die Ergebnisse dieser Evaluation erzwingen somit eine Relativierung der Hypothese H1.1: Durch hardwarebasierte Optimierungen wie *Early-Z* wird auf modernen Grafikkarten auch ohne dedizierten, softwarebasierten *Z-Prepass* ein signifikanter Teil der redundanten Rechenlast der Beleuchtungsberechnung verdeckter Fragmente vermieden [3, S. 801 u. 803] (vgl. Kap. 2.2.1).

5.5 Hybride Stilisierung & Entropie

In den vorangegangenen Untersuchungskapiteln wurde das grundlegende Verhalten der evaluierten Rendering-Architekturen aufgezeigt und somit zugleich die Fähigkeiten und Grenzen des HyDra-Frameworks demonstriert. Die nachfolgenden Tests widmen sich dem Kern der Forschung und evaluieren die Wechselwirkungen von Stil-Entropie und den allgemeinen Einfluss selektiver NPR-Techniken auf die verschiedenen Pipelines. Der Fokus liegt hierbei, im Speziellen auf der Evaluation von Warp Divergence und Registerdruck.

5.5.1 Stil-Entropie

In den nachfolgenden, zusammengefassten Versuchen werden die Einflüsse von stilistischer Komplexität mittels der Anwendung diverser NPR-Stile analysiert. Neben dem bisher standardmäßig verwendeten Blinn-Phong-Shading finden auch Toon-Shading und Gooch-Shading Anwendung (vgl. Kap. 4.3). In der vorliegenden Untersuchung A wird eine sortierte (niedrige Entropie) mit einer gestreuten (hohe Entropie) Objektverteilung verglichen, wobei die Anzahl der Stile konstant bei drei bleibt. In der weiteren Untersuchung B wird die Anzahl der simultan aktiven Shading-Stile von 1 bis 3 (Blinn, Gooch, Toon) skaliert. In diesem verteilten Szenario (Versuch B) führt die schrittweise Zunahme der Stile zudem zur Isolation des algorithmischen Einflusses der NPR-Effekte. Die vorliegende Untersuchung zielt darauf ab, die Hypothesen der Warp-Divergenz und des Registerdrucks zu analysieren und zu verifizieren. Diese Analyse erfolgt vornehmlich mit Blick auf die Performance der Deferred-Uber-Pipeline.

Versuchs-Setup

Versuch A: Räumliche Entropie

Unabhängige Variable: Räumliche Verteilung ($E_{\text{Spatial}} \in \{\text{Sortiert, Gestreut}\}$)
Kontrollvariablen: $N_{\text{Inst}} = 15.000$, $N_{\text{Light}} = 250$, $r_{\text{Spawn}} = 175,0$, Drei aktive Stile ($S_{\text{Count}} = 3$)

Versuch B: Kombinatorische Degradation

Unabhängige Variable: Anzahl aktiver Rendering-Stile ($S_{\text{Count}} \in \{1, 2, 3\}$)
Kontrollvariablen: Gleicher Aufbau wie Test A, Räumliche Entropie geklustert ($E_{\text{Spatial}} = \text{Geklustert}$)
Zielgröße: GPU-Beleuchtungs-Zeit (t_{Light})

In beiden Ergebnisgraphen (siehe Abbildung 5.11 & 5.13) ist zu erkennen, dass sowohl Forward als auch Deferred-Volume bei gestreuter Verteilung der Stile (hohe Entropie) im Vergleich zu sortierten Verhältnissen extrem stabil bleiben. Auch der erhöhte algorithmische Aufwand in der Lichtberechnung, der durch die komplexeren Aufwände der Toon- und Gooch-Shader entsteht, zeigt nur minimalen Einfluss auf Forward- und Deferred-Volume. Die Deferred-Uber-Pipeline weist in beiden Szenarien allerdings einen signifikanten Einbruch auf und verdoppelt nahezu ihren Aufwand (Anstieg der Beleuchtungsberechnungszeiten um $\approx 85\%$) bei einer gestreuten Stil-Verteilung (hohe Entropie). Abbildung 5.12 zeigt den Versuchsaufbau innerhalb des *HyDra* Frameworks.

In den theoretischen Grundlagen in Kapitel 2.2.3 wurde dargelegt, dass die GPU Instruktionen im SIMT-Verfahren in sogenannten *Warps* verarbeitet. Der signifikante Leistungseinbruch der Deferred-Uber-Pipeline im gestreuten Szenario stellt die praktische Manifestation der *Warp Divergence* dar und verifiziert Hypothese H2 vollumfänglich. Da benachbarte Pixel innerhalb eines Warps unterschiedliche Shading-Modelle evaluieren, wird die Hardware

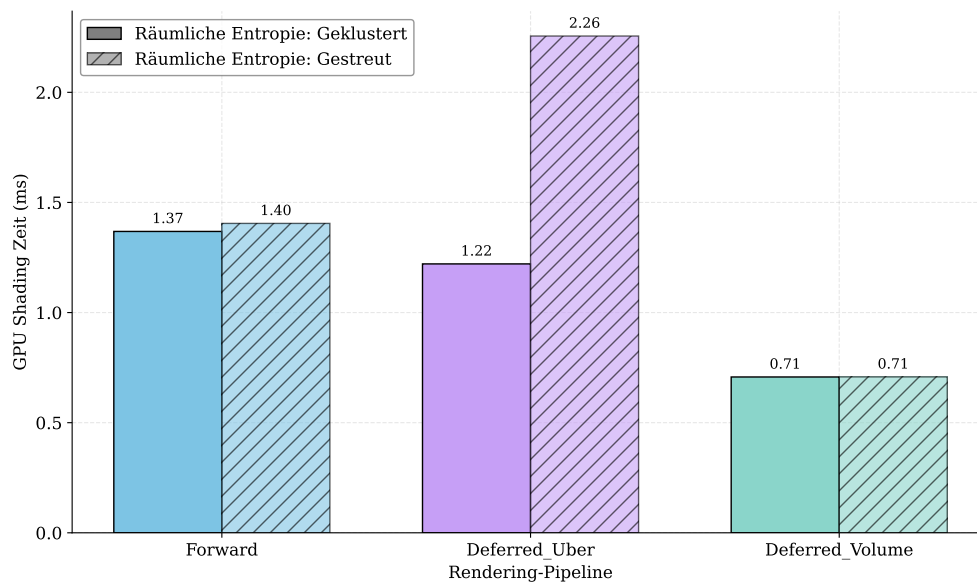


Abbildung 5.11: Messung der GPU-Shading-Zeit in Abhängigkeit von der räumlichen Verteilung (sortiert vs. gestreut) der Rendering-Stile.

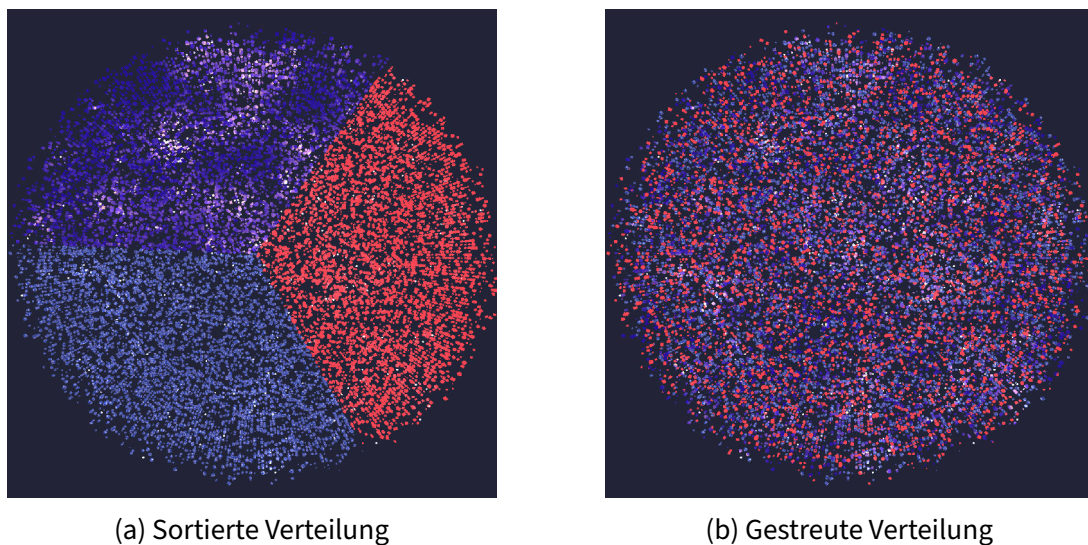


Abbildung 5.12: Vergleich der Stil-Entropie Verteilungen in HyDra-Framework während der 5.5.1 Test-Suite in Test A (räumliche Entropie).

durch diese räumliche Entropie dazu gezwungen, die divergierenden Code-Pfade sequenziell abzuarbeiten. Die Forward- und Deferred-Volume-Architekturen hingegen erweisen sich als systembedingt immun gegen diesen architektonischen Engpass. Die strikte Trennung der *Draw-Calls* nach Material (Forward) oder die Nutzung von mehreren insgesamt kleineren Shader-Pässen (Deferred-Volume) gewährleistet eine Homogenität der aktiven Warps [3, S. 886].

In Bezug auf die lineare Degradation des Deferred-Shaders bei der Kombination mehrerer Stile lässt sich die zusätzliche Hypothese eines erhöhten Registerdrucks aufstellen. Die Implementierung eines zusätzlichen Rendering-Stils im monolithischen Code-Pfad bedingt

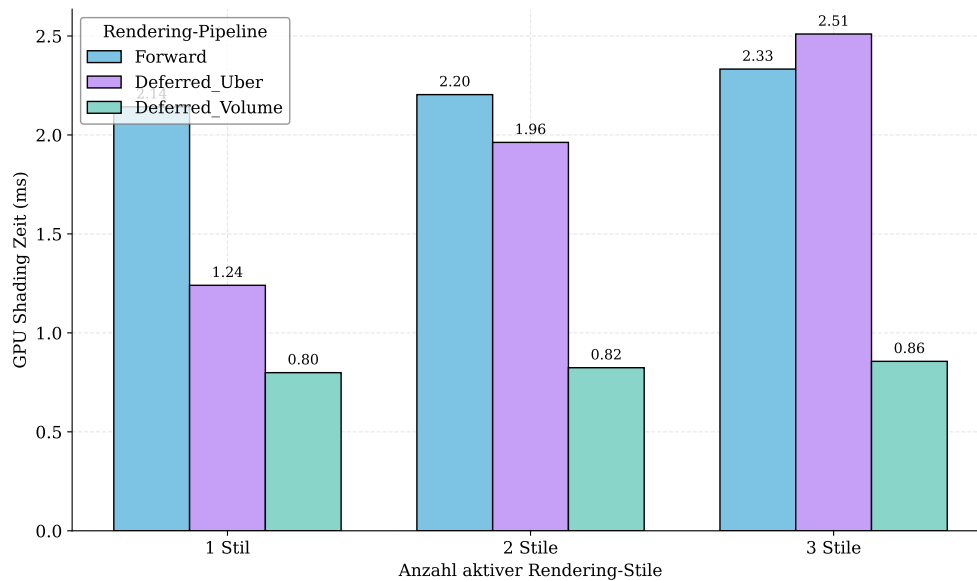


Abbildung 5.13: Messung der GPU-Shading-Zeit in Abhängigkeit von der Anzahl simultan aktiver Rendering-Stile (1 bis 3 Stile).

potenziell die Allokation weiterer Register pro Thread.

An dieser Stelle ist jedoch eine methodische Limitierung der vorliegenden Testarchitektur zu berücksichtigen. Aufgrund der Tatsache, dass die Messung der isolierten Beleuchtungs-Laufzeit (t_{Light}) keine Low-Level-Telemetriedaten liefert, kann diese spezifische Ursache nicht abschließend empirisch verifiziert werden. Ohne den Einsatz dedizierter Hardware-Profiler lässt sich nicht zweifelsfrei differenzieren, inwieweit ein erhöhter Registerdruck zum Leistungsabfall der Deferred-Uber-Pipeline beiträgt.

5.5.2 Kernel Bandbreiten Limits

Der folgende Versuch fokussiert sich ausschließlich auf die Deferred-Pipelines (Deferred-Uber und Deferred-Volume). Die vorliegende Untersuchung widmet sich der Analyse der Einflüsse kernelbasierter NPR-Effekte. Durch die Tatsache, dass solche kernelbasierten Effekte in der Forward-Pipeline auf Objektebene nicht realistisch anwendbar sind (vgl. Kapitel 4.2.1) und nur auf gesamter Screen-Space-Ebene anwendbar sind, kann keine Zielbild-Parität erreicht werden. Ein aussagekräftiger Vergleich mit den Deferred-Pipelines ist somit nicht möglich. Für die nachfolgende Evaluation wird der implementierte Kuwahara-Filter herangezogen. Der Suchradius (R) des stilisierenden Kuwahara-Filters wird sukzessive von 2 auf bis zu 16 Pixel erweitert. Da der Filter im zweidimensionalen Raum agiert, wächst die Anzahl der Textur-Samples quadratisch ($O(R^2)$).

Versuchs-Setup

Unabhängige Variable:	Kuwahara-Kernel-Radius ($R \in \{2, 4, 8, 12, 16\}$)
Kontrollvariablen:	Alle Objekte nutzen Kuwahara, $N_{\text{Inst}} = 15.000$, $N_{\text{Light}} = 250$, Räumliche Entropie gestreut ($E_{\text{Spatial}} = \text{Gestreut}$)
Architektur-Limitierung:	Forward-Pipeline übersprungen (keine native Kuwahara-Unterstützung)
Zielgröße:	Beleuchtungs-Laufzeit (t_{Light})

Wie in Abbildung 5.14 zu erkennen ist, verhalten sich Deferred-Uber und Deferred-Volume bei kleinen Radien ($R = 2$ bis $R = 4$) mit einer Zeit von 1.8 ms bis 3.0 ms nahezu identisch. Ab einem Kuwahara-Radius von 8 (Probe der umliegenden Pixel) lässt sich allerdings ein langsamer Trend zugunsten der Deferred-Uber-Pipeline erkennen. Bei einem Radius von 16 ist Deferred-Uber um fast ≈ 2.5 ms schneller als Deferred-Volume bezüglich der Beleuchtungs-Laufzeit. Die folgende Darstellung der steigenden Komplexität bei Erhöhung des Kuwahara-Radius dient der Vollständigkeit sowie der kontextuellen Einordnung der Graphen und der Effizienz der Rendering-Pipelines: Da die Anzahl der Texturzugriffe T im zweidimensionalen Filterraum quadratisch mit dem Radius R skaliert, lässt sich der theoretische Komplexitätszuwachs über das Verhältnis der Kernel-Flächen bestimmen. Es ergibt sich beim Sprung von $R = 2$ (27 Samples) auf $R = 16$ (1.175 Samples) der exakte Skalierungsfaktor:

$$F = \frac{T(16)}{T(2)} \approx 43,5$$

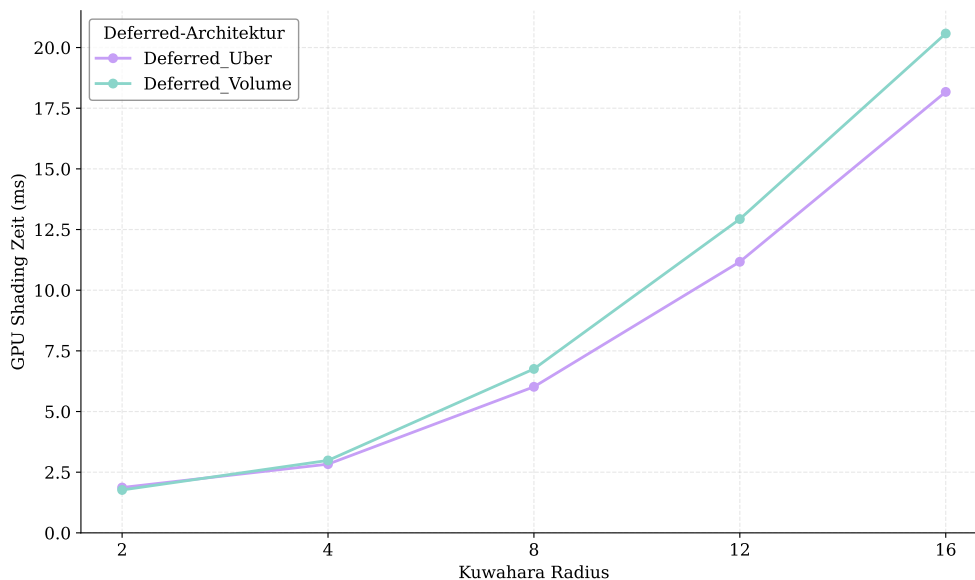


Abbildung 5.14: Messung der GPU-Shading-Zeit in Abhängigkeit vom Kuwahara-Kernel-Radius (2 bis 16).

Die Tatsache, dass die Deferred-Volume-Pipeline im Vergleich zur Deferred-Uber-Pipeline eine geringere Effizienz aufweist, lässt sich auf eine algorithmische Asymmetrie in der

Filter-Implementierung zurückführen. Wie bereits in Kapitel 4.2.2 dargelegt, erfordern die beiden Architekturen unterschiedliche Ansätze bei der Evaluierung des Kuwahara-Filters. Die Deferred-Über-Pipeline operiert mittels *Pre-Lit*-Kuwahara-Filter, um einen algorithmischen Ineffizienz-Kollaps durch redundante Beleuchtungsberechnungen pro Nachbarpixel zu verhindern. Sie liest innerhalb der $O(R^2)$ -Schleife ausschließlich Basis-Materialdaten (Albedo und Style-IDs) aus und berechnet die aufwendige Beleuchtung erst nach Abschluss des Filters für das finale, gemittelte Fragment. Die Deferred-Volume-Architektur appliziert den Filter im *Resolve*-Pass hingegen auf das bereits akkumulierte Licht der Szene (*Post-Lit*). Dies ermöglicht zwar eine „echtere“ Darstellung des Kuwahara-Effektes, erzwingt jedoch innerhalb der innersten Kuwahara-Schleife zusätzliche Texturlesezugriffe auf den *Light Accumulation Buffer* (vergleiche Kapitel 4.2.3).

Über die Auswirkungen der asymmetrischen Implementationen des Filters hinaus kann kein signifikanter Unterschied in der Handhabung von Kernel-Effekten zwischen den untersuchten Deferred-Pipelines festgestellt werden.

5.6 Makro-Benchmark-Synthese

In der finalen Testreihe erfolgt die Integration aller übergeordneter Einflüsse aus den vorangegangenen Benchmarks in ein umfassendes praxisnahes Szenario, um die Interaktion der bislang untersuchten Effekte in ihrer Gesamtheit zu analysieren. Die nachfolgenden Evaluationen werden in einem Schritt zusammengefasst, bestehen allerdings aus drei verschiedenen Einzelpässen, in welchen die verschiedenen Einflüsse von Geometrie, Beleuchtung und selektiver Silisierung gemeinsam betrachtet werden.

Die Makro-Benchmark-Synthese nutzt einen deterministischen Kamera-Flythrough, um die Leistungsfähigkeit der Architekturen in einem praxisnahen Produktionsszenario zu evaluieren (vgl. Kap 4.4). Das zugrunde liegende Szenario generiert eine dichte, prozedurale Umgebung mit 15.000 verteilten Instanzen bei einem konstanten Detailgrad ($\text{LOD} = 1$) und vollflächiger Bodenabdeckung. Dabei ist die räumliche Entropie der Rendering-Stile bewusst auf eine gestreute Verteilung konfiguriert. Der programmierte Kamera-Flug erstreckt sich über 3.000 Frames und durchquert die Szenerie auf einem festgelegten Pfad. Dabei werden kontinuierlich wechselnde Sichtfeldkomplexitäten, stark schwankende Frustum-Culling-Raten sowie variable Tiefen- und Lichtüberlappungen erzwungen, um eine ganzheitliche und dynamische Belastung der Pipelines zu gewährleisten. Die Evaluation erfolgt in drei isolierten, aufeinander aufbauenden Pässen zur exakten Dekonstruktion der Leistungskosten. Die Ergebnisse werden kombiniert dargestellt. In Pass 1 (*Geometry Baseline*) wird vollständig auf Beleuchtung und Stilisierung verzichtet, um die reine Grundlast der Geometrieverarbeitung und G-Buffer-Initialisierung zu erfassen. In Pass 2 (*Shading Tax*) werden 250 dynamische Punktlichter als Umgebungsbeleuchtung hinzugefügt, um die Effizienz der additiven Lichtakkumulation zu messen. In Pass 3 (*NPR-Parity*) werden zusätzlich

die hybriden Stilisierungen (Toon-Shading, Gooch-Shading und Outlines; vgl. Kap. 4.3) aktiviert, um die kombinierte Auswirkung von Beleuchtungs- und Entropie-Last zu evaluieren. Als zentrale Metrik wird in allen Messungen die akkumulierte GPU-Gesamtlaufzeit (t_{GPU}) herangezogen.

Versuchs-Setup	
Versuchs-Typ:	Dynamischer Kamera-Flythrough (3.000 Frames Gesamtdauer)
Kontrollvariablen:	$N_{\text{Inst}} = 15.000$, $r_{\text{spawn}} = 150,0$, $LOD = 1$, Boden aktiviert, Gestreute Stile ($E_{\text{Spatial}} = \text{Gestreut}$)
Pass 1 (Geometry Baseline):	$N_{\text{Light}} = 0$, Alle NPR-Effekte deaktiviert
Pass 2 (Shading Tax):	$N_{\text{Light}} = 250$, Alle NPR-Effekte deaktiviert
Pass 3 (NPR-Parity):	$N_{\text{Light}} = 250$, Gooch, Toon und Outlines aktiviert
Zielgröße:	GPU-Gesamtlaufzeit (t_{GPU})

Die Betrachtung der akkumulierten Einzelpässe in Abbildung 5.15 schlüsselt die architektonischen Kostenstrukturen der Rendering-Pipelines detailliert auf. Im ersten Pass (Geometry Baseline) erweist sich die Forward-Architektur mit einer initialen Ausführungszeit von 0.51 ms als minimal effizienter als die Deferred-Architekturen. Die Deferred-Pipelines weisen mit 0.68 ms (Deferred-Uber) respektive 0.77 ms (Deferred-Volume) die erwartete architektonische Grundlast auf, die durch die initialen Schreibvorgänge in den G-Buffer bedingt ist (vgl. Basislast in 5.3.2). Unter der additiven Zuschaltung von 250 Lichtquellen im zweiten Pass (Shading Tax) kollabiert die Forward-Pipeline unter der multiplikativen Instruktionslast der Lichtberechnungen und verzeichnet eine massive Zusatzlast von +1.69 ms. Die Deferred-Pipelines profitieren hier signifikant von der zweidimensionalen Screen-Space-Beleuchtung und fangen die Rechenlast mit 0.72 ms (Deferred-Uber) sowie +1.01 ms (Deferred-Volume) deutlich effizienter ab (vgl. Ergebnisse aus 5.4.1). Die Aktivierung der gestreuten NPR-Stile im dritten Pass (NPR-Parity) offenbart schließlich die architektonischen Schwächen des Deferred-Uber-Shaders, der durch *Warp Divergence* und Registerdruck eine Zusatzlast von 0.10 ms erfährt (vgl. 5.5.1). Das Deferred-Volume-Rendering erweist sich mit einer minimalen Steigerung von lediglich +0.03 ms als nahezu immun gegen diese stilistische Entropie, während die Forward-Pipeline um weitere +0.17 ms degradiert. Der zeitliche Verlauf des Kamera-Flythroughs in Abbildung 5.16 bestätigt diese Beobachtungen über einen Zeitraum von 3.000 Frames. Es lässt sich deutlich erkennen, dass die Leistungskurven mit zunehmender Komplexität des Sichtfelds ansteigen. Sobald die Kamera in die Objekt- und Licht-Wolke eintaucht, können die jeweiligen Spitzen beobachtet werden.

Die Ergebnisse dieser Makro-Synthese vereinen die in den vorangegangenen Evaluierungskapiteln isoliert gewonnenen Erkenntnisse in einem kohärenten Gesamtbild. Als primäres

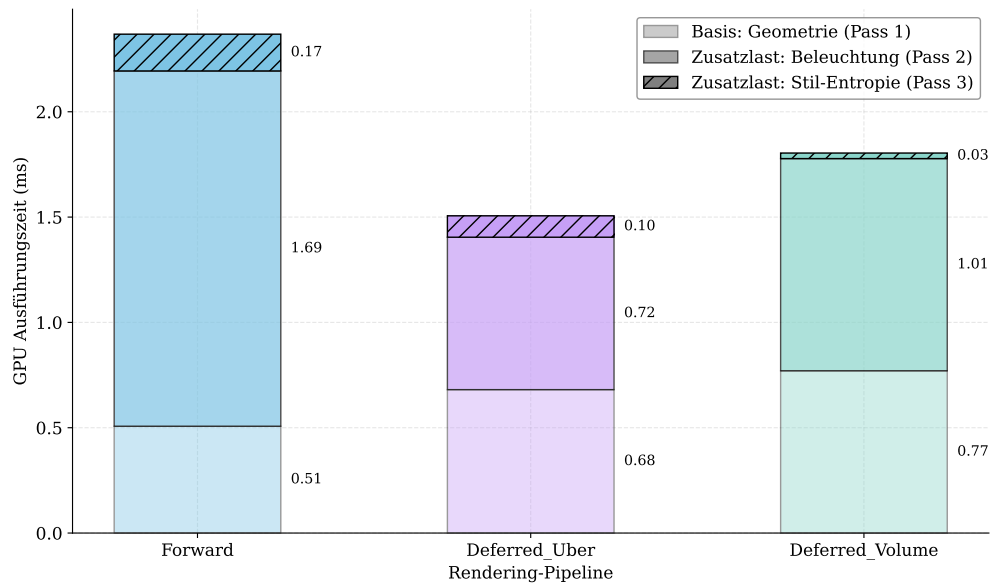


Abbildung 5.15: Aufschlüsselung der GPU-Ausführungszeit nach Pipeline-Komponenten (Geometrie, Beleuchtung und Stil-Entropie).

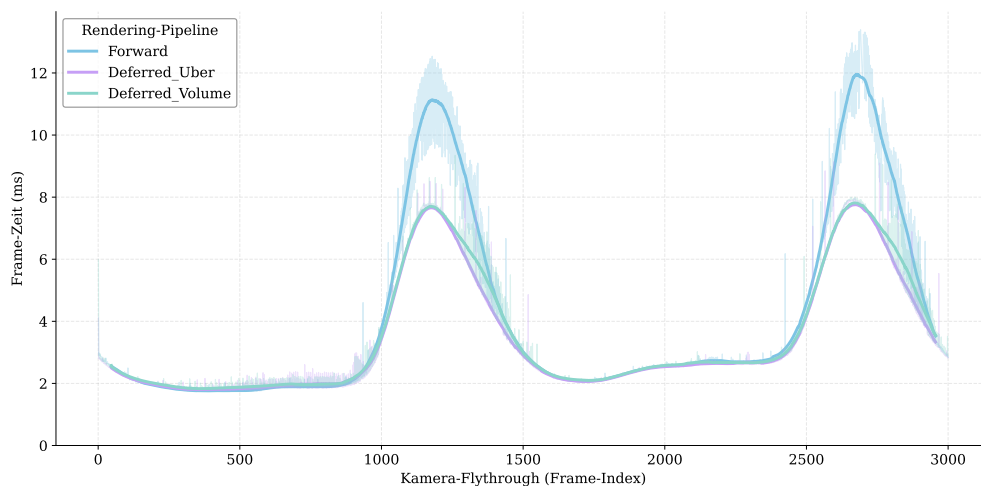


Abbildung 5.16: Verlauf der Frame-Zeit über die Dauer von 3000 Frames des Kamera-Flythroughs unter kombinierter Geometrie-, Beleuchtungs- und Stil-Last.

Vorab-Fazit lässt sich konstatieren, dass es im Kontext der untersuchten Paradigmen keine universell überlegene Pipeline-Architektur gibt. Die Leistungsfähigkeit der einzelnen Architekturen unterliegt einer dynamischen Veränderung, die in direkter Abhängigkeit zur dominanten Szenenlast steht. Die naive Forward-Pipeline zeigt eine hohe Effizienz bei der reinen Geometrieverarbeitung und einer geringen Lichtdichte (vgl. Ergebnisse aus 5.2). Aufgrund der untrennbaren Verknüpfung von Geometrieauswertung und Beleuchtungsrechnung kollabiert sie jedoch unter der multiplikativen Instruktionslast, sobald lichtstarke Produktionsszenarien mit hunderten Lichtquellen gefordert werden (vgl. Ergebnisse aus 5.4.1). Die einfache Deferred-Uber-Architektur löst dieses Beleuchtungsproblem effizient im Screen-Space, scheitert jedoch architektonisch an der *Warp Divergence*, sobald eine hohe stilistische Entropie durch divergierende NPR-Code-Zweige erzwungen wird (vgl. Er-

gebnisse aus 5.5.1). Die Deferred-Volume-Architektur umgeht diese Branching-Problematik zwar durch die strikte architektonische Isolierung der Code-Pfade und Verwendung von Proxy-Geometrien für die Punktlichtquellen, weist jedoch fundamentale Schwächen in der Speicherbandbreite auf. Ihr primärer architektonischer Engpass manifestiert sich auf dem VRAM-Speicherbus, der durch zahlreiche additive RMW-Zyklen bei extremer räumlicher Überlappung der Lichtquellen gesättigt wird (vgl. Ergebnisse aus 5.4.2).

6 Diskussion

Dieses Kapitel führt die theoretischen Anforderungen und architektonischen Grenzen der untersuchten Rendering-Architekturen aus Kapitel 4 sowie die praktischen Messergebnisse aus Kapitel 5 zusammen und stellt die Ergebnisse in Zusammenhang mit den in Kapitel 1.2 formulierten Forschungsfragen, um diese zu beantworten. Daraus resultierend wird das finale heuristische Entscheidungsmodell abgeleitet und anhand der erarbeiteten Ergebnisse in Form von Umsetzungsempfehlungen dargelegt. Abschließend werden die in dieser Arbeit gesammelten Erkenntnisse, getroffenen Entscheidungen und umgesetzten Maßnahmen einer kritischen Reflexion unterzogen.

6.1 Zusammenführung und Interpretation der Ergebnisse

Die in Kapitel 1.2 dargelegten Forschungsfragen lassen sich für die in dieser Arbeit gewählten Rendering-Pipelines, deren Implementation die absoluten Basis-Architekturen der jeweiligen Rendering-Paradigmen darstellen, individuell beantworten.

Wie in Kapitel 3 dargelegt und Kapitel 5.1.1 postuliert, wird das klassische Single-Pass-Forward-Rendering auf Performance-Ebene theoretisch primär durch zwei Flaschenhälse definiert: Fragment-Overdraw und die Verarbeitung großer Anzahlen von Lichtquellen. Wie die Ergebnisse aus 5.2.3 und 5.4.3 darstellen, wird auf Hardwareebene moderner GPUs bereits ein extrem großer Teil an Overdraw-Engpässen mitigiert. Auch wenn Overdraw, wie Thaler [11, Abschn. 1, Punkt 5] darlegt, nie gänzlich eliminiert werden kann, sind die inhärenten Funktionen moderner GPUs bereits ausreichend, um auch ohne zusätzliche Algorithmen, wie ein dedizierter Z-Prepass, Overdraw als primären Bottleneck weitestgehend zu umgehen. Versuch 5.7 bestätigt hingegen die grundlegende These des signifikanten Bottlenecks hinsichtlich extrem hoher Lichtquellen. Weiterhin stellt Versuch 5.2.2 den wenig evaluierten Bottleneck auf CPU-Ebene für Forward-Pipelines in Form von ansteigende CPU-Dispatch-Zeiten durch State-Changes, Culling und Lichtquellen-Suchläufe. Wie in Abbildung 5.2 dargestellt, ist dieser Effekt allerdings vergleichsweise weniger signifikant für die Performance des Gesamtsystems.

Die Beantwortung der grundlegenden Frage nach der Umsetzung von Forward-Pipelines bezüglich selektiver Stilisierung wirft, wie in Kapitel 4.2.1 beschrieben, eine grundlegende Restriktion auf. Forward-Pipelines weisen zwar in der reinen Verwaltung verschiedener Shader und Materialien signifikante praktische Vorteile gegenüber Deferred-Paradigmen

auf [34, Abschn. 9.2], aufgrund ihrer grundlegenden Eigenschaft, Shading-Operationen auf Objektebene (Object Space) darzustellen, sind sie jedoch nicht in der Lage, klassische Bildraum (Screen-Space) basierte Verfahren, wie etwa Kernel-basierte Effekte, selektiv darzustellen. Wie in Kapitel 4.2.1 aufgezeigt, ist es je nach Anwendungsfall jedoch möglich, dass vergleichbare NPR-Effekte auf Objektebene existieren, die ersatzweise verwendet werden können (beispielsweise die Verwendung von Inverted-Hull statt Sobel-Outlines).

Eine weitere Einschränkung der untersuchten Forward-Pipeline abseits von Performance und selektiver Stilisierungen ist die begrenzte Anzahl maximal darstellbarer Lichtquellen durch *Uniform-Buffer-Limits*. Diese Begrenzung lässt sich allerdings weniger auf die Natur von Forward-Rendering-Architekturen im Allgemeinen zurückführen und mehr auf die Natur von Single-Pass-Architekturen [11, Abschn. 1]. Diese Einschränkung trifft also ebenfalls auf die untersuchte Deferred-Uber-Pipeline zu. In der Deferred-Uber-Pipeline ist durch den Single-Pass Umstand zudem der Zwang zu *Pre-Lit*-Verfahren, wie in Kapitel 4.2.2 aufgezeigt, festgestellt worden. Soll ein bestimmter NPR-Effekt also unbedingt selektiv als reiner Post-Lit-Effekt verarbeitet werden, um die Integrität eines Post-Processing Effektes zu gewährleisten (vgl. 4.3) muss auch innerhalb der Deferred-Paradigmen auf Multi-Pass-Verfahren zurückgegriffen werden.

Deferred Rendering in seinen zwei ausgeführten Formen verspricht, wie in Kapitel 3 erörtert, diverse Vorteile gegenüber klassischem Forward-Rendering und für beide untersuchte Pipelines konnte die Grundannahme der effizienteren Verarbeitung hoher Anzahlen von Lichtquellen bestätigt werden (vgl. Kapitel 5.7).

In Bezug auf selektive Stilisierung in Deferred Pipelines und Effizienz bei hoher Szeneentropie konnte zudem auf Performanceebene die Hypothese der Warp-Divergence (vgl. Kapitel 2.2.3) für die Deferred-Uber-Pipeline in Versuch 5.5.1 bestätigt werden, auch wenn eine eindeutige Zuweisung des Phänomens auf Hardwareebene durch den empirischen Aufbau der Versuche nicht endgültig verifiziert werden kann (etwa welchen anteiligen Einfluss Registerdruck hier hat). Obwohl der Deferred-Multi-Pass-Ansatz der Deferred-Volume-Pipeline in diesem Kontext keine Schwächen gezeigt hat, konnten in den Versuchen 5.3.2 und 5.4.2 die durch Blending in die Höhe getriebenen RMW-Zyklen eindeutig identifiziert werden (vgl. Kapitel 2.2.2). Die Deferred-Multi-Pass-Ansätze zeigen zudem, wie in Kapitel 4.2.3 dargelegt, den weiteren Nachteil der erhöhten VRAM-Belastung durch größere G-Buffer-Formate, um Artefakte wie Banding (siehe Abbildung 4.4) zu vermeiden.

Es ist anzumerken, dass viele der Benannten Bottlenecks und Drawbacks der einzelnen Paradigmen, wie in Kapitel 3.2 und 3.1 dargelegt, zumindest teilweise durch moderne Verfahren und hybride Ansätze kompensiert werden können. Verfahren wie beispielsweise *Clustered Forward* [18] oder *Forward+* [5] erweitern die Basisparadigmen hierfür grundlegend. Moderne Entwicklungen aus der Spieleindustrie zeigen hingegen die Idee auf, hybride Verfahren mit einer Kombination aus Forward und Deferred Ansätzen für den Ausgleich der jeweiligen Schwächen einer Rendering-Architektur zu nutzen [7], [10], [23].

6.2 Heuristisches Entscheidungsmodell

Dieses Kapitel konsolidiert die bisherigen Erkenntnisse über die Basis-Paradigmen der untersuchten Rendering-Pipelines in einem heuristischen Entscheidungsmodell. Wie in Kapitel 3 dargelegt, ist die Wahl einer Rendering-Architektur in der Praxis stark vom angestrebten Zielbild und dessen spezifischen Anforderungen abhängig. Die nachfolgende Auswertung differenziert zwischen deterministischen Ausschlusskriterien (**Hard Limits**), welche durch architektonische oder API-bedingte Restriktionen definiert sind (vgl. Kapitel 4), und empirischen Skalierungsheuristiken (**Soft Limits**), die Restriktionen und Kippunkte auf Hardwareebene beschreiben (vgl. Kapitel 5). Die Synthese dieser Ergebnisse wird in Abbildung 6.1 visualisiert.

Deterministische Ausschlusskriterien (Hard Limits) Im Folgenden werden die architektonischen und Grafik-API-bedingten Restriktionen evaluiert, die eine Pipeline-Wahl deterministisch ausschließen:

Selektive Stilisierung von Bildraum-Effekten

Durch die inhärente Limitierung von Forward-Rendering-Paradigmen Geometrien ausschließlich auf Objekten (Object-Space), ist Forward-Rendering in seiner Grundform ungeeignet, um Bildraumeffekte (Screen-Space) wie Kuwahara-Filter oder Sobel-Outlines selektiv darzustellen (vgl. 4.2.1). Es muss auf Deferred-Paradigmen oder visuell äquivalente Effekte (vgl. Abb. 4.2) zurückgegriffen werden.

Licht-Quantität (1)

Wird die Anzahl der aktiven Lichtquellen in einer Szene erwartet größer als das von der Grafik-API zugrunde gelegte Limit für Uniform-Buffer (etwa 512 in OpenGL), so ist der Einsatz einer Multi-Pass-Pipeline mit aufgeteilten Lichtberechnungspässen erforderlich (vgl. 4.2.1). Simple Single-Pass Architekturen wie eine naive Forward-Rendering-Pipeline oder Deferred-Pipelines mit einem einzigen monolithischen Uber-Shader zur Lichtberechnung, sind hier nicht hinreichend (vgl. Abb. 5.7).

NPR-Integrität (Pre-Lit vs Post-Lit)

Bei der Anwendung von kernelbasierten Effekten, wie beispielsweise dem Kuwahara-Filter, in selektiven Deferred-Pipelines erweisen sich Single-Pass-Deferred-Pipelines als unzureichend, sofern die vollständige Integrität des resultierenden Effekts gewahrt werden muss (vgl. 4.2.2). Aufgrund der signifikanten Zunahme des algorithmischen Aufwands, die Beleuchtungsberechnungen der notwendigen Umgebungspixel für jeden bearbeiteten Bildpunkt durchzuführen, ist es erforderlich die Lichteinwirkung bereits vor der Anwendung des eigentlichen NPR-Effekts in einem vorgelagerten Pass zu berechnen (vgl. 4.2.3).

Empirische Skalierungsheuristiken (Soft Limits) Im Folgenden werden die graduellen Performance-Kipppunkte auf Hardwareebene erläutert, ab denen die Performance bei zunehmender Szenenkomplexität unverhältnismäßig stark sinkt.

Licht-Quantität (2)

Forward-Rendering ist hinreichend geeignet Szenen mit einer niedrigen Anzahl an Lichtquellen darzustellen, in Umgebungen mit einer signifikanten Anzahl von Lichtquellen ist von der Verwendung von Forward-Paradigmen zur Lichtauswertung allerdings abzusehen (vgl. Abb. 5.7). Der Forward-inhärente quadratische Aufwand von $O(N_{\text{obj}} \cdot N_{\text{light}})$ ist dem linearen Aufwand von $O(N_{\text{obj}} + N_{\text{light}})$ in Deferred signifikant unterlegen (vgl. 2.1.1 u. 2.1.2).

Monolithische Shader-Strukturen

Wenn Szenen mit einer extrem hohen Stil- oder Material-Entropie vorliegen, ist bei der Verwendung von monolithischen Shader-Strukturen mit einer großen Anzahl von `if/else`- oder `switch`-Bedingungen, die sich auf nicht triviale Shading-Operationen beziehen, in der selektiven Stilisierung mit Einschränkungen zu rechnen (vgl. 5.5.1). Dies gilt insbesondere für Deferred-Rendering-Paradigmen, die eine höhere Tendenz zu solchen monolithischen Shader-Strukturen durch vereinheitlichte Shading-Pässe aufweisen (vgl. 2.1.2).

Light-Volumes

Bei der Verwendung von Light-Volumes zur Lichtberechnung, die im Deferred Rendering weit verbreitet sind, ist in komplexen Szenen mit einer hohen Lichtdichte und der daraus resultierenden großen Menge an Überlappungen der Lichteinflüsse mit Performanceeinbußen zu rechnen (vgl. 5.3.2 u. 5.4.2). Die durch Additives Blending der Lichteinflüsse erhöhten RMW-Zyklen auf der GPU können sich als signifikanter Performance-Engpass herausstellen (vgl. 2.2.2). In diesem Zusammenhang muss weiterhin beachtet werden, dass das Deferred Rendering grundlegend höhere Speicherbelastungen aufweist, um Artefaktbildungen zu vermeiden und damit eine höhere Grundlast auf der GPU verursacht (vgl. 4.2.3). Dies könnte bei kleinen und sehr speichertechnisch begrenzten Systemen einen signifikanten Faktor darstellen (vgl. 5.3.2 u. 5.6).

Objekt-Quantität

In komplexen Szenen mit einer hohen Anzahl individueller Objekte weisen klassische Forward-Rendering-Pipelines eine erhöhte Last auf der Host-Seite (CPU) durch insgesamt höhere Dispatch-Zeiten auf (vgl. 5.2.2). In Umgebungen mit wenig Rechenkapazität auf CPU-Seite sind Deferred-Architekturen somit möglicherweise vorzuziehen, auch wenn dieser Effekt im Gesamtsystem eher marginal auffällt (vgl. Abb. 5.2).

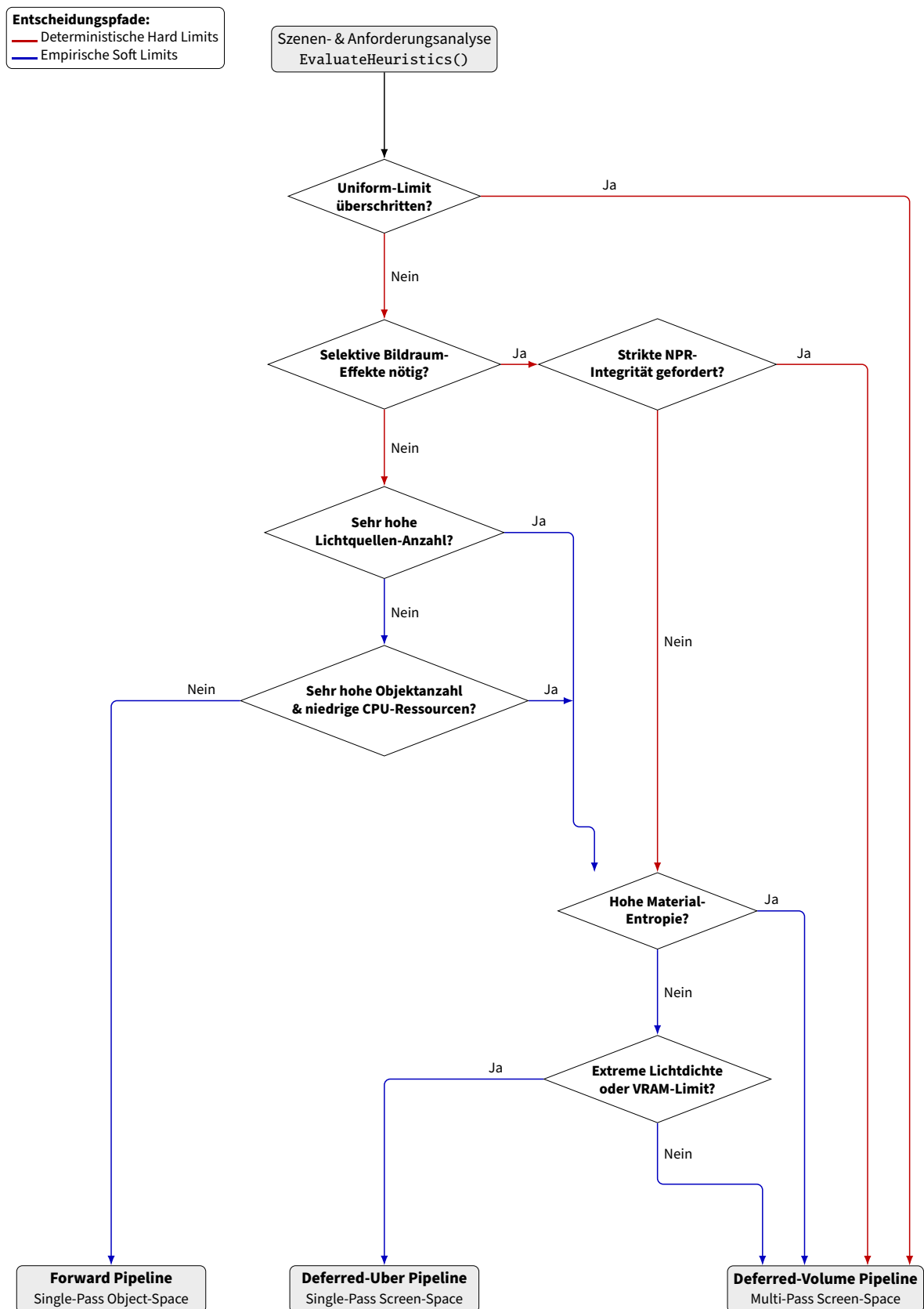


Abbildung 6.1: UML-Aktivitätsdiagramm des heuristischen Entscheidungsmodells zur Wahl der optimalen Basis-Rendering-Architektur. Eine farbliche Trennung verdeutlicht die Gewichtung zwischen zwingenden Architekturvorgaben (Rot) und Hardware-Skalierungsgrenzen (Blau).

6.3 Kritische Reflexion und Limitationen

In der nachfolgenden Analyse werden die Grenzen des erstellten Evaluations-Frameworks HyDra, die Lücken der empirischen Evaluationen sowie sonstige limitierende Faktoren und Entscheidungen der vorliegenden Arbeit reflektiert.

Eine Einschränkung des erstellten Frameworks ist im grundlegenden Umfang der implementierten Rendering-Verfahren zu verzeichnen. Die Beschränkung der Umgebung auf die gewählten drei Rendering-Architekturen ist im Sinne einer Evaluation der Basisparadigmen zwar vertretbar, in der Praxis sind jedoch, wie in Kapitel 3.2 aufgezeigt, wesentlich fragmentiertere Umgebungen vorhanden und die entsprechenden Basisparadigmen kommen kaum in ihrer Reinform vor. Was in der Praxis als „Standard“ oder „Basis“ moderner Rendering-Architekturen gilt, ist entsprechend nur schwer einzugrenzen und zu evaluieren. Dies zeigen mittlerweile weit verbreitete Ansätze wie *Forward+* [5], *Clustered*-Ansätze [18], *Tiled Shading* [35] oder die Nutzung von *Visibility-Buffern* in Deferred-Architekturen [15] als Erweiterungen der Basisparadigmen. Insbesondere im Hinblick auf die Forward-Rendering-Baseline besteht eine signifikante Lücke bezüglich der Multi-Pass-Variante des Renderingverfahrens.

Bestimmte grundlegende Materialeigenschaften, wie etwa Transluzenzen oder Spiegelungen, stellen ebenfalls gängige und grundlegende Anforderungen an Echtzeiterendering-Systeme dar [23, Abschn. *Deferred G-buffer*], [10, Abschn. *G-Buffer: Our approach*], [3, S. 887]. In der vorliegenden Untersuchung werden sie jedoch nicht berücksichtigt. Dies erschwert eine ganzheitliche Evaluation und Ableitung für diverse Praxisfälle.

In der Industriepraxis gehören darüber hinaus verschiedene Typen von Lichtquellen, wie Richtungs- (Directional) oder Flächenlichter (Area Lights), und die damit verbundenen Anforderungen, insbesondere das *Shadow Mapping*, zum Standardrepertoire einer Szene in der Spieleentwicklung [6, Abschn. 2.4.1], [14, Kap. 11.1.3.3]. Das HyDra-Framework weist hier ebenfalls Lücken durch die reine Verwendung von dynamisch generierten Punktlichtquellen auf.

Innerhalb der angewandten Evaluationsmetriken und des grundlegenden Ansatzes der empirischen Untersuchung können ebenfalls signifikante Limitationen der implementierten Software sowie der daraus resultierenden Untersuchungen festgestellt werden. Im Rahmen der Analyse spezifischer, hardwareinterner Phänomene und Flaschenhälse erweisen sich empirische Vorgehensweisen, die sich ausschließlich auf die Messung von Verarbeitungszeiten konzentrieren, häufig als unzureichend. Ein umfassendes Bild über die Ursachen und Limitierungen der betreffenden Phänomene und Hardware-Kippunkte ist so nur schwer zu gewinnen. Dies zeigen vor allem die Versuche 5.3.2 und 5.5.1. Zwar ermöglichen rein empirische Messungen eine Untersuchung des grundlegenden Verhaltens einer Rendering-Pipeline, sie sind jedoch nicht dazu geeignet, entsprechende detaillierte Handlungsempfehlungen bezüglich Hardwareoptimierungen abzuleiten. Das detaillierte

Auslesen von hardwarinternen Daten müsste durch die zusätzliche Integration von spezifischen Hardware-Profilern wie *NVIDIA Nsight* oder der *Radeon Developer Tool Suite (RDTs)* erfolgen [3, Kap. 18.1].

Über die strukturellen Lücken der vorliegenden Software hinaus ist anzumerken, dass die empirischen Evaluationen aus Kapitel 5 in ihrem Geltungsbereich an die zugrunde liegende Hardware (vgl. A.1) gebunden sind. Um beispielsweise die Hardware-Kippunkte auf mobilen Geräten genauer bestimmen zu können, müsste die zugrundeliegende Hardware ebenfalls als dynamischer Vektor dargestellt werden, um eine allgemeingültigere Evaluation zu ermöglichen.

6.4 Lessons Learned

Im Entwicklungsverlauf ist insbesondere die Verwendung von *OpenGL* [36] sowie vor allem der Bibliothek *raylib* [24], [25] zur Abstraktion der OpenGL-API als positiv zu bewerten. Ein signifikanter Anteil des rudimentären OpenGL-Overheads konnte durch die Verwendung von *raylib* vermieden werden. Für die granulare Manipulation diverser Teile des Frameworks, wie etwa der direkten Manipulation der G-Buffer, musste allerdings auch innerhalb von *raylib* auf Low-Level-Boilerplate-Code zurückgegriffen werden (*r1gl* [26]). Die Granularität der Software stellte sich in der Analyse als signifikante Herausforderung heraus, da die reine Menge an Boilerplate-Code zur Ausführung der Rendering-Pipelines einen wesentlichen Beitrag zur Komplexität und damit zur Fehleranfälligkeit der Software leistet. Zukünftige Erweiterungen und Anpassungen der Software sollten daher streng modular ablaufen, um monolithische Codestrukturen, vor allem hinsichtlich der Grafik-API-Instruktionen und damit assoziierten C++-Code, zu vermeiden.

Im Rahmen diverser Implementationsschritte war, durch diesen granularen Ansatz und die daraus resultierende Code-Komplexität, teilweise eine vollständige Neuentwicklung großer Teile des Codes erforderlich. Der Wechsel von einer statischen, hardcodierten Umgebung der Rendering-Pipelines zu einem für die Hardware-Benchmarks notwendigen Input-getriebenen Design stellte einen extremen Paradigmenwechsel dar. Für alle potenziellen zukünftigen Untersuchungsvektoren muss demnach im Voraus eine Mindestflexibilität des Frameworks eingehalten werden.

Für die Erzeugung belastbarer Ergebnisse der durchgeführten Evaluationen stellte die genaue Definition der Untersuchungsvektoren und des daraus resultierenden Szeneaufbaus eine Herausforderung dar. Auch in ihren grundlegendsten Formen existieren für die untersuchten Rendering-Architekturen eine extrem schwer eingrenzbare Anzahl an Eigenschaften. Die wichtigsten Eigenschaften innerhalb des extrem weit gefassten Themas des Echtzeit-Rendering zu isolieren, ohne sich in irrelevante Methodiken oder unnötige Optimierungen zu verlieren, erforderte eine iterative und zeitintensive Abwägung. Auf

Implementationsebene manifestierte sich dies insbesondere in der Erzeugung einer belastbaren Szene. Es ist essenziell, dass die Szene selbst nicht zu synthetisch aufgebaut ist, da dies die Übertragbarkeit auf reale Szenarien in der Industriepraxis einschränkt. Gleichzeitig ist darauf zu achten, dass die Komplexität der Szene nicht zu hoch ist, um eine Kontrolle über die untersuchten Variablen zu gewährleisten. Es ist festzustellen, dass sich Metriken generell nur schwer isolieren lassen. Eine Änderung der Beleuchtung beeinflusst etwa unweigerlich die Speicherbandbreite. Die strikte Isolation und eindimensionale Gestaltung von Testvektoren innerhalb kompletter Rendering-Architekturen kann daher weitestgehend als methodischer Trugschluss betrachtet werden.

Die entwickelten Benchmark- und Telemetrie-Module sind besonders bezüglich ihres resultierenden Aufwands hervorzuheben. Der Entwicklungsaufwand eines deterministischen und belastbaren Messsystems (vgl. 4.4) mit sauberen Hardwareschleifen (Ring-Buffer-Verfahren) hat sich, neben der Implementation der Rendering- und Shading-Verfahren, als eine der größten Herausforderungen herausgestellt. In Zukunft ist von einer Entwicklung der Messgrundlagen im Nachgang der Kernentwicklungen abzusehen. Telemetrie und Benchmark-Module sollten als integraler Bestandteil der Basisarchitektur konzipiert werden, nicht als nachgelagerte Evaluationsebene.

In Anlehnung an die schwierige Abgrenzung auf Implementationsebene wurde auch bei der Definition der genauen Testszenarien in Kapitel 5 die Frage sinnvoller Rahmenbedingungen aufgeworfen. Aufgrund des Umfangs der untersuchten Thematik erweist sich die Entwicklung eines Designs für belastbare Ergebnisse von Testszenarien und -variablen als anspruchsvoll. Die darauffolgende Interpretation der Ergebnisse auf rein empirischer Basis erforderte zudem eine weitreichende und zeitintensive Deduktion anhand der vorliegenden Literatur. Ohne tiefgreifende Profiler (bspw. *NVIDIA Nsight*) bleibt die Interpretation von Frame-Times und Bandbreiten stark deduktiv und spekulativ. Dies sollte in zukünftigen Erweiterungen der Software durch die Integration jener Profiler vermieden werden.

7 Zusammenfassung und Ausblick

Im Folgenden werden die in der vorliegenden Ausarbeitung gewonnenen Erkenntnisse in einem abschließenden Fazit unter Rückbesinnung auf die Kern-Forschungsfragen zusammengefasst. Es erfolgt die Ableitung eines Resümees bezüglich der Leistungsfähigkeit der untersuchten Rendering-Paradigmen im Kontext der selektiven Stilisierung. Auf dieser Grundlage werden konkrete Anforderungen für zukünftige Forschungs- und Entwicklungsarbeiten abgeleitet, insbesondere in Bezug auf das entwickelte Softwareartefakt (HyDra-Framework).

7.1 Fazit der Arbeit

In der vorliegenden Untersuchung wurde der fundamentale, architektonische Konflikt zwischen der Skalierbarkeit komplexer Echtzeit-Rendering-Szenen unter hoher Stil-Entropie für selektives Non-Photorealistic Rendering (NPR) adressiert. Das eigens entwickelte Evaluationsframework (HyDra) wurde als isolierte Testumgebung zur Analyse der untersuchten Basis-Rendering-Paradigmen verwendet. Die Zusammenfassungen orientieren sich an den in Kapitel 6 dargelegten Ergebnissen.

Das methodische Vorgehen basierte auf der Durchführung systematischer, empirischer Tests der Rendering-Paradigmen unter kontrollierten Bedingungen. Im Rahmen der Untersuchung wurden die geometrische Dichte, die Beleuchtungskomplexität, Auflösungen, Bandbreiten und die räumliche Stil-Entropie quantifiziert.

Die empirischen Messungen identifizierten spezifische Hardware-Bottlenecks für jede der untersuchten Rendering-Pipelines. Die Single-Pass-Forward-Pipeline wies vor allem bei ansteigender Lichtkomplexität, durch ALU-Last bedingt, Schwächen auf. Die Single-Pass-Deferred-Over-Pipeline zeigte bei hoher räumlicher Stil-Entropie signifikante Leistungseinbrüche auf, die durch Warp-Divergenz und Registerdruck bedingt waren. Die Deferred-Volume-Pipeline wies bei extrem überlappenden Lichtquellen eine fundamentale Bandbreitenlimitierung durch erhöhte RMW-Zyklen auf.

Auf architektonischer Ebene zeigte die Single-Pass-Forward-Pipeline zudem die signifikante Einschränkung, Bildraumeffekte nicht selektiv darstellen zu können. Die Single-Pass-Deferred-Über-Pipeline wies zudem Schwächen in der Lichtberechnung auf, was zu Pre-Light Zwängen in der Verarbeitung bestimmter Kernel-basierter NPR-Effekte geführt

hat. Insgesamt erwiesen sich Single-Pass Architekturen als unzureichend, um extrem hohe Lichtmengen darzustellen.

Die Synthese dieser Ergebnisse lässt die zentrale Forschungsfrage dahingehend beantworten, dass im Kontext der selektiven Stilisierung keine universell überlegene Basis-Rendering-Architektur existiert. Die Leistungsfähigkeiten wurde dynamisch und ausschließlich anhand der dominierenden Szenenlast determiniert. Aus den gewonnenen Erkenntnissen resultierte in Kapitel 6.2 ein heuristisches Entscheidungsmodell, welches Entwicklern ein konkretes Werkzeug zur objektiven, frühzeitigen Architekturbewertung anhand deterministischer Hard Limits und empirischer Soft Limits bereitstellt. Das entwickelte HyDra-Framework stellt Entwicklern zudem eine empirische und gekapselte Testumgebung für die untersuchten Rendering-Architekturen, frei von proprietärem Engine-Overhead, zur Verfügung.

7.2 Ausblick und zukünftige Arbeiten

Abschließend werden mögliche Erweiterungen der entwickelten Software vorgestellt und auf Forschungsebene relevante Erweiterungsmetriken anhand der in Kapitel 6.3 dargestellten Limitationen der Software, der Methodik und Untersuchungsvektoren vorgestellt.

Die in der Methodik identifizierten Schwächen der reinen Zeitmessung machten für Folgestudien eine technologische Weiterentwicklung der Telemetrie-Erfassung erforderlich. Wenn zwingend die genauen Auswirkungen von Hardware-Phänomenen wie beispielsweise Registerdruck, Cache-Misses oder exakter Speichersättigung evaluiert werden müssen, ist die Integration systemnaher Hardware-Profiler wie *NVIDIA Nsight* oder der *Radeon Developer Tool Suite (RDTs)* in künftigen Studien notwendig.

Im Rahmen der architektonischen Weiterentwicklung des HyDra-Frameworks sind Ergänzungen weiterer Rendering-Pipelines von zentralem Interesse. Neben einer simplen Multi-Pass-Forward-Pipeline sind hier etwa Forward+ [5], Clustered-Forward/Deferred [18], Tiled Forward/Deferred[35] oder Deferred-Pipelines mit Visibility-Buffern [15] mögliche relevante weitere Pipelines.

In diesem Kontext sind insbesondere die Implementierung unterschiedlicher Lichtquellen (Richtungs- und Flächenlichter) und das damit einhergehende Shadow-Mapping sowie erweiterte Materialeigenschaften (Spiegelungen und Transluzenzen) von primärer Relevanz.

Des Weiteren besteht die Möglichkeit, neben der stilistischen Entropie durch komplexere Materialeigenschaften auch zusätzliche Studien zu Anti-Aliasing aus den untersuchten Rendering-Paradigmen durchzuführen. Dies ist besonders im Hinblick auf die selektive Stilisierung durch dadurch verursachte harte Materialkanten von Interesse. Anti-Aliasing

kann somit als weitere semantische Evaluationsgrundlage der Rendering-Pipelines herangezogen werden.

Abschließend ist auch eine Portierung der Evaluationssoftware auf unterschiedliche Systeme und Hardwarebedingungen von Interesse, da der Kern der Evaluationen in Form von Hardwarekipppunkten unter anderen Hardwarearchitekturen grundlegend anders ausfallen kann. Dadurch wird es möglich, spezifische Hardware-Phänomene präziser zu isolieren und zu untersuchen.

Literatur

- [1] M. Magdics, C. Sauvaget, R. J. García und M. Sbert, „Post-Processing NPR Effects for Video Games,“ in *Proceedings of the 12th ACM SIGGRAPH International Conference on Virtual-Reality Continuum and Its Applications in Industry*, Ser. VRCAI '13, New York, NY, USA: Association for Computing Machinery, 2013, S. 147–156. doi: 10.1145/2534329.2534348.
- [2] Gearbox Software. „Borderlands 4 Screenshots | RPGFan,“ RPGFan. (2026), Adresse: <https://www.rpgfan.com/gallery/borderlands-4-screenshots/> (besucht am 13.07.2026).
- [3] T. Akenine-Möller, E. Haines, N. Hoffman, A. Pesce, M. Iwanicki und S. Hillaire, *Real-Time Rendering* (A Balkema Book), Fourth edition, first issued in hardback. Boca Raton, London, New York: CRC Press, 2019. doi: 10.1201/b22086.
- [4] A. Gooch, B. Gooch, P. Shirley und E. Cohen, „A Non-Photorealistic Lighting Model for Automatic Technical Illustration,“ in *Proceedings of the 25th Annual Conference on Computer Graphics and Interactive Techniques*, Ser. SIGGRAPH '98, New York, NY, USA: Association for Computing Machinery, 24. Juli 1998, S. 447–452. doi: 10.1145/280814.280950.
- [5] T. Harada, J. McKee und J. C. Yang, „Forward+: Bringing Deferred Lighting to the Next Level,“ in *Eurographics 2012 - Short Papers*, The Eurographics Association, 2012. doi: 10.2312/CONF/EG2012/SHORT/005-008.
- [6] F. Policarpo und F. Fonseca, *Deferred Shading Tutorial*, 2005. Adresse: <https://www.gamedevs.org/uploads/deferred-shading-tutorial.pdf> (besucht am 05.06.2026).
- [7] K. Tanaka und T. Komada, „Toon Rendering in Hi-Fi RUSH,“ präsentiert auf der Game Developers Conference, 2024. Adresse: https://media.gdcvault.com/gdc2024/Slides/GDC+slide+presentations/Tanaka_Kosuke_3D_Toon_Rendering.pdf (besucht am 06.06.2026).
- [8] K. Fatahalian, S. Boulos, J. Hegarty et al., „Reducing Shading on GPUs Using Quad-Fragment Merging,“ in *ACM SIGGRAPH 2010 Papers*, Ser. SIGGRAPH '10, New York, NY, USA: Association for Computing Machinery, 2010, S. 1–8. doi: 10.1145/1833349.1778804.

- [9] C. Schied und C. Dachsbacher, „Deferred Attribute Interpolation for Memory-Efficient Deferred Shading,“ in *Proceedings of the 7th Conference on High-Performance Graphics*, Ser. HPG '15, New York, NY, USA: Association for Computing Machinery, 2015, S. 43–49. doi: 10.1145/2790060.2790066.
- [10] M. Valient, „Deferred Rendering in Killzone 2,“ präsentiert auf der Guerrilla Develop Conference, 2007. Adresse: https://www.guerrilla-games.com/media/News/Files/Develop07_Valient_DeferredRenderingInKillzone2.pdf (besucht am 06. 06. 2026).
- [11] J. Thaler, „Deferred Rendering,“ in *Seminar in Computer Graphics*, Vienna, Austria: TU Wien (Vienna University of Technology), 2011. Adresse: https://www.researchgate.net/profile/Jonathan_Thaler2/publication/323357208_Deferred_Rendering/links/5a8fce31aca272140560aaad/Deferred-Rendering.pdf (besucht am 31. 05. 2026).
- [12] I. Mallett und C. Yuksel, „Deferred Adaptive Compute Shading,“ in *Proceedings of the Conference on High-Performance Graphics*, Ser. HPG '18, New York, NY, USA: Association for Computing Machinery, 2018, S. 1–4. doi: 10.1145/3231578.3232160.
- [13] A. Lauritzen, „Deferred Rendering for Current and Future Rendering Pipelines,“ in *ACM SIGGRAPH 2010 Courses*, (Los Angeles, CA), ACM. Adresse: <https://www.intel.cn/content/dam/develop/external/us/en/documents/lauritzen-deferred-shading-siggraph-2010-181241.pdf> (besucht am 07. 03. 2026).
- [14] J. Gregory, *Game Engine Architecture*, 3. Aufl. New York: A K Peters/CRC Press, 2018. doi: 10.1201/9781315267845.
- [15] C. A. Burns und W. A. Hunt, „The Visibility Buffer: A Cache-Friendly Approach to Deferred Shading,“ *Journal of Computer Graphics Techniques*, Jg. 2, Nr. 2, 2013. Adresse: <https://www.jcgt.org/published/0002/02/04/paper.pdf> (besucht am 31. 05. 2026).
- [16] A. Kaplanyan, „CryEngine 3: Reaching the Speed of Light,“ präsentiert auf der SIGGRAPH (Advanced Real-Time Rendering Course), 2010. Adresse: <https://advances.realtimerendering.com/s2010/Kaplanyan-CryEngine3%28SIGGRAPH%202010%20Advanced%20RealTime%20Rendering%20Course%29.pdf> (besucht am 06. 06. 2026).
- [17] C. Kubisch. „Pixel Jet Stream: Life of a Triangle - NVIDIA's Logical Pipeline,“ Life of a triangle - NVIDIA's logical pipeline. (2015), Adresse: <https://pixeljetstream.blogspot.com/2015/02/life-of-triangle-nvidias-logical.html> (besucht am 21. 06. 2026).
- [18] O. Olsson, M. Billeter und U. Assarsson, „Clustered Deferred and Forward Shading,“ in *High-Performance Graphics Symposium*, 2012. doi: 10.2312/EGGH/HPG12/087-096.

- [19] B. Karis, R. Stubbe und G. Wihlidal, „A Deep Dive into Nanite Virtualized Geometry,“ präsentiert auf der ACM SIGGRAPH 2021 Courses (Virtual Event), 2021. Adresse: https://advances.realtimerendering.com/s2021/Karis_Nanite_SIGGRAPH_Advances_2021_final.pdf (besucht am 05. 06. 2026).
- [20] B. Karis, „High Quality Temporal Supersampling,“ präsentiert auf der Advances in Real-Time Rendering in 3D Graphics and Games, ACM SIGGRAPH 2014 Courses, 2014. Adresse: https://advances.realtimerendering.com/s2014/#_HIGH-QUALITY_TEMPORAL_SUPERSAMPLING (besucht am 05. 06. 2026).
- [21] R. El Garawany, „Deferred Lighting in Uncharted 4,“ präsentiert auf der SIGGRAPH (Advances in Real-Time Rendering in Games Course) (Anaheim, CA), 2016. Adresse: <https://advances.realtimerendering.com/s2016/> (besucht am 06. 06. 2026).
- [22] T. Sousa und J. Geffroy, „The Devil Is in the Details: idTech 6.66,“ präsentiert auf der SIGGRAPH (Advances in Real-Time Rendering in Games Course) (Anaheim, CA), 2016. Adresse: https://advances.realtimerendering.com/s2016/Siggraph2016_idTech6.pdf (besucht am 06. 06. 2026).
- [23] Muhammad. „Behind the Pretty Frames: PRAGMATA.“ (2026), Adresse: <https://mamoniem.com/behind-the-pretty-frames-pragmata/> (besucht am 06. 06. 2026).
- [24] R. Santamaría. „Raylib,“ raylib. (2026), Adresse: <https://www.raylib.com> (besucht am 17. 07. 2026).
- [25] R. Santamaría. „Raylib - examples,“ raylib. (2026), Adresse: <https://www.raylib.com> (besucht am 17. 07. 2026).
- [26] R. Santamaría, *rlgl.h*, Version 6.0, 2026. Adresse: <https://github.com/raysan5/raylib/blob/master/src/rlgl.h> (besucht am 17. 07. 2026).
- [27] C. Lux, „Chapter 36: Asynchronous Queries in OpenGL,“ in *OpenGL Insights*, P. Cozzi und C. Riccio, Hrsg., Boca Raton, FL: CRC Press, 2012, ISBN: 978-1-4398-9376-0.
- [28] M. Segal und K. Akeley, „The OpenGL Graphics System: A Specification (Version 3.3 (Core Profile)),“ The Khronos Group, 2010. Adresse: <https://registry.khronos.org/OpenGL/specs/gl/glspec33.core.pdf> (besucht am 05. 06. 2026).
- [29] J. E. Kyprianidis, H. Kang und J. Döllner, „Image and Video Abstraction by Anisotropic Kuwahara Filtering,“ *Computer Graphics Forum*, Jg. 28, Nr. 7, S. 1955–1963, 2009, ISSN: 1467-8659. doi: 10.1111/j.1467-8659.2009.01574.x.
- [30] G. Papari, N. Petkov und P. Campisi, „Artistic Edge and Corner Enhancing Smoothing,“ *IEEE Transactions on Image Processing*, Jg. 16, Nr. 10, S. 2449–2462, Okt. 2007, ISSN: 1941-0042. doi: 10.1109/TIP.2007.903912.
- [31] A. Lake, C. Marshall, M. Harris und M. Blackstein, „Stylized Rendering Techniques for Scalable Real-Time 3D Animation,“ in *Proceedings of the 1st International Symposium on Non-photorealistic Animation and Rendering*, Ser. NPAR '00, New York, NY, USA: Association for Computing Machinery, 5. Juni 2000, S. 13–20. doi: 10.1145/340916.340918.

- [32] T. Saito und T. Takahashi, „Comprehensible Rendering of 3-D Shapes,“ in *Proceedings of the 17th Annual Conference on Computer Graphics and Interactive Techniques*, Ser. SIGGRAPH '90, New York, NY, USA: Association for Computing Machinery, 1990, S. 197–206. doi: 10.1145/97879.97901.
- [33] NVIDIA Corporation, „NVIDIA Turing GPU Architecture Whitepaper,“ NVIDIA, 2018. Adresse: <https://images.nvidia.com/aem-dam/en-zz/Solutions/design-visualization/technologies/turing-architecture/NVIDIA-Turing-Architecture-Whitepaper.pdf> (besucht am 21. 06. 2026).
- [34] O. Shishkovtsov, „Deferred Shading in S.T.A.L.K.E.R.,“ in *GPU Gems 2: Programming Techniques for High-Performance Graphics and General-Purpose Computation*, M. Pharr, Hrsg., Boston, MA: Addison-Wesley Professional, 2005, S. 143–166, ISBN: 0-321-33559-7. Adresse: <https://developer.nvidia.com/gpugems/gpugems2/part-ii-shading-lighting-and-shadows/chapter-9-deferred-shading-stalker> (besucht am 26. 06. 2026).
- [35] O. Olsson, View Profile, M. Billeter, View Profile, U. Assarsson und View Profile, „Tiled and Clustered Forward Shading,“ in *ACM SIGGRAPH 2012 Talks*, Ser. ACM Conferences, 5. Aug. 2012, S. 1–1. doi: 10.1145/2343045.2343095.
- [36] The Khronos Group. „Khronos OpenGL® Registry - The Khronos Group Inc.“ (2026), Adresse: https://registry.khronos.org/OpenGL/index_gl.php (besucht am 17. 07. 2026).

A Systemkonfiguration und Variablen-Mapping

Tabelle A.1: Spezifikation der primären Test-Hardware.

Hardware-Komponente	Spezifikation / Wert
Prozessor (CPU)	Intel Core i5-6600K @ 4 × 3.50 GHz
Hauptspeicher (RAM)	16 GB DDR4 (Dual-Channel) @ 3000 MT/s
Grafikkarte (GPU)	NVIDIA GeForce GTX 1660
GPU-Architektur	Turing (TU116)
VRAM-Kapazität	6 GB GDDR5
Theoretische VRAM-Bandbreite	192,1 GB/s
Raster Operation Pipelines (ROPs)	48

Tabelle A.2: Spezifikation der Software-Umgebung und API-Schnittstellen.

Software / API	Version
Betriebssystem	Linux Mint 22.3 (Kernel 6.8.0)
Grafiktreiber	NVIDIA 595.71.05
C++ Compiler	GCC 13.3.0
Anwendungs-Framework	raylib 5.5
Grafik-API	OpenGL 3.3.0 Core Profile
Shading Language	GLSL 3.30

Tabelle A.3: Spezifikation der unabhängigen Variablen, deren formale Definition und Zuweisung zu den Test-Suites.

Variable (Symbol & CSV-Mapping)	Wertebereich	Kapitel	Beschreibung
Render-Architektur (A) activeRenderPath	<i>Enum</i>	Alle	Die primäre architektonische Variable. Rotiert durch Forward, Deferred-Uber und Deferred-Volume.
Geometrischer Detailgrad (LOD) currentLodIndex	12 bis 8.192 Dreiecke	5.2.1	Skaliert die Dreiecksanzahl pro zu renderndem Objekt.
Instanzen-Anzahl (N_{Inst}) activeObstacleCount	1.000 bis 50.000	5.2.2, 5.2.3	Anzahl der gerenderten Geometrie-Instanzen.
Framebuffer-Auflösung (R) renderResolution	480p bis 4K	5.3.1	Absolute Render-Auflösung.
Screen-Space-Abdeckung (C_{Floor}) renderFloor	<i>Boolean</i>	5.3.2	Erzeugt eine Bildfüllende Geometrie.
Lichtquellen-Anzahl (N_{Light}) activeLightCount	10 bis 5.000	5.4.1, 5.4.2	Anzahl der aktiven dynamischen Punktlichter im View-Frustum.
Beleuchtungs-Radius (r_{Light}) lightIntensity	0,5 bis 20,0	5.4.2	Skaliert das Proxy-Volumen der Lichter.
Räumliche Entropie ($E_{Spatial}$) useClusteredStyles	<i>Boolean</i>	5.5.1	Steuert die Verteilung der Stile (geklustert vs. gestreut).
Stil-Kombinatorik (S_{Count}) enableGooch/Toon	1 bis 3 Stile	5.5.2	Skaliert die Anzahl der aktiv evaluierten Shading-Modelle.
Filter-Radius ($r_{Kuwahara}$) kuwaharaRadius	2 bis 16	5.5.3	Ausdehnung des Post-Processing-Kernels.
Makro-Pass (P_{Macro}) Feature Accumulation	Pass 1 bis 4	5.6	Kumulative Aktivierung von Geometrie, Shading und Stil-Entropie über einen fixierten Kamerapfad.

Tabelle A.4: Spezifikation der abhängigen Variablen (Metriken), deren formale Definition und CSV-Mapping.

Metrik (Symbol & CSV-Mapping)	Kategorie	Beschreibung
Logik-Laufzeit (t_{Logic}) cpuLogicMs	CPU	Dauer des logischen Szenen-Passes (Frustum Culling, Sichtbarkeitsprüfung und Matrix-Updates).
Dispatch-Laufzeit (t_{Dispatch}) cpuRenderMs	CPU	Dauer der API-Befehlsübermittlung an den Grafiktreiber (<i>Draw-Call Submission</i>).
Geometrie-Laufzeit (t_{Geom}) geomMs	GPU	Dauer des asynchronen Hardware-Queries für den Geometrie-Pass (G-Buffer-Generierung, entfällt bei Forward).
Beleuchtungs-Laufzeit (t_{Light}) lightMs	GPU	Dauer des asynchronen Hardware-Queries für Beleuchtung und NPR-Berechnung (bei Forward inkludiert dies zwangsläufig die Geometrielast).
GPU-Gesamtlaufzeit (t_{GPU}) totalGpuMs	GPU	Aggregierte GPU-Ausführungszeit des gesamten Frames, gemessen über Hardware-Timer-Queries.
System-Frame-Time (t_{Frame}) totalFrameTimeMs	System	Absolute Frame-Latenz (Delta Time), gemessen über CPU-High-Resolution-Timer (std: :chrono).
Sichtbare Instanzen (N_{Vis}) activeInstances	Szene	Anzahl der verarbeiteten Objekt-Instanzen nach dem Frustum Culling.
Gerenderte Dreiecke (N_{Tris}) activeTris	Szene	Berechnete Summe der an die GPU übermittelten Dreiecke ($N_{\text{Vis}} \times \text{Dreiecke des aktiven LODs}$).
Overdraw-Faktor (O_{Draw}) currentOverdraw	Szene	Theoretischer Überzeichnungsfaktor, basierend auf Frustum-Tiefe und räumlicher Instanzdichte.

Tabelle A.5: Spezifikation der Kontrollvariablen und der statischen System-Baseline.

Kategorie	Variable (Symbol & Mapping)	Standardwert (Baseline)
Rendering & Pipeline	HDR-Genauigkeit (HDR_{16bit}) use16BitHDR	Aktiviert (Basis für RMW-Blending)
	Basis-Auflösung (R_{Base}) renderResolution	1920 × 1080 Pixel (1080p)
	Screen-Space-Abdeckung (C_{Floor}) renderFloor	Aktiviert
Verteilung & Geometrie	Basis-Instanzen (N_{Inst_Base}) activeObstacleCount	5.000 Objekte
	Spawn-Radius (r_{Spawn}) objectSphereRadius	200,0 Einheiten
	Basis-Detailgrad (LOD_{Base}) currentLodIndex	1
	Räumliche Entropie ($E_{Spatial}$) useClusteredStyles	Deaktiviert (Gestreute Verteilung)
Licht-Physik	Umgebungslicht ($I_{Ambient}$) ambientLightStrength	1,0 (Neutrale Grundausleuchtung)
	Licht-Radius (r_{Light}) lightIntensity	2,0 Einheiten
	Licht-Singularität (S_{Light}) useLightSingularity	Deaktiviert (Zufällige Verteilung)
NPR / Stilisierung	Aktive Stile (S_{Count}) enableGooch/Toon	Deaktiviert (0 Stile, reine Basis-Beleuchtung)
	Post-Processing (P_{FX}) enableOutlines/Kuwahara	Deaktiviert
	Filter-Radius ($r_{Kuwahara}$) kuwaharaRadius	4 Pixel
	Filter-Intensität ($i_{Kuwahara}$) kuwaharaIntensity	4,0
Test-Orchestrierung	Warm-up-Dauer (T_{Warmup}) warmupDuration	10,0 Sekunden (Cache-Sättigung)
	Sampling-Dauer (T_{Sample}) captureDuration	1.000 Frames (Statistische Mittelwertbildung)

B Struktur des digitalen Anhangs

—